Київський столичний університет імені Бориса Грінченка

Володимир Соколов

ТЕХНОЛОГІЇ ПРОТИДІЇ ЗЛОЯКІСНОМУ ПРОГРАМНОМУ КОДУ

Навчальний посібник

для здобувачів другого (освітнього) рівня вищої освіти
спеціальності 125/F5 «Кібербезпека та захист інформації»
галузі знань 12/F «Інформаційні технології»

Київ — 2027

УДК 004.056:004.492.3(075.8)
ББК 32.973
С594

Препринт

Соколов В. Ю. Технології протидії злоякісному програмному коду : навчальний посібник для здобувачів другого (освітнього) рівня вищої освіти. Київ, КСУБГ, 2027, 188 с. — 17 розділів.

Навчальний посібник присвячено технологіям виявлення, аналізу та протидії злоякісному програмному коду. Систематично розглянуто понятєвий апарат і таксономію шкідливого програмного забезпечення, нормативно-правове й стандартизаційне забезпечення, теоретичні та математичні основи виявлення, моделювання загроз і ланцюги ураження, внутрішню будову та життєвий цикл шкідливого коду. Докладно висвітлено статичний і динамічний аналіз, сигнатурне та поведінкове виявлення, техніки ухилення й обфускації, застосування машинного навчання, захист кінцевих точок і телеметрію, інженерію виявлення, розвідку загроз і атрибуцію, протидію цілеспрямованим сталим загрозам (APT) та полювання за загрозами. Окремо розглянуто захист спеціалізованих середовищ (промислові системи, Інтернет речей, мобільні платформи, хмару), атаки на ланцюг постачання, реагування на інциденти й цифрову форензику, а також організаційне управління та стратегію кіберстійкості. Матеріал спирається на актуальні міжнародні стандарти й наукові джерела та враховує досвід протидії кіберзагрозам в Україні.

Кожен розділ завершується висновками та питаннями для самоперевірки, серед яких — дослідницькі завдання, придатні як відправна точка для наукової статті чи фрагмента дисертаційного дослідження. Посібник призначено для магістрантів галузі знань 12/F «Інформаційні технології» спеціальності 125/F5 «Кібербезпека та захист інформації», а також буде корисним науковим і науково-педагогічним працівникам, фахівцям із захисту інформації та кібербезпеки.

© Володимир Соколов, 2027
© Київський столичний університет імені Бориса Грінченка, 2027

Зміст

Перелік умовних скорочень **1**

Вступ **3**

1 Злоякісний програмний код як об'єкт протидії: поняття, таксономія та простір загроз **5**
1.1 Поняття та термінологія 5
1.2 Таксономія злоякісного програмного коду 6
1.3 Еволюція злоякісного коду 8
1.4 Життєвий цикл злоякісної кампанії 10
1.5 Модель порушника та мотивація 10
1.6 Простір загроз і поверхня атаки 11
1.7 Український контекст 12
1.8 Принципи протидії та структура курсу 13
Висновки до розділу 13
Питання для самоперевірки 14

2 Нормативно-правове та стандартизаційне забезпечення протидії злоякісному коду **17**
2.1 Структура нормативного поля 17
2.2 Кримінально-правова протидія 18
2.3 Законодавство України про кібербезпеку 19
2.4 Право Європейського Союзу 20
2.5 Стандарти управління та реагування 21
2.6 Технічні бази знань і специфікації 22
2.7 Оцінювання відповідності та обов'язки суб'єктів 23
2.8 Порівняння підходів України та ЄС 24
2.9 Кейс: правові вимоги для оператора критичної інфраструктури 24
Висновки до розділу 25
Питання для самоперевірки 25

3 Теоретичні та математичні основи протидії злоякісному коду **27**
3.1 Постановка задачі виявлення 27
3.2 Теоретико-обчислювальні межі виявлення 28
3.3 Теорія статистичного виявлення 29
3.4 Проблема базової частоти 30

3.5 Інформаційно-теоретичні міри 31
3.6 Ознаковий простір і межі навчання 32
3.7 Моделі поширення злоякісного коду 33
3.8 Метрики ефективності 34
3.9 Розгорнутий приклад оцінювання детектора 35
Висновки до розділу 35
Питання для самоперевірки 36

4 Моделювання загроз, ланцюги ураження та оцінювання ризиків злоякісного коду 39
4.1 Процес управління ризиками 39
4.2 Моделювання загроз через ланцюг ураження 40
4.3 Дерева атак 42
4.4 Кількісне оцінювання ризику 43
4.5 Пріоритизація заходів та економіка 44
4.6 Кейс: модель загроз організації 45
Висновки до розділу 46
Питання для самоперевірки 46

5 Внутрішня будова та життєвий цикл шкідливого коду 49
5.1 Формати виконуваних файлів 49
5.2 Завантаження та виконання 51
5.3 Техніки виконання й ін'єкції коду 52
5.4 Персистентність 53
5.5 Підвищення привілеїв 54
5.6 Приховування: руткіти 54
5.7 Модульна архітектура сучасного ЗПК 55
5.8 Кейс: аналіз будови типового імпланта 56
Висновки до розділу 56
Питання для самоперевірки 57

6 Статичний аналіз та сигнатурне виявлення 59
6.1 Місце й рівні статичного аналізу 59
6.2 Поверхневий статичний аналіз 60
6.3 Сигнатурне виявлення 61
6.4 Дизасемблювання та аналіз коду 63
6.5 Виявлення за подібністю та кластеризація 64
6.6 Евристичний статичний аналіз та його межі 65
6.7 Кейс: розроблення правила виявлення для сімейства 66
Висновки до розділу 66
Питання для самоперевірки 67

7 Динамічний та поведінковий аналіз шкідливого коду 69
7.1 Принципи й місце динамічного аналізу . . . 69
7.2 Пісочниці та середовища спостереження . . . 70
7.3 Спостережувані артефакти поведінки . . . 72
7.4 Аналіз пам'яті . . . 72
7.5 Формалізація та кластеризація поведінки . . . 73
7.6 Межі методу та протидія ухиленню . . . 74
7.7 Кейс: динамічний аналіз зразка-вимагача . . . 75
Висновки до розділу . . . 76
Питання для самоперевірки . . . 76

8 Техніки ухилення, обфускації та антианалізу 79
8.1 Таксономія технік ухилення . . . 79
8.2 Пакування та шифрування . . . 80
8.3 Обфускація коду . . . 82
8.4 Поліморфізм та метаморфізм . . . 83
8.5 Антианалізні прийоми . . . 84
8.6 Деобфускація та розпакування . . . 85
8.7 Кейс: підготовка обфускованого зразка до аналізу . . . 85
Висновки до розділу . . . 86
Питання для самоперевірки . . . 86

9 Машинне навчання та статистичне виявлення 89
9.1 Місце машинного навчання у виявленні . . . 89
9.2 Ознаки та подання зразків . . . 90
9.3 Типові моделі та їх властивості . . . 91
9.4 Оцінювання якості . . . 92
9.5 Змагальне машинне навчання . . . 93
9.6 Практичні обмеження й інтеграція . . . 94
9.7 Кейс: статичний класифікатор PE-файлів . . . 95
Висновки до розділу . . . 95
Питання для самоперевірки . . . 96

10 Захист кінцевих точок, моніторинг і телеметрія 97
10.1 Еволюція й архітектура захисту кінцевих точок . . . 97
10.2 Телеметрія й джерела подій . . . 99
10.3 Виявлення й реагування . . . 100
10.4 Профілактичні заходи й зниження поверхні атаки . . . 101
10.5 Обмеження й протидія ухиленню . . . 102
10.6 Кейс: виявлення безфайлової атаки за телеметрією . . . 103
Висновки до розділу . . . 104
Питання для самоперевірки . . . 104

11 Інженерія виявлення: правила, індикатори та безфайлові загрози 107
11.1 Правила виявлення як код 107
11.2 Піраміда болю та ієрархія індикаторів 108
11.3 Безфайлові загрози та living-off-the-land 110
11.4 Життєвий цикл і метрики виявлення 111
11.5 Покриття технік і баланс 112
11.6 Кейс: розроблення виявлення для техніки 112
Висновки до розділу 113
Питання для самоперевірки 113

12 Розвідка загроз, індикатори компрометації та атрибуція 115
12.1 Поняття й рівні розвідки загроз 115
12.2 Індикатори компрометації та їх життєвий цикл 116
12.3 Стандарти обміну розвідкою 117
12.4 Моделі аналізу й атрибуція 118
12.5 Розвідка загроз в Україні: CERT-UA 119
12.6 Кейс: від зразка до розвідки (WannaCry, NotPetya) 120
Висновки до розділу 120
Питання для самоперевірки 121

13 Цілеспрямовані сталі загрози та полювання за загрозами 123
13.1 Феномен APT та його життєвий цикл 123
13.2 Час присутності (dwell time) 124
13.3 Парадигма полювання за загрозами 125
13.4 Методологія й гіпотези 126
13.5 Бокове переміщення та living-off-the-land як об'єкти полювання 127
13.6 Зрілість і метрики полювання 127
13.7 Кейс: полювання за прихованим угрупованням 128
Висновки до розділу 129
Питання для самоперевірки 129

14 Протидія злоякісному коду у спеціалізованих середовищах 131
14.1 Спільні принципи й виклики 131
14.2 Промислові системи керування (ICS/SCADA) 132
14.3 Інтернет речей (IoT) 134
14.4 Мобільні платформи 135
14.5 Хмарні та контейнерні середовища 135
14.6 Кейс: захист промислового об'єкта критичної інфраструктури 136
Висновки до розділу 137
Питання для самоперевірки 137

15 Атаки на ланцюг постачання програмного забезпечення 139
15.1 Поняття й анатомія атаки 139
15.2 Типологія та кейси 140
15.3 Довіра, цілісність та походження 141
15.4 Прозорість: SBOM та інвентаризація 142
15.5 Стратегії протидії 143
15.6 Кейс: захист конвеєра розробки 144
Висновки до розділу 144
Питання для самоперевірки 145

16 Реагування на інциденти та цифрова форензика шкідливого коду 147
16.1 Життєвий цикл реагування на інцидент 147
16.2 Стримування, усунення, відновлення 148
16.3 Цифрова форензика: докази та їх цілісність 148
16.4 Аналіз інциденту ЗПК 150
16.5 Антифорензика та протидія їй 150
16.6 Нормативні й організаційні аспекти в Україні 151
16.7 Кейс: наскрізне реагування на зараження 151
Висновки до розділу 152
Питання для самоперевірки 153

17 Організаційне управління та стратегія протидії злоякісному коду 155
17.1 Управління безпекою й рамкові моделі 155
17.2 Управління ризиком та економіка безпеки 156
17.3 Чинник людини й культура безпеки 156
17.4 Стратегія кіберстійкості 157
17.5 Метрики, зрілість і національний вимір 158
17.6 Кейс: побудова програми протидії ЗПК 159
Висновки до розділу 160
Питання для самоперевірки 160

Список використаних джерел 163

Додатки 179
Додаток А. Короткий словник ключових термінів 179
Додаток Б. Зіставлення методів виявлення 180
Додаток В. Контрольний перелік зрілості протидії ЗПК 180

Перелік умовних скорочень

АСК ТП	автоматизована система керування технологічними процесами
ЗПК	злоякісний програмний код
КК	Кримінальний кодекс України
МН	машинне навчання
ОС	операційна система
ПЗ	програмне забезпечення
AMSI	Antimalware Scan Interface
APT	Advanced Persistent Threat (цілеспрямована стала загроза)
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge (MITRE)
C2	Command and Control (командний сервер керування)
CERT-UA	Урядова команда реагування на комп'ютерні надзвичайні події України
CFG	Control-Flow Graph (граф потоку керування)
CI/CD	Continuous Integration / Continuous Delivery
DDoS	Distributed Denial of Service
EDR	Endpoint Detection and Response
ELF	Executable and Linkable Format
ETW	Event Tracing for Windows
ICS	Industrial Control System (промислова система керування)
IoC	Indicator of Compromise (індикатор компрометації)
IoT	Internet of Things (Інтернет речей)
IT/OT	Information Technology / Operational Technology
LotL	Living off the Land
PE	Portable Executable
PPV	Positive Predictive Value (прогностична цінність)
ROC	Receiver Operating Characteristic
SBOM	Software Bill of Materials
SCADA	Supervisory Control and Data Acquisition
SIEM	Security Information and Event Management
SLSA	Supply-chain Levels for Software Artifacts

SOC	Security Operations Center (центр безпеки)
STIX	Structured Threat Information Expression
TAXII	Trusted Automated Exchange of Intelligence Information
TTP	Tactics, Techniques, and Procedures
XDR	Extended Detection and Response
YARA	Yet Another Recursive Acronym (мова правил виявлення)

Вступ

Злоякісний програмний код — шкідливе програмне забезпечення, віруси, хробаки, троянські програми, вимагачі, руткіти, шпигунський код — залишається одним із найпоширеніших і найнебезпечніших інструментів кіберзагроз. За останні десятиліття він пройшов шлях від поодиноких експериментальних вірусів до індустрії, що породжує сотні тисяч нових зразків щодоби, та до високотехнологічної зброї, здатної руйнувати фізичну інфраструктуру й завдавати збитків державного масштабу. Протидія цьому явищу перетворилася з вузькотехнічного завдання антивірусних лабораторій на комплексну наукову та інженерну дисципліну, що поєднує теорію обчислень, математичну статистику, аналіз програм, машинне навчання, мережеві технології, право та організаційне управління.

Україна посідає особливе місце в історії протидії злоякісному коду. Саме тут було вперше застосовано низку найруйнівніших кампаній сучасності — від атак на енергосистему за допомогою спеціалізованого коду до глобальних деструктивних епідемій, що поширилися через довірені канали постачання програмного забезпечення. Цей гіркий, але безпрецедентно багатий досвід зробив українських фахівців і дослідників носіями знань, цінних для всього світу, і водночас поставив підготовку висококваліфікованих спеціалістів із протидії шкідливому коду в розряд завдань національної безпеки.

Цей навчальний посібник призначено для здобувачів другого (освітнього) рівня вищої освіти — майбутніх докторів філософії у сфері кібербезпеки, — а також для науковців і фахівців-практиків. Його мета — сформувати цілісне, системне розуміння технологій протидії злоякісному коду: не лише окремих методів і засобів, а й принципів, що їх об'єднують, їхніх теоретичних меж і місця в загальній стратегії захисту. Виклад свідомо зосереджено на *захисних* аспектах — виявленні, аналізі, протидії та реагуванні; посібник не містить працездатних зразків шкідливого коду й не навчає його створювати, натомість озброює читача знаннями, потрібними для побудови ефективного захисту.

Структурно посібник побудовано за принципом руху від основ до стратегії. Перші розділи закладають фундамент: понятєвий апарат і таксономію злоякісного коду (розділ 1), нормативно-правове й стандартизаційне забезпечення (розділ 2), теоретичні та математичні основи, зокрема принципову нерозв'язність точного виявлення (розділ 3), моделювання загроз і ланцюги ураження (розділ 4), внутрішню будову та життєвий цикл шкідливого коду (розділ 5). Наступні розділи розкривають методи аналізу та ви-

явлення: статичний аналіз і сигнатурне виявлення (розділ 6), динамічний і поведінковий аналіз (розділ 7), техніки ухилення й обфускації, яким протистоїть захист (розділ 8), машинне навчання та статистичне виявлення (розділ 9).

Далі посібник переходить від окремих методів до їх практичної реалізації та оркестрування: захист кінцевих точок, моніторинг і телеметрія (розділ 10), інженерія виявлення (розділ 11), розвідка загроз і атрибуція (розділ 12), протидія цілеспрямованим сталим загрозам і полювання за загрозами (розділ 13). Окремі розділи присвячено специфічним середовищам і векторам: протидії шкідливому коду в промислових системах, Інтернеті речей, мобільних і хмарних середовищах (розділ 14) та атакам на ланцюг постачання програмного забезпечення (розділ 15). Завершують посібник розділи про реагування на інциденти й цифрову форензику (розділ 16) та про організаційне управління й стратегію кіберстійкості (розділ 17), що синтезують увесь матеріал у цілісну систему.

Крізь увесь посібник проходить кілька наскрізних ідей. Перша — принципова неможливість абсолютного виявлення й абсолютного захисту, що випливає з теорії обчислень і теорії ймовірностей, а отже й необхідність мислити категоріями ризику, а не гарантій. Друга — перевага стійких, семантично значущих ознак над поверхневими, бо саме вони опираються спробам ухилення. Третя — ешелонування й каскад методів із зростанням вартості й покриття, у якому дешеві й точні рубежі звільняють дорогі для найскладніших випадків. Четверта — припущення компрометації й зміщення мети від недопущення атаки до кіберстійкості. П'ята — нерозривний зв'язок технологій, процесів і людей: найкращі засоби безсилі поза зрілою системою управління.

Кожен розділ супроводжується теоретичним викладом, ілюстраціями, таблицями, формалізованими моделями, практичним кейсом і переліком запитань для самоперевірки, зокрема дослідницьких завдань для самостійної наукової роботи. Наприкінці наведено єдиний список використаних джерел, що охоплює основоположні монографії, міжнародні стандарти, провідні наукові публікації та вітчизняні дослідження. Посібник не замінює, а доповнює практичну роботу з реальними засобами аналізу та захисту й передбачає, що читач поглиблюватиме знання самостійним дослідженням.

Автор висловлює сподівання, що цей посібник стане надійною основою для підготовки дослідників і фахівців, здатних протистояти постійно еволюційним загрозам злоякісного програмного коду, і зробить свій внесок у зміцнення кіберстійкості України.

РОЗДІЛ 1

Злоякісний програмний код як об'єкт протидії: поняття, таксономія та простір загроз

Злоякісний програмний код став одним із головних інструментів у сучасних кібератаках і водночас одним із найскладніших об'єктів захисту. На відміну від пасивних загроз, він є активним, самодостатнім артефактом, що виконується на обчислювальній системі й цілеспрямовано порушує її безпеку: викрадає дані, шифрує або знищує їх, перехоплює керування, забезпечує прихований доступ зловмисника. За останні десятиліття злоякісний код пройшов шлях від експериментальних саморозмножуваних програм до промислової екосистеми з розподілом праці, ринком послуг і державним спонсоруванням, а кількість нових зразків зросла до сотень мільйонів на рік. Для України ця проблематика має особливу гостроту: країна стала полігоном найруйнівніших кампаній в історії (NotPetya, Industroyer, хвиля деструктивних «вайперів» 2022 року), що зробило протидію злоякісному коду питанням національної безпеки. Розділ формує понятійний апарат і системне бачення предмета: дає означення й термінологію, будує таксономію злоякісного коду, простежує його еволюцію, описує життєвий цикл злоякісної кампанії, модель порушника та простір загроз, аналізує український контекст і окреслює принципи протидії й структуру курсу.

1.1 Поняття та термінологія

1.1.1 Означення злоякісного програмного коду

Злоякісний програмний код (ЗПК; у міжнародній літературі — *malware*, від *malicious software*) — це програмний код, навмисно створений для виконання несанкціонованих, шкідливих для власника інформаційної системи дій. Ключовими в означенні є три ознаки: *навмисність* (код створено з наміром завдати шкоди, що відрізняє ЗПК від звичайних програмних помилок), *несанкціонованість* (дії виконуються без відома та згоди законного власника) і *функціональна спрямованість на порушення безпеки* (конфіденційності, цілісності, доступності, підзвітності ресурсів). У вітчизняній нормативній традиції (НД ТЗІ 1.1-003-99 [28]) споріднене поняття — «програмна закладка» та «комп'ютерний вірус»; у Кримінальному кодексі України [18] вживається термін «шкідливі програмні чи технічні засоби». У цьому посібнику терміни «злоякісний програмний код» і «шкідливе програмне забезпечення» (ШПЗ) вживаються як синоніми.

Принципово, що «злоякісність» визначається не внутрішніми властивостями коду, а *наміром* і *контекстом* його використання. Той самий технічний прийом (наприклад, віддалене виконання команд, шифрування файлів, перехоплення введення) може бути легітимною функцією адміністрування чи захисту або складовою атаки. Ця двоїстість (**dual-use**) є фундаментальною для предметної галузі: вона ускладнює автоматичне розрізнення й зумовлює принципову неможливість суто технічного визначення злоякісності у загальному випадку (розділ 3).

Прикладом служить клас інструментів подвійного призначення: засоби віддаленого адміністрування (RAT у легітимному сенсі), утиліти стиснення й шифрування, фреймворки тестування на проникнення (Cobalt Strike, Metasploit), системні утиліти командного рядка. Усі вони мають законне застосування, але масово зловживаються в атаках; розмежування спирається на поведінковий контекст (хто, коли, за яких обставин запустив, з якими параметрами), а не на сам факт наявності коду. Саме тому сучасні засоби захисту оцінюють не лише артефакт, а й ланцюг подій навколо його виконання (розділи 7, 10). Окремою складністю є **потенційно небажані програми** (PUP/PUA) — рекламне ПЗ, агресивні тулбари, програми стеження, що формально не є злоякісними, але порушують інтереси користувача; їх класифікація залежить від політики й згоди користувача.

1.1.2 Злоякісний код у системі понять безпеки

Злоякісний код є *засобом реалізації загрози*, а не самою загрозою. У термінах управління ризиками (розділ 4) джерело загрози (зловмисник) використовує ЗПК для експлуатації вразливості з метою завдати впливу на актив. Протидія злоякісному коду (**anti-malware**, захист від шкідливого коду) — це сукупність технологій, процесів і організаційних заходів, спрямованих на запобігання проникненню, виявлення, аналіз, стримування, видалення ЗПК і відновлення після його дії. Відповідно до функцій рамки NIST CSF 2.0 [164] протидія охоплює ідентифікацію, захист, виявлення, реагування й відновлення, а настанова NIST SP 800-83 [205] присвячена саме запобіганню інцидентам зі шкідливим кодом та їх обробці.

1.2 Таксономія злоякісного програмного коду

Злоякісний код класифікують за кількома незалежними ознаками [114, 58, 214]: за *механізмом поширення й автономності*, за *функціональним призначенням* (корисним навантаженням) і за *способом маскування*. Ці осі ортогональні: конкретний зразок одночасно належить до класів за кожною з них (наприклад, самопоширюваний черв'як-вимагач з руткіт-компонентом).

1.2.1 Класифікація за механізмом поширення

За здатністю до самопоширення й потребою в носії розрізняють три базові типи. **Вірус** — код, що вбудовується в інший файл-носій (виконуваний файл, документ із макросом, завантажувальний сектор) і розмножується, заражаючи нові носії під час виконання. **Черв'як** (worm) — самостійна програма, що поширюється мережею без носія й без участі користувача, експлуатуючи вразливості або слабкі облікові дані. **Троян** (троянський кінь) — програма, що маскується під легітимну й не має механізму самопоширення, покладаючись на дії користувача (запуск, встановлення). Формальну основу теорії вірусів заклав Коен [86, 85], спираючись на модель саморозмножуваних автоматів фон Неймана [225]; перший масштабний мережевий черв'як проаналізував Споффорд [206].

Коен також довів принциповий результат: задача точного виявлення вірусів у загальному випадку є алгоритмічно нерозв'язною (зводиться до проблеми зупинки), що має прямі наслідки для теорії протидії (розділ 3). Механізм самопоширення визначає *динаміку епідемії*: для черв'яків, що сканують адресний простір, початкове зростання числа заражених вузлів є експоненційним і описується епідемічними моделями (розділ 4), чим пояснюється швидкість історичних епідемій на кшталт Slammer, який уразив більшість вразливих вузлів за лічені хвилини. Швидкість поширення обмежується щільністю вразливих вузлів і пропускною здатністю, а не можливостями самого коду, тому своєчасне усунення вразливостей (розділ 10) кардинально змінює динаміку.

1.2.2 Класифікація за функціональним призначенням

За корисним навантаженням виділяють (табл. 1.1): **бекдор** (backdoor) — забезпечує прихований віддалений доступ; **завантажувач** (downloader/dropper) — доставляє й встановлює інші компоненти; **програма-вимагач** (ransomware) — шифрує дані з вимогою викупу (розділ 12); **вайпер** (wiper) — незворотно знищує дані, імітуючи вимагач або без маскування; **шпигунське ПЗ** (spyware) — викрадає інформацію; **клавіатурний шпигун** (keylogger) — перехоплює введення; **банківський троян** — краде фінансові облікові дані; **руткіт** (rootkit) — приховує присутність інших компонентів, модифікуючи систему; **бот** — робить вузол частиною керованої мережі (ботнету); **криптомайнер** — використовує ресурси для видобутку криптовалюти.

Таблиця 1.1 — Класифікація злоякісного коду за функціональним призначенням

Клас	Призначення (корисне навантаження)	Приклад
Бекдор	прихований віддалений доступ і керування	багато APT-імплантів
Завантажувач	доставка й встановлення інших компонентів	Emotet, dropper'и
Вимагач	шифрування даних із вимогою викупу	розділ 12
Вайпер	незворотне знищення даних	HermeticWiper, CaddyWiper
Шпигунське ПЗ	викрадення інформації, стеження	інфостілери
Банківський троян	крадіжка фінансових даних	розділ 13
Руткіт	приховування присутності в системі	розділ 5
Бот	включення вузла до ботнету	розділ 11
Промислове ЗПК	вплив на технологічні процеси	Industroyer, Stuxnet

1.2.3 Класифікація за способом маскування

За здатністю ухилятися від аналізу й виявлення розрізняють (розділ 8): **пакований** код (стиснутий/зашифрований пакувальником, що розгортається в пам'яті); **поліморфний** (змінює власний код у кожному поколінні, зберігаючи функцію, через шифрування зі змінним ключем); **метаморфний** (переписує власне тіло без шифрування); **безфайловий** (виконується лише в пам'яті, не залишаючи файлу на диску, розділ 13); код із **антианалізом** (виявляє пісочницю, налагоджувач, віртуальне середовище й змінює поведінку).

1.2.4 Багатовимірність класифікації

Жоден зразок не вичерпується одним класом: сучасний ЗПК є *модульним* і *багатофункціональним*. Тому практичні таксономії є фасетними (багатовимірними), що відображають одночасну належність зразка до класів за різними осями; розроблення такої багатовимірної фасетної таксономії, зокрема технік обфускації, є предметом окремих досліджень [48]. Для узгодженого опису *поведінки* (а не формальної належності) галузевим стандартом де-факто є база знань MITRE ATT&CK [159], що каталогізує тактики й техніки, а база MITRE D3FEND [160] — відповідні контрзаходи (розділ 4).

1.3 Еволюція злоякісного коду

Еволюцію ЗПК зручно подати як зміну *домінівної мотивації* та *моделі організації* (рис. 1.1). *Епоха цікавості* (1980-ті – середина 1990-х): експериментальні віруси й черв'яки, що поширювалися через дискети й ранні мережі; мотив — дослідницький чи вандальний. *Епоха масових епідемій* (кінець 1990-х – 2000-ті): поштові й мережеві черв'яки (ILOVEYOU, Code Red,

Slammer), що уражали мільйони вузлів за години; поява комерційної мотивації (спам-ботнети, банківські трояни). *Епоха кіберзлочинності як індустрії* (2010-ті): професіоналізація, ринок шкідливого коду як послуги (**MaaS**), партнерські програми, спеціалізація (розробники, оператори, брокери доступу). *Епоха державних операцій та деструкції* (з початку 2010-х): кіберзброя проти промислових систем (Stuxnet [100, 230]), складні постійні загрози (APT), деструктивні кампанії проти критичної інфраструктури й цілих держав (Україна, 2015–2022 рр. [112]).

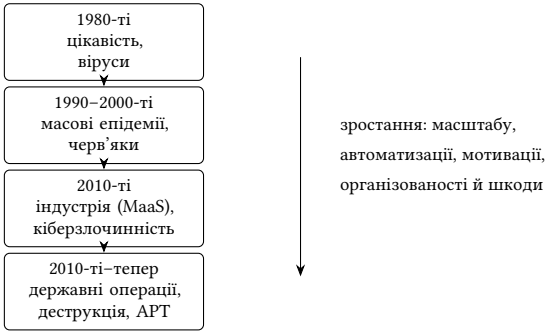


Рис. 1.1 — Еволюція домінівної мотивації та організації злоякісного коду

Ключовою рисою сучасної епохи є **комодитизація** злоякісного коду — перетворення його на товар і послугу. Склалася багаторівнева тіньова економіка з розподілом праці: розробники створюють шкідливий код і продають його або здають в оренду (**Malware-as-a-Service**); брокери початкового доступу (**Initial Access Brokers**) продають вже отриманий доступ до скомпрометованих мереж; оператори вимагачів працюють за партнерськими програмами (**Ransomware-as-a-Service**), ділячи викуп; окремо діють сервіси обфускації, тестування на виявлюваність антивірусами, оренди ботнетів і відмивання коштів. Ця спеціалізація різко знизила поріг входу: атаку здатен провести виконавець без глибоких технічних навичок, орендувавши готові інструменти. Для захисту це означає зростання кількості й різноманітності атак та розмивання межі між «простими» й «складними» загрозами, оскільки навіть малокваліфікований зловмисник отримує доступ до професійно розробленого коду.

Кількісно масштаб проблеми ілюструє зростання числа унікальних зразків. Якщо позначити через $N(t)$ накопичену кількість відомих зразків на момент t, то емпірично впродовж тривалого періоду вона зростала приблизно експоненційно:

$$N(t) = N_0\, e^{\,\alpha(t-t_0)}, \tag{1.1}$$

де α — коефіцієнт зростання. За даними антивірусних лабораторій [55] щорічно реєструють сотні мільйонів нових зразків, а загальна кількість перевищила мільярд. Проста наслідкова оцінка пояснює кризу суто сигнатурного підходу: якщо щодня з'являється ΔN нових зразків, а лабораторія здатна створити S сигнатур за добу, то частка невкритих зразків не менша за $1 - S/\Delta N$; за $\Delta N \gg S$ сигнатурне виявлення принципово відстає (розділ 6), що й зумовило перехід до поведінкових і статистичних методів (розділи 7, 9).

1.4 Життєвий цикл злоякісної кампанії

Злоякісний код є лише інструментом у ширшій *операції*. Її життєвий цикл описують моделлю ланцюга ураження (**kill chain**) [119], детально розглянутою в розділі 4. Узагальнено кампанія проходить етапи: розвідка (збір інформації про ціль); озброєння (підготовка ЗПК і доставки); доставка (фішинг, заражені носії, вразливий сервіс, ланцюг постачання); експлуатація (виконання коду через вразливість чи дію користувача); встановлення (закріплення в системі, персистентність); командування й керування (зв'язок з інфраструктурою зловмисника, C2); дії за цілями (крадіжка, шифрування, знищення, рух мережею). Кожен етап є точкою потенційного виявлення й розриву ланцюга — ключова ідея інтелектуально-керованого захисту: достатньо надійно перервати *будь-який* обов'язковий етап, щоб зірвати операцію.

Життєвий цикл власне злоякісного артефакта вкладений у цей ширший цикл і охоплює: доставку й первинне виконання; розгортання (розпакування, розшифрування тіла в пам'яті — розділ 8); перевірку середовища (антианаліз, виявлення пісочниці); закріплення (персистентність через автозапуск, служби, заплановані завдання, модифікацію системи — розділ 5); підвищення привілеїв; встановлення зв'язку з C2; виконання корисного навантаження; за потреби — самовидалення й приховування слідів. Розуміння цього циклу є основою як для аналізу (розділи 6, 7), так і для виявлення: кожна фаза залишає характерні артефакти й поведінкові ознаки, що стають індикаторами компрометації. Важлива асиметрія часу: зловмиснику вигідно діяти швидко до спрацювання навантаження, тоді як захиснику критично скоротити час від проникнення до виявлення (**dwell time**), який для складних загроз історично вимірювався тижнями й місяцями (розділ 13).

1.5 Модель порушника та мотивація

Протидія має відповідати моделі порушника (розділ 4). Профілі порушників, що застосовують ЗПК, суттєво різняться за ресурсами, мотивацією й наполегливістю (табл. 1.2).

Таблиця 1.2 — Профілі порушників, що застосовують злоякісний код

Профіль	Мотивація	Характер і ресурси
Кіберзлочинці	фінансова вигода	масовість, MaaS, вимагачі, інфостілери; помірні ресурси
Організовані угруповання	великий прибуток	партнерські програми, подвійне вимагання; значні ресурси
Хактивісти	ідеологія, розголос	дефейси, DDoS, витоки; змінні ресурси
Інсайдери	помста, вигода	легітимний доступ; складно виявити
Державні APT	розвідка, вплив, деструкція	тривала прихована присутність, 0-day, кіберзброя; великі ресурси

Для кількісного порівняння корисна інтегральна оцінка *рівня загрози*, що агрегує можливості й мотивацію порушника:

$$T = \frac{C \cdot M \cdot O}{D}, \tag{1.2}$$

де C — технічні можливості (capability); M — мотивація; O — наполегливість (persistence); D — стримувальні чинники (ризик викриття й покарання). Формула (1.2) якісно пояснює особливу небезпеку державних APT: висока C і M, екстремальна O за практично відсутнього D (атакувальник діє з юрисдикції, що його не переслідує). Для таких порушників традиційні засоби стримування не працюють, і захист має будуватися на припущенні компрометації (розділ 13).

1.6 Простір загроз і поверхня атаки

Поверхня атаки для проникнення ЗПК — сукупність шляхів доставки й виконання коду. Основні вектори: електронна пошта (фішинг з вкладеннями та посиланнями — домінівний вектор за даними [222]); вебзагрози (заражені сайти, шкідлива реклама, завантаження); зовнішні носії (USB); експлуатація вразливих сервісів, доступних з мережі; компрометація облікових даних і легітимного віддаленого доступу; ланцюг постачання програмного забезпечення (розділ 15); мобільні застосунки з ненадійних джерел (розділ 14). Поверхня атаки зростає з ускладненням середовищ: хмарні, контейнерні, IoT- та OT-системи додають власні вектори (розділ 14).

Зменшення поверхні атаки — перший і найдешевший рубіж протидії: обмеження вкладень і макросів, контроль застосунків, вимкнення непотрібних сервісів, сегментація, своєчасне оновлення (розділ 10). Проте жодне скорочення поверхні не усуває ризик повністю, тому необхідні глибоко ешелоновані виявлення й реагування.

Розподіл векторів доставки нерівномірний і залежить від типу порушника й цілі. За даними галузевих звітів [222, 97] для масових фінансо-

во вмотивованих кампаній домінують фішинг і експлуатація загальновідомих вразливостей, тоді як для цільових операцій проти критичної інфраструктури зростає роль компрометації ланцюга постачання й легітимного віддаленого доступу. Ця відмінність визначає пріоритети захисту для конкретної організації: модель загроз (розділ 4) має спиратися на релевантний профіль порушника, а не на абстрактний «усереднений» ландшафт. Важливо також, що успішна атака зазвичай поєднує кілька векторів послідовно: початкове проникнення через фішинг, закріплення через легітимні засоби, рух мережею через викрадені облікові дані, — тому ефективний захист покриває весь ланцюг, а не лише точку входу.

1.7 Український контекст

Україна посідає унікальне місце в історії протидії злоякісному коду як країна, що зазнала найруйнівніших відомих кампаній і водночас накопичила унікальний досвід захисту. *Атаки на енергосистему* (2015, 2016): шкідливий код BlackEnergy і згодом Industroyer [77] уперше в історії спричинили знеструмлення через кіберзасоби, а Industroyer безпосередньо маніпулював протоколами промислової автоматики (розділ 14). *NotPetya* (2017): замаскований під вимагач вайпер, доставлений через компрометований механізм оновлення бухгалтерського ПЗ (атака на ланцюг постачання, розділ 15), завдав глобальних збитків у мільярди доларів, ставши найдорожчою кібератакою в історії [99, 112]. *Хвиля деструктивних кампаній 2022 року*: з початком повномасштабного вторгнення Україна зазнала безпрецедентної кількості атак вайперами (WhisperGate, HermeticWiper, IsaacWiper, CaddyWiper та інші), спрямованих на знищення даних державних і критичних систем [92], а також оновленого Industroyer2 проти енергетики [200]. Аналіз діяльності APT-угруповань проти України систематизовано у вітчизняних дослідженнях [3].

Ці події дали ключові уроки, що проходять через увесь посібник. По-перше, у воєнному контексті домінують *деструктивні* атаки: мета зловмисника — не вигода, а знищення даних і порушення функціонування, тому класична логіка «заплати викуп — віднови дані» непридатна, а єдиним захистом стають ізольовані резервні копії й протестовані плани відновлення (розділ 16). По-друге, NotPetya продемонстрував критичність ланцюга постачання: компрометація одного довіреного постачальника ПЗ дала зловмиснику доступ до тисяч організацій одночасно (розділ 15). По-третє, маскування вайпера під вимагач показало, що зовнішня поведінка коду може бути навмисно оманливою, тому атрибуція наміру потребує глибокого аналізу (розділи 6, 7). По-четверте, швидкість поширення й руйнування (NotPetya паралізував уражені мережі за хвилини) означає, що запобігання й виявлення мають діяти *до* спрацювання корисного навантаження, а реа-

гування — бути заздалегідь відпрацьованим. По-п'яте, вирішальну роль відіграла національна й міжнародна координація: швидкий обмін індикаторами між CERT-UA [17], Держспецзв'язку [38] і світовою спільнотою дослідників дозволив оперативно випустити засоби виявлення й пом'якшення. Статистику й хронологію інцидентів узагальнюють звіти CERT-UA [74], а систематизацію діяльності APT-угруповань — вітчизняні дослідження [3]. Український досвід перетворив протидію злоякісному коду з технічної на стратегічну дисципліну національної стійкості й зробив вітчизняний науковий доробок у цій сфері затребуваним світовою спільнотою.

1.8 Принципи протидії та структура курсу

Протидія злоякісному коду ґрунтується на кількох наскрізних принципах. *Глибоко ешелонований захист*: кілька незалежних рубежів, щоб компрометація одного не означала повного ураження. *Припущення компрометації*: виходити з того, що частина атак подолає запобіжні рубежі, і будувати спроможність виявлення й реагування. *Пріоритет виявлення за поведінкою* над сигнатурами, зумовлений масштабом і мінливістю ЗПК (1.1). *Розрив ланцюга ураження* на найраніших досяжних етапах. *Науковість*: оцінювання ефективності засобів за явними метриками з урахуванням базової частоти (розділ 3).

Структуру курсу побудовано від засад до застосувань. Блок засад (розділи 1–4) формує поняття, нормативну базу, теоретичні основи й методологію оцінювання ризиків. Блок аналізу (розділи 5–9) розглядає будову ЗПК, статичний і динамічний аналіз, техніки ухилення та машинне навчання. Блок технологій захисту (розділи 10–11) — захист кінцевих точок і мереж. Блок класів загроз і середовищ (розділи 12–15) — вимагачі, APT, спеціалізовані середовища, ланцюг постачання. Блок операційної безпеки й управління (розділи 16–17) — реагування, криміналістика й управління протидією. Методологічною основою слугують класичні праці з будови й аналізу ЗПК [214, 58, 199] та інженерії безпеки [51].

Висновки до розділу

Злоякісний програмний код — це навмисно створений, несанкціонований код, спрямований на порушення безпеки інформаційної системи; його «злоякісність» визначається наміром і контекстом, а не внутрішніми властивостями, що породжує фундаментальну проблему dual-use і принципову складність автоматичного розрізнення. Таксономія ЗПК є багатовимірною: зразок одночасно класифікується за механізмом поширення (вірус, черв'як, троян), функціональним призначенням (бекдор, вимагач, вайпер, шпигунське ПЗ, руткіт, бот тощо) і способом маскування (пакування, поліморфізм, безфайловість, антианаліз); сучасний код є модульним і багато-

функціональним. Еволюція ЗПК відображає зміну мотивації від цікавості до індустрії кіберзлочинності й державних деструктивних операцій, а експоненційне зростання кількості зразків (1.1) зробило суто сигнатурний підхід недостатнім. Злоякісний код є інструментом у ширшій операції, життєвий цикл якої описує ланцюг ураження з численними точками виявлення й розриву. Модель порушника варіюється від масових кіберзлочинців до державних APT, для яких відсутні стримувальні чинники (1.2), що вимагає захисту за припущенням компрометації. Україна, зазнавши найруйнівніших кампаній в історії (Industroyer, NotPetya, хвиля вайперів 2022 року), накопичила унікальний досвід, що визначає наскрізні принципи протидії: ешелонованість, припущення компрометації, пріоритет поведінкового виявлення, розрив ланцюга ураження й науковість оцінювання.

Питання для самоперевірки

1. Дайте означення злоякісного програмного коду. Які три ознаки є в ньому ключовими і чому «злоякісність» не визначається внутрішніми властивостями коду?
2. У чому полягає проблема dual-use і які наслідки вона має для автоматичного виявлення ЗПК?
3. Назвіть три осі таксономії злоякісного коду. Чому класифікація є багатовимірною, а не ієрархічною?
4. Порівняйте вірус, черв'як і троян за механізмом поширення й потребою в носії та участі користувача.
5. Опишіть основні класи ЗПК за функціональним призначенням (табл. 1.1). Чим вайпер відрізняється від вимагача?
6. Які способи маскування злоякісного коду ви знаєте? Як вони пов'язані зі складністю виявлення (розділ 8)?
7. Охарактеризуйте етапи еволюції ЗПК за мотивацією та організацією (рис. 1.1).
8. За формулою (1.1) і співвідношенням $S/\Delta N$ поясніть, чому суто сигнатурний підхід принципово відстає від появи нових зразків.
9. Опишіть етапи життєвого циклу злоякісної кампанії. Чому розрив будь-якого обов'язкового етапу зриває операцію?
10. Порівняйте профілі порушників (табл. 1.2). Використовуючи (1.2), поясніть особливу небезпеку державних APT.
11. Перелічіть основні вектори доставки ЗПК. Чому зменшення поверхні атаки є необхідним, але недостатнім заходом?
12. Які уроки для протидії злоякісному коду дали атаки Industroyer, NotPetya та хвиля вайперів 2022 року в Україні?

13. Сформулюйте наскрізні принципи протидії злоякісному коду. Чому пріоритет надається виявленню за поведінкою?
14. Дослідницьке: побудуйте фасетну таксономію для вибраного набору реальних зразків ЗПК, визначивши осі класифікації та критерії віднесення, і порівняйте її з категоріями MITRE ATT&CK.
15. Дослідницьке: оцініть за даними відкритих звітів динаміку $N(t)$ (1.1) за останнє десятиліття, визначте коефіцієнт α і проаналізуйте, як він співвідноситься зі спроможністю сигнатурного виявлення.

РОЗДІЛ 2

Нормативно-правове та стандартизаційне забезпечення протидії злоякісному коду

Технічна протидія злоякісному коду діє в межах правового й нормативного поля, що визначає, які дії є протиправними, хто й за що відповідає, які вимоги обов'язкові для організацій і як узгоджуються засоби захисту. Для фахівця важливо розуміти не лише окремі документи, а й те, як вони взаємодіють: кримінальне право встановлює відповідальність за створення й поширення ЗПК; законодавство про кібербезпеку й критичну інфраструктуру зобов'язує організації впроваджувати захист і повідомляти про інциденти; стандарти управління конкретизують процеси; технічні бази знань забезпечують спільну мову опису загроз і контрзаходів. Розділ систематизує правове поле України, ЄС і міжнародне, аналізує кримінально-правову протидію, стандарти управління інцидентами й технічні бази знань, механізми оцінювання відповідності та завершується кейсом визначення вимог для оператора критичної інфраструктури. Процеси реагування й криміналістики, що спираються на ці норми, розглянуто в розділі 16.

2.1 Структура нормативного поля

Нормативне забезпечення протидії злоякісному коду доцільно подавати як ієрархію рівнів (рис. 2.1), що відрізняються за юридичною силою й предметом.

Міжнародний рівень формують конвенції й угоди, насамперед Конвенція Ради Європи про кіберзлочинність (Будапештська конвенція) [88], що гармонізує склади злочинів і механізми міжнародної співпраці, та її Другий додатковий протокол щодо транскордонного доступу до електронних доказів [195].

Наднаціональний рівень для України визначається зобов'язаннями за Угодою про асоціацію з ЄС [46] і орієнтацією на право ЄС: Директиву NIS 2 [91], Акт про кіберстійкість [182], Акт про кібербезпеку [180], GDPR [179].

Національний законодавчий рівень становлять закони України (про основні засади забезпечення кібербезпеки, про критичну інфраструктуру, про захист інформації в ІКС, про захист персональних даних), Кримінальний кодекс [18] і підзаконні акти.

Рівень стандартів містить міжнародні й національні стандарти (ISO/IEC 27000, NIST SP, ДСТУ), що набувають обов'язковості через посилання в нормативних актах або договорах.

Технічний рівень формують бази знань і специфікації (MITRE ATT&CK, CVE, CVSS, STIX/TAXII, YARA, Sigma), які де-факто є спільною мовою галузі.

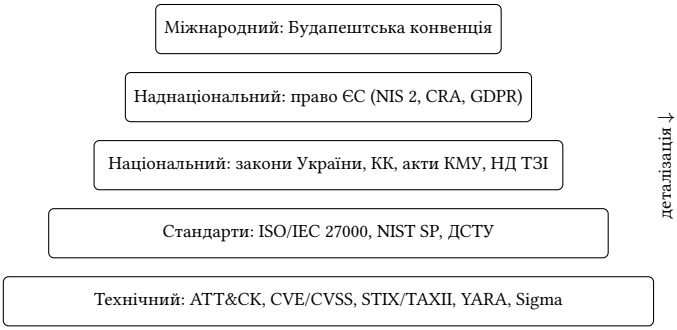


Рис. 2.1 — Рівні нормативного забезпечення протидії злоякісному коду

2.2 Кримінально-правова протидія

Створення, поширення й використання злоякісного коду є кримінальним правопорушенням. У Кримінальному кодексі України [18] склади злочинів у сфері використання ЕОМ, систем та мереж електрозв'язку зосереджено в розділі XVI. Ключовою для предмета є **стаття 361-1** [18] — «Створення з метою використання, поширення або збуту шкідливих програмних чи технічних засобів, а також їх поширення або збут»: вона встановлює відповідальність саме за дії зі шкідливим кодом як таким, незалежно від настання наслідків. Стаття 361 карає несанкціоноване втручання в роботу інформаційних систем (типовий наслідок застосування ЗПК), статті 362–363-1 — несанкціоновані дії з інформацією, її перехоплення й перешкоджання роботі мереж. Процесуальні аспекти (вилучення, фіксація й дослідження електронних доказів) регулює Кримінальний процесуальний кодекс [19], а методичні вимоги до роботи з цифровими доказами — стандарт ISO/IEC 27037 [128] (розділ 16).

Важливою для фахівців є проблема *правомірності дослідження* злоякісного коду. Створення, зберігання й аналіз зразків ЗПК є невід'ємною частиною роботи антивірусних лабораторій, дослідників безпеки, спеціалістів з реагування на інциденти та викладачів. Стаття 361-1 криміналізує дії, вчинені *з метою* використання, поширення або збуту шкідливих засобів; отже, визначальним є намір (mens rea), а не сам факт володіння зразком чи його дослідження в захищеному середовищі з науковою або захисною метою. Проте межа тонка, тому дослідницька й освітня діяльність має спиратися на чіткі організаційні засади: ізольовані лабораторні середовища, контроль доступу до колекцій зразків, документування мети робіт, заборону поши-

рення працездатних зразків поза контрольованим контуром, етичні норми відповідального розкриття вразливостей. У цьому посібнику виклад має суто захисну й аналітичну спрямованість: розглядаються принципи й методи *протидії* та *виявлення*, а не створення працездатного шкідливого коду. Такий підхід відповідає і міжнародній практиці, де законодавство про «хакерські інструменти» (dual-use) супроводжується застереженнями щодо законного застосування засобів тестування безпеки й дослідження.

Специфіка кримінально-правової протидії ЗПК — *транскордонність*. Зловмисник, інфраструктура й жертва часто перебувають у різних юрисдикціях, а докази — у хмарних сервісах третіх країн. Будапештська конвенція [88] гармонізує склади злочинів і запроваджує механізми взаємної правової допомоги й збереження даних, а її Другий протокол [195] спрощує транскордонний доступ до електронних доказів. Проте дієвість цих механізмів обмежена, якщо держава походження атаки не співпрацює, — саме тому для державних APT (розділ 1) кримінально-правове стримування практично не працює, і наголос зміщується на технічний захист та атрибуцію (розділ 16).

2.3 Законодавство України про кібербезпеку

Базовим актом є Закон України «Про основні засади забезпечення кібербезпеки України» [41], що визначає суб'єктів національної системи кібербезпеки, об'єкти кіберзахисту й повноваження. Для протидії ЗПК ключовими є обов'язки щодо кіберзахисту об'єктів критичної інфраструктури, повідомлення про кіберінциденти й взаємодії з CERT-UA [17]. Закон «Про критичну інфраструктуру» [40] встановлює категоризацію об'єктів і вимоги до їхнього захисту, деталізовані Загальними вимогами до кіберзахисту [10, 38], які прямо охоплюють захист від шкідливого коду: антивірусний захист, контроль застосунків, сегментацію, моніторинг, реагування на інциденти й відновлення.

Закон «Про захист інформації в інформаційно-комунікаційних системах» [39] і порядок створення комплексних систем захисту інформації (НД ТЗІ 3.7-003-05 [27]) визначають вимоги до захисту інформації з обмеженим доступом, зокрема до засобів захисту від несанкціонованого доступу й шкідливого коду; критерії оцінки захищеності встановлює НД ТЗІ 2.5-004-99 [26], а термінологію — НД ТЗІ 1.1-003-99 [28]. Важливо, що для систем, де обробляється інформація з обмеженим доступом, вимога застосування засобів захисту з підтвердженою відповідністю (позитивним висновком державної експертизи) є обов'язковою, що впливає на вибір антивірусних і EDR-рішень: для таких систем потрібні засоби, що пройшли відповідне оцінювання, тоді як для загальних систем вибір ширший і визначається ризик-орієнтованим підходом. Закон «Про електронні комунікації» [37] додатко-

во покладає на постачальників обов'язки щодо цілісності й безпеки мереж, зокрема протидії зловмисному трафіку й повідомлення про інциденти. Типове положення про службу захисту інформації [29] визначає організаційні засади. Стратегічні пріоритети, зокрема розвиток спроможностей протидії, сформульовано у Стратегії кібербезпеки України [43]. Досвід масштабних атак (розділ 1) і наближення до NIS 2 зумовили оновлення законодавства в напрямі ризик-орієнтованого підходу, розширення кола суб'єктів і чітких строків повідомлення про інциденти.

2.4 Право Європейського Союзу

2.4.1 Директива NIS 2

Директива (ЄС) 2022/2555 [91] встановлює високий спільний рівень кібербезпеки в ЄС і є орієнтиром для гармонізації українського законодавства. Для протидії ЗПК ключовими є: обов'язок уживати відповідних і пропорційних заходів управління ризиками (ст. 21), серед яких політики оброблення інцидентів, неперервність діяльності, безпека ланцюга постачання, кібергігієна, використання криптографії; багатоетапне повідомлення про значні інциденти (ст. 23); персональна відповідальність керівництва (ст. 20); суттєві адміністративні штрафи. Графік повідомлення задає систему обмежень на моменти подання звітів t_1, t_2, t_3 відносно моменту t_0, коли суб'єкту стало відомо про значний інцидент:

$$t_1 - t_0 \le 24 \text{ год}, \qquad t_2 - t_0 \le 72 \text{ год}, \qquad t_3 - t_2 \le 1 \text{ міс}. \tag{2.1}$$

Обмеження (2.1) мають прямі технічні наслідки: система виявлення й реагування (розділи 11, 16) повинна забезпечувати середній час виявлення (MTTD) і первинної класифікації інциденту з запасом менший за 24 год. Для інцидентів зі шкідливим кодом це означає потребу в автоматизованому виявленні й відпрацьованих процедурах тріажу.

2.4.2 Акт про кіберстійкість та інші акти

Регламент (ЄС) 2024/2847 — Акт про кіберстійкість (CRA) [182] — встановлює вимоги кібербезпеки до продуктів із цифровими елементами протягом життєвого циклу: безпека за замовчуванням, відсутність відомих експлуатованих вразливостей на момент випуску, захищене оновлення, оброблення вразливостей, специфікація складу програмного забезпечення (SBOM, розділ 15), повідомлення про активно експлуатовані вразливості. CRA є інструментом зменшення поверхні атаки на рівні екосистеми продуктів: він зменшує кількість вразливостей, які зловмисники використовують для доставки й виконання ЗПК, і підвищує прозорість складу ПЗ, що критично для протидії атакам на ланцюг постачання (розділ 15). Зобов'язання щодо повідомлення про активно експлуатовані вразливості застосовуються

раніше за основні вимоги, що створює механізм раннього попередження для всієї екосистеми. Для України, що гармонізує законодавство з правом ЄС, CRA задає орієнтир для вимог до постачальників цифрових продуктів, зокрема для критичної інфраструктури. Акт про кібербезпеку [180] заснував європейську рамку сертифікації, а GDPR [179] встановлює обов'язки щодо повідомлення про витоки персональних даних, які часто є наслідком дії ЗПК. Європейський центр компетенцій з кібербезпеки [181] координує дослідження й розвиток спроможностей.

2.5 Стандарти управління та реагування

Загальну рамку управління безпекою задають стандарти серії ISO/IEC 27000, прийняті в Україні як ДСТУ EN ISO/IEC 27001 [8] і 27002 [9]; останній містить конкретні заходи щодо захисту від шкідливого коду, керування вразливостями й технічними заходами. Управління інцидентами регламентує ISO/IEC 27035 [127], настанови з інтернет-безпеки — ISO/IEC 27032 [126], управління ризиками — ISO/IEC 27005 [125] та ISO 31000 [124], неперервність — ISO 22301 [123]. Рамка NIST CSF 2.0 [164] з функціями «Керувати», «Ідентифікувати», «Захищати», «Виявляти», «Реагувати», «Відновлювати» зручна для побудови профілів. Спеціалізованими є настанови NIST: SP 800-83 щодо запобігання інцидентам зі шкідливим кодом та їх обробки [205] і SP 800-61 щодо обробки інцидентів [81]. Пріоритетний набір практичних заходів дають CIS Controls [82], зокрема засоби захисту від шкідливого коду, керування вразливостями й контроль застосунків. Настанова NIST SP 800-83 [205] заслуговує окремої уваги як профільний документ: вона структурує протидію за чотирма групами заходів — політика (що дозволено й заборонено), підвищення обізнаності (навчання користувачів розпізнавати фішинг і соціальну інженерію), пом'якшення вразливостей (оновлення, конфігурація, принцип найменших привілеїв) і пом'якшення загроз (антивірусні засоби, контроль застосунків, системи виявлення вторгнень). Вона ж деталізує фази обробки інциденту зі шкідливим кодом — підготовку, виявлення й аналіз, стримування, викорінення, відновлення та висновки, — що узгоджується з ISO/IEC 27035 і становить основу розділу 16. Важливо, що стандарти розрізняють *запобіжні* заходи (зменшують імовірність і масштаб зараження) і *детективно-реактивні* (забезпечують виявлення й реагування за припущення, що частина атак подолає запобіжні рубежі), і вимагають обох груп. Відповідність основних документів функціям протидії подано в табл. 2.1.

Таблиця 2.1 — Нормативні документи за функціями протидії злоякісному коду

Функція	Документи
Управління ризиками	ISO/IEC 27005, ISO 31000, NIST CSF 2.0
Захист (заходи)	ДСТУ EN ISO/IEC 27002, CIS Controls, NIST SP 800-83
Виявлення	NIST SP 800-137, MITRE ATT&CK/CAR
Реагування на інциденти	ISO/IEC 27035, NIST SP 800-61, НД ТЗІ
Криміналістика	ISO/IEC 27037, NIST SP 800-86
Неперервність і відновлення	ISO 22301
Безпека продуктів і ланцюга	CRA, NIST SP 800-218, 800-161

2.6 Технічні бази знань і специфікації

Окремий рівень — технічні бази знань, що забезпечують узгоджений опис загроз і обмін інформацією. База MITRE ATT&CK [159] каталогізує тактики й техніки зловмисників, стаючи спільною мовою для опису поведінки ЗПК (розділ 4); MITRE D3FEND [160] описує контрзаходи, CAR [161] — аналітику виявлення. Вразливості каталогізує CVE [90], їхню серйозність оцінює CVSS [102], патерни атак — CAPEC [71]. Для обміну розвідданими про загрози стандартизовано формат STIX і протокол TAXII [169, 215], а платформи на кшталт MISP [156] забезпечують практичний обмін індикаторами. Для виявлення застосовують правила YARA (ознаки у файлах і пам'яті) [234] і Sigma (ознаки в журналах) [176]. Ці інструменти детально розглянуто в розділах 6, 11 і 17.

Принципова перевага відкритих баз знань і форматів — *сумісність і колективна оборона*: індикатори, виявлені однією організацією, швидко поширюються й використовуються іншими, що особливо важливо для протидії масштабним кампаніям (розділ 1). Водночас обмін має враховувати чутливість даних і довіру між учасниками, що регулюється моделями поширення (наприклад, світлофорний протокол TLP).

Корисною концептуальною рамкою для оцінювання цінності індикаторів є «піраміда болю» [63]: що вищий рівень індикатора, тим дорожче зловмиснику його змінити й тим більшої шкоди завдає його блокування. У основі піраміди — геш-значення файлів (їх зловмисник змінює тривіально), далі — IP-адреси й доменні імена (змінюються легко), мережеві й хостові артефакти, засоби, і нарешті — тактики, техніки й процедури (TTP), змінити які найважче, бо вони відображають спосіб мислення й робочі звички зловмисника. Ця рамка пояснює стратегічну перевагу поведінкового виявлення (розділ 7) над суто сигнатурним (розділ 6): виявлення за TTP,

закодованими, зокрема, у правилах Sigma й базі ATT&CK, витісняє зловмисника на рівень, де адаптація коштує йому найдорожче. Нормативно обмін розвідданими заохочується NIS 2 (обмін інформацією про загрози між суб'єктами) і реалізується через національні й галузеві майданчики за координації CERT-UA [17].

2.7 Оцінювання відповідності та обов'язки суб'єктів

Для практичного аналізу стану відповідності множині вимог використовують кількісні показники. Нехай $\mathcal{R} = \{r_1, \dots, r_N\}$ — множина застосовних вимог, $w_i > 0$ — вага вимоги (за критичністю), $c_i \in [0;1]$ — ступінь її виконання. Тоді **зважений індекс розриву відповідності**

$$G = 1 - \frac{\sum_{i=1}^{N} w_i\, c_i}{\sum_{i=1}^{N} w_i}, \qquad G \in [0;1]. \tag{2.2}$$

Індекс (2.2) є адитивною згорткою й тому компенсаторний: повне виконання багатьох другорядних вимог може замаскувати невиконання критичної. Для обов'язкових вимог застосовують додаткову блокувальну умову $\min_{i:\, r_i \in \mathcal{R}_{\text{обов}}} c_i = 1$, тобто відповідність визнається лише за повного виконання кожної обов'язкової вимоги. Такий двоступеневий підхід узгоджується з логікою оцінювання ризиків (розділ 4) і дозволяє пріоритизувати усунення розривів за критерієм «зменшення G на одиницю витрат».

Суб'єктами обов'язків є: оператори об'єктів критичної інфраструктури (кіберзахист, повідомлення, взаємодія з CERT-UA); власники ІКС з інформацією з обмеженим доступом (КСЗІ); постачальники цифрових продуктів (вимоги CRA); оператори персональних даних (захист від витоків). Одна організація може належати до кількох категорій одночасно.

Окремо слід наголосити на зв'язку відповідності й реальної захищеності. Формальне виконання вимог (наявність антивірусу, задокументована політика) не тотожне ефективній протидії: засіб може бути некоректно налаштований, сигнатури — застарілі, моніторинг — без реагування, резервні копії — доступні зловмиснику з тієї самої мережі. Тому сучасні підходи доповнюють перевірку відповідності *перевіркою ефективності*: моделюванням атак (red team, purple team), валідацією засобів контролю за реальними техніками ATT&CK, навчаннями з реагування на кіберполігонах [35]. Це узгоджується з переходом від «паперової» відповідності до ризик-орієнтованого й доказового підходу, закладеного в NIS 2 і NIST CSF 2.0. Для протидії ЗПК така перевірка особливо важлива, оскільки зловмисники цілеспрямовано тестують свій код проти поширених засобів захисту (розділ 8), і формальна наявність засобу не гарантує виявлення конкретної загрози.

2.8 Порівняння підходів України та ЄС

Порівняння ключових аспектів регулювання наведено в табл. 2.2. Воно показує, що національне законодавство рухається в напрямі, заданому правом ЄС, із збереженням специфіки (обов'язкова державна експертиза засобів, система ТЗІ).

Таблиця 2.2 — Порівняння підходів України та ЄС до протидії злоякісному коду

Аспект	Україна	ЄС
Рамковий акт	ЗУ «Про основні засади забезпечення кібербезпеки»	Директива NIS 2
Криміналізація ЗПК	КК, ст. 361-1 та суміжні	національне право за Будапештською конвенцією
Повідомлення про інциденти	CERT-UA, строки за підзаконними актами	24 год / 72 год / 1 міс.
Безпека продуктів	загальні вимоги	Акт про кіберстійкість (CRA)
Міжнародна співпраця	Будапештська конвенція, CERT-мережі	ENISA, мережа CSIRT, спільні операції

2.9 Кейс: правові вимоги для оператора критичної інфраструктури

Розглянемо оператора об'єкта критичної інфраструктури (наприклад, енергетичної компанії), що визначає застосовні вимоги протидії ЗПК. Процедура складається з кроків.

1. *Ідентифікація ролей*: об'єкт критичної інфраструктури та об'єкт кіберзахисту (ЗУ «Про критичну інфраструктуру», «Про основні засади забезпечення кібербезпеки»); власник ІКС, можливо з інформацією з обмеженим доступом (ЗУ «Про захист інформації в ІКС»); оператор персональних даних.
2. *Формування реєстру вимог*: обов'язкові заходи кіберзахисту [10, 38] (антивірусний захист, контроль застосунків, сегментація, моніторинг, резервне копіювання, реагування), вимоги до повідомлення про інциденти, вимоги ТЗІ.
3. *Відображення на технічні заходи*: вимога захисту від шкідливого коду — на засоби захисту кінцевих точок і мережі (розділи 10, 11); вимога виявлення — на моніторинг і EDR; вимога неперервності — на ізольовані резервні копії й план відновлення (розділ 16); вимога повідомлення — на процедуру з MTTD у межах (2.1).
4. *Оцінювання розриву*: розрахунок індексу (2.2) з перевіркою блокувальної умови для обов'язкових вимог.
5. *Планування*: пріоритизація заходів за ризиком і вартістю (розділ 4).

Числовий приклад. Реєстр містить 30 вимог: 10 обов'язкових із вагою 3, 14 стандартів із вагою 2, 6 рекомендацій із вагою 1 (сума ваг 64). Виконано: обов'язкових — 8 повністю, 2 частково (по 0,5); стандартів — 10 повністю, 4 частково; рекомендацій — 3 повністю. Зважена сума: $3(8{+}1){+}2(10{+}2){+}1 \cdot 3 = 27{+}24{+}3 = 54$, звідки $G = 1 - 54/64 \approx 0{,}156$. Попри помірний індекс, блокувальна умова не виконана через дві частково реалізовані обов'язкові вимоги (приміром, відсутність ізольованих резервних копій і неповне покриття моніторингом), тож саме вони першочергові, навіть якщо усунення інших розривів дало б більше зменшення G. Приклад ілюструє, що для об'єктів критичної інфраструктури формальний індекс доповнюється безумовним виконанням критичних вимог, спрямованих на найруйнівніші сценарії (розділ 1). Інтеграцію засобів виявлення й реагування та перевірку відповідних процедур на кіберполігонах розглянуто у вітчизняних дослідженнях [35], а підходи до класифікації загроз — у [48].

Висновки до розділу

Протидія злоякісному коду діє в багаторівневому нормативному полі: міжнародні конвенції гармонізують кримінальну відповідальність і співпрацю, право ЄС і національне законодавство встановлюють обов'язки організацій, стандарти конкретизують процеси, а технічні бази знань забезпечують спільну мову. Кримінально-правова протидія (в Україні — насамперед стаття 361-1 КК, що карає дії зі шкідливим кодом як такі) ускладнена транскордонністю й неефективна проти державних порушників, що зміщує наголос на технічний захист і атрибуцію. Законодавство України про кібербезпеку й критичну інфраструктуру прямо зобов'язує до захисту від шкідливого коду, виявлення, реагування й відновлення, а наближення до NIS 2 запроваджує ризик-орієнтований підхід і строки повідомлення про інциденти (2.1), що визначають вимоги до MTTD. Стандарти ISO/IEC 27000, NIST SP 800-83/61, CIS Controls задають процеси й заходи, а відкриті бази знань (ATT&CK, CVE/CVSS, STIX/TAXII, YARA, Sigma) — технічну основу колективної оборони. Для практики оцінювання корисний двоступеневий підхід: блокувальні обов'язкові вимоги плюс зважений індекс розриву (2.2), причому для критичної інфраструктури пріоритет надається безумовному виконанню вимог, спрямованих на найруйнівніші сценарії.

Питання для самоперевірки

1. Назвіть рівні нормативного забезпечення протидії злоякісному коду (рис. 2.1) і наведіть документи кожного рівня.
2. Яку відповідальність встановлює стаття 361-1 КК України? Чим вона відрізняється від статті 361 за об'єктом?

3. У чому полягає транскордонна специфіка кримінально-правової протидії ЗПК? Які механізми пропонує Будапештська конвенція і чому вони не працюють проти державних APT?
4. Які обов'язки щодо протидії шкідливому коду встановлює законодавство України про критичну інфраструктуру?
5. Які заходи управління ризиками вимагає стаття 21 Директиви NIS 2? Як строки повідомлення (2.1) впливають на вимоги до MTTD?
6. Що встановлює Акт про кіберстійкість і як він зменшує поверхню атаки на рівні екосистеми продуктів?
7. Зіставте нормативні документи з функціями протидії (табл. 2.1). Які документи стосуються криміналістики й реагування?
8. Яку роль відіграють технічні бази знань (ATT&CK, CVE/CVSS, STIX/TAXII, YARA, Sigma)? У чому перевага відкритих форматів для колективної оборони?
9. Обчисліть індекс розриву (2.2) для прикладу підрозділу 2.9, якщо обидві частково виконані обов'язкові вимоги буде виконано повністю. Чи виконується тепер блокувальна умова?
10. Хто є суб'єктами обов'язків щодо протидії ЗПК? Чому одна організація може належати до кількох категорій?
11. Порівняйте підходи України та ЄС (табл. 2.2). Які напрями гармонізації ви вважаєте першочерговими?
12. Чому для об'єктів критичної інфраструктури формальний індекс відповідності доповнюється безумовним виконанням критичних вимог?
13. Як стандарт ISO/IEC 27037 і НД ТЗІ пов'язані з кримінальним процесом щодо інцидентів зі шкідливим кодом?
14. Дослідницьке: розробіть реєстр вимог протидії ЗПК для організації обраного сектору, зіставте вимоги з технічними заходами й обчисліть індекс розриву з перевіркою блокувальних умов.
15. Дослідницьке: проаналізуйте застосовність механізмів Другого протоколу до Будапештської конвенції для отримання електронних доказів у справах про поширення ЗПК і визначте обмеження за наявної міжнародної практики.

РОЗДІЛ 3

Теоретичні та математичні основи протидії злоякісному коду

Протидія злоякісному коду лише на перший погляд є суто інженерною задачею. За практичними засобами виявлення стоять фундаментальні теоретичні результати, що визначають принципові межі можливого: чи взагалі можливий досконалий детектор; якою ціною досягається задана якість виявлення; чому системи з високою точністю на практиці породжують переважно хибні тривоги; як виміряти «схожість» двох зразків і стиснутість коду. Ігнорування цих основ призводить до хибних очікувань (віра в «абсолютний антивірус»), неправильного оцінювання засобів (без урахування базової частоти) і неефективних рішень. Розділ систематизує теоретичний апарат протидії: теоретико-обчислювальні межі виявлення, теорію статистичного виявлення й робочі характеристики, проблему базової частоти, інформаційно-теоретичні міри (ентропію й подібність), моделі ознакового простору й метрики ефективності, і завершується розгорнутим числовим прикладом оцінювання детектора. Матеріал є математичним ядром посібника, на яке спираються розділи аналізу (6–9) і захисту (10–11).

3.1 Постановка задачі виявлення

Задачу виявлення злоякісного коду формалізують як задачу класифікації. Нехай $\mathcal{X}$ — простір об'єктів (файлів, процесів, потоків подій), кожному з яких відповідає істинний клас $y \in \{0, 1\}$: 1 — злоякісний, 0 — доброякісний. **Детектор** — це функція $d : \mathcal{X} \to \{0, 1\}$, що наближає невідому істинну розмітку. Якість детектора визначають чотири результати: істинно-позитивні (TP, злоякісний виявлено), хибно-позитивні (FP, доброякісний помилково позначено злоякісним), істинно-негативні (TN) і хибно-негативні (FN, злоякісний пропущено). З них виводять базові показники: **ймовірність виявлення** (чутливість, повнота) $P_D = \mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$ і **ймовірність хибної тривоги** $P_{FA} = \mathrm{FP}/(\mathrm{FP} + \mathrm{TN})$.

Принципово розрізняти два режими. У *статичному* аналізі детектор досліджує об'єкт без його виконання (вміст файлу, структуру, рядки, ознаки); у *динамічному* — спостерігає поведінку під час виконання (виклики системних функцій, зміни в системі, мережеву активність). Кожен режим має власні теоретичні межі: статичний обмежений нерозв'язністю аналізу коду, динамічний — неповнотою спостереження (один запуск не розкриває всіх гілок поведінки). Ці обмеження детально розглянуто в розділах 6 і 7.

Порівняння теоретичних властивостей статичного й динамічного підходів наведено в табл. 3.1.

Таблиця 3.1 — Теоретичні властивості статичного й динамічного виявлення

Властивість	Статичний аналіз	Динамічний аналіз
Що досліджує	структуру й вміст без виконання	поведінку під час виконання
Повнота покриття	усі гілки коду (за ідеалу), але нерозв'язність	лише виконані гілки (неповнота)
Головна межа	нерозв'язність/обфускація (розділ 8)	антианаліз, один шлях виконання
Вартість	низька, швидкий	висока, потребує ізоляції
Ризик	відсутній (без запуску)	виконання реального коду

Слід також розрізняти *виявлення* (detection) і *класифікацію* (classification). Виявлення відповідає на бінарне запитання «злоякісний чи ні», тоді як класифікація визначає сімейство, тип або конкретну загрозу. Виявлення є передумовою реагування, а класифікація — атрибуції й розуміння наміру (розділ 16). Обидві задачі спираються на спільний апарат, але мають різні критерії якості: для виявлення критичні повнота й частота хибних тривог, для класифікації — матриця плутанини між класами. Ще одна важлива вісь — *момент* рішення: виявлення до виконання (превентивне, блокує запуск), під час виконання (детективне, зупиняє в процесі дії) і після (ретроспективне, за журналами й телеметрією для розслідування). Чим раніший момент, тим менша шкода, але тим менше інформації доступно й тим вища невизначеність рішення — фундаментальний компроміс, що проходить через усі технології захисту.

3.2 Теоретико-обчислювальні межі виявлення

3.2.1 Нерозв'язність досконалого виявлення

Фундаментальний результат теорії полягає в тому, що *досконалий детектор злоякісного коду у загальному випадку неможливий.* Коен [86] звів задачу точного виявлення вірусів до проблеми зупинки машини Тюрінга й довів її алгоритмічну нерозв'язність. Інтуїція доведення: припустимо, існує програма D, що для будь-якої програми P безпомилково визначає, чи є P вірусом. Тоді можна побудувати «суперечливу» програму V, яка запитує D про себе й поводиться протилежно до відповіді — якщо D каже «вірус», V не розмножується, і навпаки. Це призводить до суперечності, отже D не існує.

Загальнішою формою є теорема Райса [183]: будь-яка нетривіальна семантична властивість програм (властивість поведінки, а не синтаксису) є алгоритмічно нерозв'язною. Оскільки «злоякісність» — саме семантична

властивість (що програма *робить*), її точне визначення для довільної програми неможливе. Чесс і Вайт [78] показали існування принципово невиявних вірусів, а Спінелліс [207] довів, що навіть для вірусів обмеженої довжини надійна ідентифікація є NP-повною. Систематизацію нерозв'язних задач аналізу ЗПК наведено в [197].

Ці результати мають і зворотну проєкцію — на можливості зловмисника. Та сама нерозв'язність, що обмежує захисника, обмежує й автоматичне *доведення* злоякісності, але водночас дає зловмиснику теоретичне підґрунтя для ухилення: він може будувати код, аналіз якого еквівалентний нерозв'язній задачі (наприклад, через непрозорі предикати, опосередковані переходи, самомодифікацію — розділ 8), змушуючи статичний аналізатор або давати невизначену відповідь, або робити консервативні припущення. Практичні статичні аналізатори обходять це апроксимаціями: аналізом у бік безпеки (over-approximation, що дає хибні тривоги) або неповним аналізом (under-approximation, що дає пропуски). Вибір напряму апроксимації — інженерне рішення, що визначає профіль помилок інструмента. Жодна апроксимація не усуває фундаментальної межі; вона лише перерозподіляє неминучі помилки між FP і FN відповідно до призначення засобу.

3.2.2 Практичні наслідки

Нерозв'язність не означає марності виявлення — вона означає, що будь-який реальний детектор *неминуче* припускається помилок обох типів (FP і FN) і працює з наближеннями. Звідси три практичні висновки. По-перше, не існує й не може існувати «абсолютного антивірусу»; будь-яке твердження про 100 % виявлення є або маркетингом, або стосується скінченної тестової вибірки. По-друге, виявлення — це завжди компроміс між P_D і P_{FA}, і вибір робочої точки є інженерним і контекстним рішенням (підрозділ 3.3.2). По-третє, оскільки точний аналіз неможливий, практичні засоби комбінують багато *неповних* ознак (сигнатури, евристики, поведінку, статистику), кожна з яких виявляє частину загроз, а разом вони дають прийнятне покриття. Саме тому глибоко ешелонований захист (розділ 1) є не лише організаційним, а й математично обґрунтованим принципом.

3.3 Теорія статистичного виявлення

3.3.1 Перевірка гіпотез і критерій Неймана–Пірсона

Виявлення зводиться до перевірки статистичних гіпотез: H_0 — об'єкт доброякісний, H_1 — злоякісний. Детектор обчислює за об'єктом ознакову статистику $T(x)$ і порівнює з порогом τ. Згідно з лемою Неймана–Пірсона [132] оптимальним за фіксованої P_{FA} є детектор відношення правдоподібності

$$\Lambda(x) = \frac{p(x \mid H_1)}{p(x \mid H_0)} \underset{H_0}{\overset{H_1}{\gtrless}} \tau, \tag{3.1}$$

де $p(x \mid H_i)$ — розподіли ознак за відповідної гіпотези. На практиці ці розподіли невідомі й оцінюються за навчальними даними (розділ 9), а Λ наближається класифікатором. Вибір порогу τ задає компроміс: зростання τ знижує P_{FA} ціною зниження P_D.

3.3.2 Робоча характеристика приймача

Залежність P_D від P_{FA} за зміни порогу — **робоча характеристика приймача** (ROC) [101] — є основним інструментом порівняння детекторів (рис. 3.1). Ідеальний детектор проходить через точку $(0, 1)$; детектор, що вгадує навмання, лежить на діагоналі. Інтегральною мірою якості, незалежною від порогу, є площа під кривою (AUC). Для нормально розподілених ознак із відстанню між класами d' (у стандартних відхиленнях) справедливо $P_D = Q\big(Q^{-1}(P_{FA}) - d'\big)$, де Q — доповнювальна функція стандартного нормального розподілу.

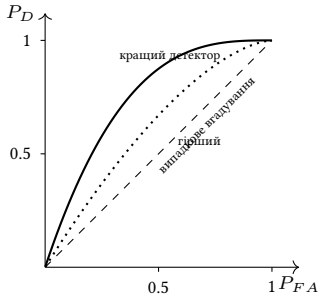


Рис. 3.1 — Робоча характеристика приймача (ROC) детекторів злоякісного коду

3.4 Проблема базової частоти

Найважливіший і найчастіше ігнорований теоретичний результат для практики — **проблема базової частоти** [57]. Для оцінювання корисності детектора вирішальні не P_D і P_{FA} самі собою, а *частка істинних тривог серед усіх тривог* — позитивна прогностична цінність. За теоремою Баєса, якщо апріорна частка злоякісних об'єктів у потоці дорівнює π, то

$$\mathrm{PPV} = \Pr\{H_1 \mid \text{тривога}\} = \frac{P_D\,\pi}{P_D\,\pi + P_{FA}\,(1-\pi)}. \tag{3.2}$$

Критична особливість протидії ЗПК: у реальному потоці подій злоякісні об'єкти *рідкісні* ($\pi \ll 1$). Нехай система моніторингу аналізує процеси з $P_D = 0{,}99$, $P_{FA} = 0{,}001$, а злоякісним є один процес із 10^5 ($\pi = 10^{-5}$). Тоді за (3.2) $\mathrm{PPV} = (0{,}99 \cdot 10^{-5})/(0{,}99 \cdot 10^{-5} + 10^{-3} \cdot (1 - 10^{-5})) \approx 0{,}0098$: понад 99 % тривог — хибні. Детектор із чудовими P_D і P_{FA} виявляється практично непридатним через рідкісність цілі.

Наслідки для проєктування: для рідкісних загроз потрібно або різко знижувати P_{FA} (на багато порядків), або підвищувати ефективну π через попередню фільтрацію (аналізувати лише вже підозрілі події), або — найважливіше — *корелювати кілька незалежних детекторів*. Для k незалежних детекторів з частотами хибних тривог $P_{FA}^{(i)}$ правило «тривога лише за згоди всіх» дає сумарну частоту $\prod_i P_{FA}^{(i)}$, що швидко знижує знаменник (3.2). Це математичне обґрунтування архітектури сучасних засобів (EDR, SIEM), які корелюють десятки слабких сигналів замість покладання на один «сильний» детектор (розділи 10, 11).

Проблема базової частоти має й зворотний бік — *вартість хибних тривог*. Кожна хибна тривога потребує часу аналітика на розгляд; за обмежених ресурсів надмірний потік хибних тривог призводить до «втоми від тривог» (alert fatigue), коли справжні загрози губляться серед шуму або ігноруються. Це не гіпотетична, а головна операційна проблема центрів безпеки: система з високою чутливістю, але неприйнятною точністю, фактично знижує захищеність, відвертаючи увагу від реальних інцидентів. Тому проєктування детекторів для рідкісних загроз має оптимізувати не чутливість, а *керованість потоку тривог* при заданому мінімальному рівні виявлення. Формально це задача оптимізації робочої точки з урахуванням пропускної спроможності аналітиків $\Lambda_{\max}$: допустима частота хибних тривог обмежена $P_{FA} \le \Lambda_{\max}/((1-\pi)\,N)$, де N — обсяг потоку. Для великих N ця межа надзвичайно жорстка, що знову приводить до необхідності кореляції й автоматизації тріажу (розділ 16).

3.5 Інформаційно-теоретичні міри

3.5.1 Ентропія та виявлення пакування

Багато технік ухилення (пакування, шифрування, розділ 8) роблять код статистично подібним до випадкових даних. Кількісною мірою є ентропія Шеннона [198] байтового розподілу фрагмента B:

$$H(B) = -\sum_{i=0}^{255} p_i \log_2 p_i, \qquad 0 \le H(B) \le 8, \tag{3.3}$$

де p_i — частота байта зі значенням i. Звичайний виконуваний код має ентропію приблизно 6,0–6,5 біт/байт, текст — 4–5, а стиснуті чи зашифро-

вані дані — близьку до 8. Лида й Гамрок [147] показали, що аномально висока ентропія секцій файлу є ефективним індикатором пакування чи шифрування. Проте ентропія сама собою не доводить злоякісності (легітимні програми теж пакують), тому вона є *однією* з ознак у багатофакторному рішенні (розділ 6). Теоретичною межею є зв'язок із колмогоровською складністю [143]: нестисливість фрагмента свідчить про відсутність регулярної структури, характерної для коду.

3.5.2 Міри подібності та нечітке гешування

Оскільки ЗПК існує сімействами (варіанти одного коду), ключовою є задача вимірювання *подібності* зразків. Криптографічний геш (MD5, SHA-256) непридатний: зміна одного байта повністю змінює геш. Тому застосовують **нечітке гешування** (context-triggered piecewise hashing, ssdeep) [137], що дає схожі геші для схожих файлів, і покускові/імпортні геші. Подібність двох об'єктів, поданих множинами ознак A і B (наприклад, n-грамів байтів [135] чи викликів функцій), вимірюють коефіцієнтом Жаккара:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}, \qquad J \in [0;1]. \tag{3.4}$$

Для ефективного обчислення подібності на великих колекціях застосовують локально-чутливе гешування (MinHash, SimHash), що зводить порівняння мільйонів зразків до пошуку близьких гешів. Міри подібності є основою кластеризації й атрибуції сімейств (розділ 6) і класифікації машинним навчанням (розділ 9).

Вибір рівня подання суттєво впливає на стійкість міри подібності до маскування. Байтові n-грами чутливі до пакування й перекомпіляції (поверхневі зміни руйнують подібність), тоді як подібність на рівні абстрактних структур (графа потоку керування, графа викликів, послідовностей системних викликів) стійкіша, бо відображає семантику, а не форму. Це ще один прояв «піраміди болю» (розділ 2): чим семантичніша ознака, тим дорожче зловмиснику її змінити, зберігши функціональність. Проте обчислення структурної подібності (наприклад, ізоморфізму підграфів) є дорогим, тому на практиці застосовують компроміси: наближені геші графів, відбитки імпортів, подібність послідовностей викликів. Задача ефективного й стійкого вимірювання подібності залишається активним напрямом досліджень, особливо в контексті великих колекцій і змагальних змін (розділ 9).

3.6 Ознаковий простір і межі навчання

Статистичні й машинно-навчальні детектори працюють в **ознаковому просторі**: об'єкт подається вектором ознак $\mathbf{x} \in \mathbb{R}^n$ (частоти n-грамів, імпортовані функції, ентропія секцій, послідовності викликів тощо), а детектор

— поверхнею, що розділяє класи. Теоретичні обмеження тут такі. *Прокляття розмірності*: зі зростанням n потрібна експоненційно більша навчальна вибірка для надійної оцінки меж. *Зсув розподілу*: розподіл ознак ЗПК змінюється з часом (нові сімейства, нові техніки), тому модель, навчена на минулому, деградує — явище **дрейфу концепту**, що вимагає постійного перенавчання (розділ 9). *Змагальність*: на відміну від природних даних, зловмисник *активно* змінює зразки, щоб уникнути виявлення, тому ознаки, які легко маніпулювати (поверхневі байтові), ненадійні, а стійкішими є семантичні й поведінкові (зв'язок із «пірамідою болю», розділ 2). Ці обмеження визначають, чому поведінкові й гібридні методи [23, 31] переважають суто статичні для стійкого виявлення.

3.7 Моделі поширення злоякісного коду

Для самопоширюваного коду (черв'яків, мережевих вимагачів) теоретичну основу протидії дають **епідемічні моделі**. Найпростіша — модель SI: якщо $I(t)$ — частка заражених вузлів, β — інтенсивність зараження, N — загальна кількість вразливих вузлів, то на початковій стадії

$$\frac{dI}{dt} = \beta\, I \left(1 - \frac{I}{N}\right), \tag{3.5}$$

що дає логістичну криву (рис. 3.2): спершу експоненційне зростання, потім насичення. Реалістичніша модель SIR додає стан «вилікуваних» (ізольованих, пропатчених) з інтенсивністю γ, а ключовим параметром стає **базовий репродуктивний коефіцієнт** $R_0 = \beta/\gamma$: епідемія поширюється, якщо $R_0 > 1$, і згасає, якщо $R_0 < 1$. Ця рамка кількісно обґрунтовує заходи протидії: зменшення β (сегментація, фільтрація, усунення вразливостей знижують імовірність успішного зараження сусіда) і збільшення γ (швидке виявлення й ізоляція, автоматичне оновлення) переводять систему нижче порога $R_0 = 1$, зупиняючи епідемію навіть без повного охоплення захистом усіх вузлів. Історичні епідемії (Slammer, Conficker) і мережеві вимагачі (WannaCry, NotPetya, розділ 12) підтверджують, що швидкість поширення визначається саме β й щільністю вразливих вузлів, а не складністю коду.

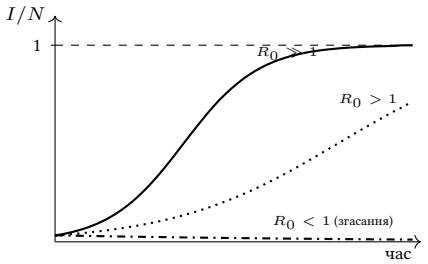


Рис. 3.2 — Динаміка поширення самопоширюваного коду за різних R_0 (моделі SI/SIR)

Докладніше моделювання поширення й ланцюгів ураження розглянуто в розділі 4.

3.8 Метрики ефективності

Для оцінювання детекторів використовують похідні від TP/FP/TN/FN метрики (табл. 3.2). Важливо вибирати метрику відповідно до задачі: за сильної незбалансованості класів (рідкісні загрози) точність (accuracy) оманлива, бо детектор, що завжди каже «доброякісний», матиме точність $1-\pi \approx 1$. Інформативнішими є повнота, точність (precision), їхнє гармонічне середнє F_1 і площа під кривою точність-повнота (PR-AUC), а для операційного оцінювання — вартість помилок з урахуванням їх несиметричності.

Таблиця 3.2 — Метрики ефективності виявлення злоякісного коду

Метрика	Формула	Призначення
Повнота (чутливість)	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$	частка виявлених загроз
Точність (precision)	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$	частка істинних серед тривог
Хибно-позитивна частота	$\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$	«шум» для аналітика
F_1-міра	$2PR/(P+R)$	баланс точності й повноти
PPV (з базовою частотою)	рівн. (3.2)	реальна цінність тривоги
AUC	площа під ROC	якість без прив'язки до порогу

Окремо для протидії ЗПК критичні *часові* метрики: час виявлення від моменту проникнення (визначає dwell time, розділ 13) і час реагування (розділ 16), прямо пов'язані з регуляторними вимогами щодо повідомлення (розділ 2).

Важливою методологічною вимогою є коректність *процедури оцінювання*. Поширені помилки, що завищують якість детектора: тестування на тих самих даних, на яких навчалися (витік даних); використання збалансованих тестових вибірок, що приховує проблему базової частоти; відсутність

часового розділення (навчання на пізніших зразках, тест на ранніх), що маскує дрейф концепту; відбір «легких» зразків. Коректне оцінювання вимагає часового розділення (train на минулому, test на майбутньому), реалістичної базової частоти, незалежної тестової вибірки й урахування змагальної адаптації. Для порівняння засобів галузь використовує публічні набори даних і майданчики незалежного тестування; проте результати таких тестувань слід інтерпретувати з урахуванням того, що зловмисники оптимізують свій код саме проти поширених засобів, тому висока оцінка на історичній вибірці не гарантує стійкості до нових загроз.

3.9 Розгорнутий приклад оцінювання детектора

Організація оцінює два детектори для потоку з 10^6 файлів на добу, серед яких у середньому 50 злоякісних ($\pi = 5 \cdot 10^{-5}$).

Детектор A (агресивна евристика): $P_D = 0{,}98$, $P_{FA} = 0{,}02$. Виявляє $0{,}98 \cdot 50 = 49$ загроз, але дає $0{,}02 \cdot (10^6 - 50) \approx 20\,000$ хибних тривог на добу. За (3.2) $\mathrm{PPV} = 49/(49 + 20\,000) \approx 0{,}0024$: аналітик потоне в хибних тривогах.

Детектор B (консервативна сигнатура): $P_D = 0{,}70$, $P_{FA} = 10^{-5}$. Виявляє 35 загроз, дає ≈ 10 хибних тривог; $\mathrm{PPV} = 35/45 \approx 0{,}78$. Практично придатний, але пропускає 15 загроз.

Каскад (B як перший ступінь, A як другий лише для підозрілих): якщо незалежні, правило «обидва спрацювали» дає $P_D = 0{,}98 \cdot 0{,}70 = 0{,}686$ і $P_{FA} = 0{,}02 \cdot 10^{-5} = 2 \cdot 10^{-7}$, звідки хибних тривог $\approx 0{,}2$/добу і $\mathrm{PPV} = 34/34{,}2 \approx 0{,}994$. Повнота при цьому впала. *Паралельне правило* «будь-який спрацював» дає $P_D = 1 - (1 - 0{,}98)(1 - 0{,}70) = 0{,}994$, але $P_{FA} \approx 0{,}02$ — знову шум.

Висновок: жодне окреме правило не оптимальне; практичне рішення — *багатоступенева кореляція* з ескалацією, де перший дешевий ступінь звужує потік, а дорогі методи застосовують лише до відфільтрованого (підвищена ефективна π), плюс пріоритизація тривог. Цей приклад кількісно обґрунтовує архітектуру розділів 10–11 і показує, що оцінювати детектор у відриві від базової частоти й робочого потоку некоректно.

Висновки до розділу

Теоретичні основи визначають принципові межі й правильні критерії протидії злоякісному коду. Фундаментальний результат — нерозв'язність досконалого виявлення (теореми Коена й Райса): «злоякісність» є семантичною властивістю, точне визначення якої для довільної програми алгоритмічно неможливе, тому будь-який реальний детектор неминуче припускається помилок обох типів, а «абсолютного антивірусу» не існує. Теорія статистичного виявлення формалізує компроміс між виявленням і хибни-

ми тривогами через відношення правдоподібності (3.1) і робочу характеристику приймача. Проблема базової частоти (3.2) — найважливіший практичний результат: для рідкісних загроз навіть детектор із чудовими показниками дає переважно хибні тривоги, що математично обґрунтовує багатоступеневу кореляцію незалежних сигналів замість покладання на один детектор. Інформаційно-теоретичні міри — ентропія (3.3) для виявлення пакування й міри подібності (3.4) з нечітким гешуванням для роботи із сімействами — є основою статичного аналізу й кластеризації. Межі навчання (прокляття розмірності, дрейф концепту, змагальність) пояснюють перевагу семантичних і поведінкових ознак над поверхневими. Метрики ефективності мають вибиратися з урахуванням незбалансованості класів і реального потоку, а оцінювати детектор у відриві від базової частоти некоректно, що наочно демонструє розгорнутий приклад.

Питання для самоперевірки

1. Сформулюйте задачу виявлення як задачу класифікації. Визначте P_D і P_{FA} через TP/FP/TN/FN.
2. Чим відрізняються теоретичні межі статичного й динамічного виявлення?
3. Викладіть ідею доведення Коена про нерозв'язність виявлення вірусів. Як із цим пов'язана теорема Райса?
4. Які три практичні висновки випливають з нерозв'язності досконалого виявлення?
5. Запишіть критерій Неймана–Пірсона (3.1). Як вибір порогу впливає на P_D і P_{FA}?
6. Що таке ROC-крива (рис. 3.1) і AUC? Як за ними порівнюють детектори?
7. Сформулюйте проблему базової частоти (3.2). Обчисліть PPV для $P_D = 0{,}95$, $P_{FA} = 0{,}005$, $\pi = 10^{-4}$.
8. Чому кореляція незалежних детекторів розв'язує проблему базової частоти? Як це пов'язано з архітектурою EDR/SIEM?
9. Що вимірює ентропія (3.3) і чому вона є індикатором пакування? Чому висока ентропія не доводить злоякісності?
10. Чому криптографічний геш непридатний для вимірювання подібності зразків? Опишіть нечітке гешування й коефіцієнт Жаккара (3.4).
11. Які теоретичні обмеження має навчання в ознаковому просторі? Що таке дрейф концепту й чому ознаки ЗПК змагальні?
12. Чому точність (accuracy) оманлива для рідкісних загроз? Які метрики інформативніші (табл. 3.1)?

13. Повторіть розрахунок прикладу підрозділу 3.9 для $\pi = 10^{-4}$ і поясніть, як змінюється придатність детекторів.
14. Дослідницьке: для набору реальних детекторів побудуйте ROC- і PR-криві на незбалансованій вибірці, оцініть AUC і проаналізуйте вплив базової частоти на операційну придатність.
15. Дослідницьке: запропонуйте схему багатоступеневої кореляції детекторів, що максимізує PPV за заданого мінімального P_D, і дослідіть її стійкість до дрейфу концепту та змагальних змін зразків.

РОЗДІЛ 4

Моделювання загроз, ланцюги ураження та оцінювання ризиків злоякісного коду

Нормативні вимоги (розділ 2) визначають, що має бути захищено, а теоретичні основи (розділ 3) — принципові межі виявлення. Між ними лежить процес, що пов'язує активи, загрози й заходи в систему обґрунтованих рішень: моделювання загроз і управління ризиками. Для протидії злоякісному коду цей процес має специфіку: загроза реалізується як багатоетапна операція (ланцюг ураження), а не одномоментна подія; шкода залежить від типу коду (крадіжка проти шифрування проти знищення); частина атак використовує самопоширення; значна частина ризику пов'язана з ланцюгом постачання й людським чинником. Розділ розглядає процес управління ризиками, методології моделювання загроз (ланцюги ураження, бази ATT&CK, дерева атак), кількісне оцінювання ризику й моделювання поширення, пріоритизацію заходів та економічне обґрунтування, і завершується кейсом побудови моделі загроз організації. Матеріал пов'язує технічні розділи (5–15) з операційними (16–17).

4.1 Процес управління ризиками

Управління ризиками щодо злоякісного коду здійснюється відповідно до ISO/IEC 27005 [125], ISO 31000 [124] і NIST SP 800-30 [168]: встановлення контексту, оцінювання ризиків (ідентифікація, аналіз, оцінка), оброблення, прийняття залишкових ризиків, наскрізні комунікація й моніторинг. NIST SP 800-30 деталізує оцінювання через ланцюг «джерело загрози → подія загрози → вразливість → вплив».

Специфіка контексту для ЗПК — визначення релевантного *профілю порушника* (розділ 1): модель загроз для рядової організації (масові кіберзлочинці, вимагачі) принципово відрізняється від моделі для об'єкта критичної інфраструктури чи державної установи (цільові APT, деструктивні кампанії). Активами, що потребують захисту, є не лише дані, а й доступність сервісів, цілісність систем керування, довіра до ланцюга постачання, безперервність діяльності. Критерії впливу мають охоплювати фінансові втрати, порушення функціонування, витоки персональних даних, регуляторні наслідки й, для критичної інфраструктури, вплив на безпеку людей.

Важлива особливість моделювання загроз щодо ЗПК полягає в тому, що загроза є *адаптивною*: на відміну від стихійних лих чи випадкових відмов, зловмисник спостерігає за захистом і змінює тактику у відповідь. Тому статична модель загроз швидко застаріває, і процес управління ризиками має

бути циклічним із регулярним переглядом на основі актуальної розвідки про загрози (threat intelligence, розділ 17). Другою особливістю є роль *людського чинника*: більшість успішних проникнень починається з дії людини (відкриття вкладення, перехід за посиланням, встановлення ПЗ), тому модель загроз має охоплювати не лише технічні вразливості, а й соціальну інженерію, і відповідно передбачати заходи навчання й технічного обмеження наслідків людських помилок. Третьою — *каскадність* впливу: зараження одного вузла в сучасних взаємопов'язаних середовищах може швидко поширитися, тому оцінювання має враховувати не лише прямий, а й поширений вплив через сегменти мережі, довірчі зв'язки й ланцюг постачання.

4.2 Моделювання загроз через ланцюг ураження

4.2.1 Класичний ланцюг ураження

Модель **ланцюга ураження** (Cyber Kill Chain) [119] описує злоякісну операцію як послідовність обов'язкових етапів (рис. 4.1): розвідка, озброєння, доставка, експлуатація, встановлення, командування й керування (C2), дії за цілями. Ключова ідея інтелектуально-керованого захисту — захиснику достатньо надійно перервати *будь-який* обов'язковий етап, щоб зірвати операцію, тоді як зловмиснику потрібно успішно пройти *всі*. Це створює асиметрію на користь захисника за умови глибоко ешелонованого захисту: кожен етап є можливістю для виявлення й реагування.

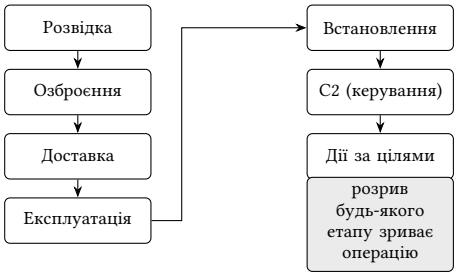


Рис. 4.1 — Ланцюг ураження злоякісної операції (за [119])

Розвитком є **уніфікований ланцюг ураження** [177], що об'єднує класичну модель із тактиками ATT&CK і краще відображає сучасні багатоетапні операції з рухом усередині мережі (lateral movement) між первинним проникненням і досягненням цілей. Для внутрішньомережевої фази характерний цикл: закріплення → розвідка всередині → підвищення привілеїв → рух до нових вузлів, що повторюється, поки зловмисник не досягне цілі.

Кожен етап ланцюга має власні характерні засоби протидії. На етапі *доставки* діють фільтрація пошти й вебтрафіку, пісочниці для вкладень (роз-

діл 7), навчання користувачів; на етапі *експлуатації* — усунення вразливостей, засоби запобігання експлойтам, принцип найменших привілеїв; на етапі *встановлення* — контроль застосунків, моніторинг персистентності (розділ 5), захист кінцевих точок (розділ 10); на етапі *C2* — мережеве виявлення, аналіз DNS і репутації, блокування відомої інфраструктури (розділ 11); на етапі *дій за цілями* — виявлення аномальної поведінки, обмеження доступу, захист резервних копій. Така відповідність «етап — контрзахід» є практичним інструментом проєктування ешелонованого захисту: для кожного обов'язкового етапу має бути щонайменше один незалежний рубіж виявлення або запобігання. Концепція «воронки виявлення» розвиває цю ідею: що далі просувається зловмисник, то менше в нього залишається способів уникнути виявлення, бо кожна дія в мережі залишає сліди.

4.2.2 База знань ATT&CK як модель загроз

База MITRE ATT&CK [159, 213] систематизує реальну поведінку зловмисників за тактиками (цілі: початковий доступ, виконання, персистентність, підвищення привілеїв, ухилення від захисту, доступ до облікових даних, розвідка, рух мережею, збір, C2, ексфільтрація, вплив) і техніками (способи досягнення). Для протидії ЗПК ATT&CK слугує одночасно моделлю загроз (що може робити зловмисник), мовою опису поведінки зразків (розділ 7) і картою покриття захисту: зіставивши наявні засоби виявлення з матрицею технік, організація бачить «білі плями». Перевага ATT&CK над абстрактними класифікаціями в тому, що вона побудована на *реально спостережених* техніках і прив'язана до конкретних угруповань і зразків, що робить її практичним інструментом як для «червоних» команд (моделювання атак), так і для «синіх» (проєктування виявлення). Для кожної техніки база містить опис, приклади використання реальними групами, способи виявлення й пом'якшення, що дозволяє від моделі загроз одразу переходити до вибору контрзаходів (зв'язок із D3FEND [160]). Важливе застереження: покриття технік не слід сприймати як просте «закривання клітинок матриці», бо різні техніки мають різну частоту, значущість і складність виявлення; пріоритизувати слід за реальною загрозою для конкретної організації, спираючись на розвідку про релевантні угруповання (розділ 17). Відображення типових технік ЗПК на тактики наведено в табл. 4.1.

Таблиця 4.1 — Типові техніки злоякісного коду за тактиками ATT&CK

Тактика	Приклади технік ЗПК
Початковий доступ	фішинг із вкладенням, експлуатація публічного сервісу, ланцюг постачання
Виконання	запуск користувачем, скрипти, інтерпретатори, планувальник
Персистентність	автозапуск, служби, заплановані завдання, модифікація реєстру (розділ 5)
Ухилення від захисту	пакування, обфускація, вимкнення засобів, LotL (розділи 8, 13)
Доступ до облікових даних	дамп пам'яті, кейлогінг, крадіжка токенів
Рух мережею	віддалене виконання, викрадені облікові дані
C2	зашифрований канал, DNS, легітимні сервіси (розділ 11)
Вплив	шифрування (вимагач), знищення (вайпер), маніпуляція процесами

4.3 Дерева атак

Дерево атак [192, 136] — ієрархічна модель, корінь якої — мета зловмисника, листя — елементарні дії, а внутрішні вузли типу «АБО» (достатньо однієї) та «І» (потрібні всі) описують альтернативи й послідовності. Атрибутам листя (ймовірність, вартість) приписують значення, що поширюються вгору: для «АБО» ймовірність $p_{\text{АБО}} = 1 - \prod(1 - p_k)$, для «І» — $p_{\text{І}} = \prod p_k$. На рис. 4.2 наведено дерево для мети «виконання ЗПК на робочій станції» з трьома альтернативними шляхами доставки.

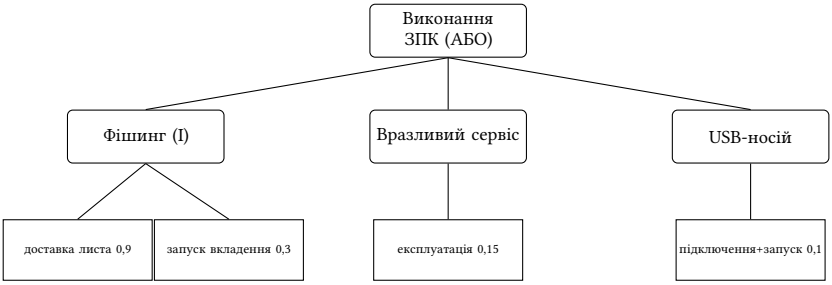


Рис. 4.2 — Дерево атак для мети «виконання злоякісного коду на робочій станції»

За наведеними ймовірностями $p_{A1} = 0{,}9 \cdot 0{,}3 = 0{,}27$, $p_{A2} = 0{,}15$, $p_{A3} = 0{,}1$, а для кореня $p_G = 1-(1-0{,}27)(1-0{,}15)(1-0{,}1) = 0{,}442$. Заходи захисту моделюють як зниження ймовірності відповідних листків: навчання користувачів і фільтрація вкладень знижують запуск вкладення з 0,3 до 0,05 ($p_{A1} = 0{,}045$); своєчасне оновлення — експлуатацію з 0,15 до 0,02; контроль USB — з 0,1 до 0,01. Тоді $p_G = 1-(1-0{,}045)(1-0{,}02)(1-0{,}01) = 0{,}074$,

тобто зниження майже в 6 разів. Дерево наочно показує, що найбільший внесок дає найімовірніший шлях (фішинг), тому пріоритет — заходи проти нього.

Розширенням дерев атак є **дерева атак і захисту** [136], у яких контрзаходи подано як окремі вузли, що взаємодіють з атакувальними: захід блокує атаку, атака може обходити захід, що моделюється вкладеними піддеревами. Такі моделі ближчі до реальності, бо відображають «гонку» між нападом і захистом: встановлення EDR змушує зловмисника застосувати техніки ухилення (розділ 8), що, своєю чергою, можуть бути виявлені поведінковим аналізом тощо. Перевага дерев — наочність і можливість кількісного аналізу; обмеження — вони погано відображають паралельність, час і стан системи, тому для складних сценаріїв застосовують графи атак, байєсові мережі чи імітаційне моделювання. Для протидії ЗПК дерева атак особливо корисні на етапі проєктування захисту, коли треба обґрунтувати, які рубежі й у якому порядку впроваджувати за обмеженого бюджету.

4.4 Кількісне оцінювання ризику

4.4.1 Ризик як сподівані втрати

Ризик від ЗПК оцінюють як математичне сподівання річних втрат (ALE):

$$\mathrm{ALE} = \sum_{i} \mathrm{ARO}_i \cdot \mathrm{SLE}_i, \qquad \mathrm{SLE}_i = \mathrm{AV}_i \cdot \mathrm{EF}_i, \tag{4.1}$$

де ARO_i — річна частота реалізації i-го сценарію; SLE_i — втрати від однієї події; AV_i — вартість активу; EF_i — фактор впливу. Методологія FAIR [104] деталізує оцінки через частоту подій загрози, уразливість і первинні/вторинні втрати з поданням невизначеності розподілами й моделюванням Монте-Карло. Для ЗПК важливо, що EF різко залежить від типу: для шпигунського коду втрати — вартість викрадених даних і репутації; для вимагача — простій плюс можливий викуп; для вайпера — $\mathrm{EF} \to 1$ (повне знищення), що робить такі сценарії пріоритетними попри меншу частоту.

4.4.2 Ймовірність компрометації ланцюга

Оскільки атака є ланцюгом, імовірність успішної компрометації оцінюють добутком імовірностей проходження обов'язкових етапів:

$$P_{\text{комп}} = \prod_{j=1}^{m} p_j(X), \qquad p_j(X) = p_j^0\,(1 - e_j), \tag{4.2}$$

де p_j^0 — імовірність успіху j-го етапу без заходів; $e_j \in [0;1]$ — ефективність заходу на цьому етапі. Формула (4.2) кількісно виражає силу ешелонованого захисту: навіть помірно ефективні заходи на кількох етапах у добу-

тку дають низьку сумарну ймовірність. Наприклад, три етапи з $p_j^0 = 0{,}8$ і заходами з $e_j = 0{,}7$ дають $P_{\text{комп}} = (0{,}24)^3 \approx 0{,}014$ замість $0{,}512$ без заходів. Для самопоширюваних загроз оцінку доповнюють епідемічною моделлю (розділ 3): навіть за успішного первинного проникнення масштаб ураження визначається R_0, тому сегментація й швидка ізоляція (зменшення R_0 нижче 1) обмежують шкоду [209, 232].

4.4.3 Інтеграція з оцінкою вразливостей

Імовірність експлуатації вразливостей, які ЗПК використовує для доставки, оцінюють поєднанням CVSS (технічна серйозність) [102] і EPSS (емпірична ймовірність експлуатації протягом 30 діб) [103]. Це дозволяє пріоритизувати усунення вразливостей за реальним ризиком, а не лише за балом серйозності: вразливість із помірним CVSS, але високим EPSS (активно експлуатується) небезпечніша за «страшну» на папері, але не експлуатовану. Прогнозування майбутніх загроз на основі історичних даних і поведінкових шаблонів досліджують із застосуванням штучного інтелекту [16].

Невизначеність оцінок — невід'ємна риса управління ризиками щодо ЗПК. Частоти атак, ймовірності етапів і розміри втрат відомі лише приблизно, тому точкові оцінки створюють хибне відчуття визначеності. Сучасна практика [104] вимагає подавати величини як калібровані інтервали (наприклад, 90-відсоткові) і обчислювати розподіл сумарного ризику методом Монте-Карло, отримуючи не одне число, а криву перевищення втрат — імовірність того, що річні втрати перевищать заданий рівень. Таку криву зіставляють із толерантністю до ризику, що її задає керівництво. Для протидії ЗПК це особливо важливо через «важкі хвости» розподілу втрат: рідкісні, але катастрофічні події (руйнівна атака, що паралізує організацію) домінують у математичному сподіванні, тому оцінювання лише за «типовим» сценарієм систематично занижує ризик. Український досвід (розділ 1) переконливо показав реальність таких хвостів.

4.5 Пріоритизація заходів та економіка

Доцільність заходу оцінюють показником рентабельності інвестицій у безпеку:

$$\text{ROSI} = \frac{\Delta\text{ALE} - C}{C}, \tag{4.3}$$

де ΔALE — зниження сподіваних втрат завдяки заходу; C — його річна вартість. Захід виправданий за $\text{ROSI} > 0$. Оскільки ΔALE залежить від уже впроваджених заходів, їх оцінюють послідовно в порядку спадання ROSI. Модель Гордона–Лоба [110] показує, що оптимальні інвестиції в захист не перевищують $\approx 37\,\%$ сподіваних втрат. Проте суто фінансові показники не

охоплюють усього: обов'язкові за законодавством заходи (розділ 2) впроваджуються незалежно від ROSI; детективні заходи мають цінність, яку важко виразити через ΔALE; для сценаріїв із загрозою безпеці людей застосовують принцип «настільки низько, наскільки практично досяжно».

Ключовим є *розміщення* заходів уздовж ланцюга ураження (4.2): заходи на ранніх етапах (зменшення поверхні атаки, навчання, фільтрація) дешевші й запобігають більшості масових атак; заходи на пізніх етапах (виявлення C2, поведінковий аналіз, реагування) дорожчі, але необхідні проти цілеспрямованих загроз, що подолали ранні рубежі. Збалансований захист покриває весь ланцюг, а не концентрується на одному етапі.

Для практичного ухвалення рішень корисно розрізняти три типи заходів за їхньою функцією. *Запобіжні* заходи знижують імовірність успіху етапу (ефективність e_j у (4.2)): оновлення, контроль застосунків, навчання. *Детективні* заходи не запобігають, але виявляють дію й уможливлюють реагування: моніторинг, EDR, аналіз журналів. *Коригувальні* заходи обмежують шкоду й відновлюють стан: сегментація, ізольовані резервні копії, план відновлення. Помилка концентрації лише на запобіганні є поширеною й небезпечною: за припущення компрометації (розділ 1) детективні й коригувальні заходи стають не менш важливими, бо визначають, наскільки швидко й з якими втратами організація виявить і подолає атаку, що неминуче подолає частину запобіжних рубежів. Кількісно це відображається в тому, що інвестиції лише в запобігання мають спадну віддачу (кожен наступний запобіжний рубіж дорожчий і дає менший приріст e_j), тоді як збалансований портфель запобіжних, детективних і коригувальних заходів мінімізує сумарний ризик за фіксованого бюджету. Саме тому сучасні рамки (NIST CSF 2.0) явно виділяють функції виявлення, реагування й відновлення поряд із захистом (розділ 2).

4.6 Кейс: модель загроз організації

Побудуємо модель загроз для організації середнього розміру (фінансовий сектор) за методикою розділу.

Контекст і профіль порушника. Релевантні: масові кіберзлочинці (вимагачі, банківські трояни), цілеспрямовані групи (крадіжка коштів), інсайдери. Активи: клієнтські дані, платіжні системи, доступність сервісів.

Ланцюги загроз. S1 — вимагач через фішинг (ARO $=$ 2, SLE $=$ 500 тис.); S2 — банківський троян через заражений сайт (ARO $=$ 1, SLE $=$ 800 тис.); S3 — APT через ланцюг постачання (ARO $=$ 0,2, SLE $=$ 3000 тис.); S4 — деструктивна атака (ARO $=$ 0,1, SLE $=$ 5000 тис.). Сумарний $\text{ALE}_0 = 1000 + 800 + 600 + 500 = 2900$ тис.

Заходи й пріоритизація. C1 — навчання + фільтрація пошти + контроль макросів ($C = 150$; знижує S1, S2); C2 — EDR з поведінковим виявленням

($C = 300$; знижує всі); C3 — ізольовані незмінні резервні копії + план відновлення ($C = 200$; знижує EF для S1, S4); C4 — контроль ланцюга постачання, SBOM ($C = 120$; знижує S3). За (4.2) і (4.1) послідовний розрахунок дає найвищий ROSI для C1 (дешевий, б'є найчастіші вектори) і C3 (різко знижує EF катастрофічних сценаріїв), далі C2 і C4.

Реєстр ризиків (фрагмент, табл. 4.2) фіксує сценарії, рівень і рішення. Для найруйнівніших сценаріїв (S4) рішення — зниження через C3 незалежно від ROSI, оскільки йдеться про виживання організації. Кейс ілюструє, що модель загроз для ЗПК має поєднувати ймовірнісне оцінювання з безумовним захистом від катастрофічних сценаріїв, а пріоритет надавати заходам на ранніх етапах ланцюга й ізольованому відновленню. Питання атрибуції, важливе для реагування, розглядають у межах багаторівневих моделей [20].

Таблиця 4.2 — Фрагмент реєстру ризиків злоякісного коду

Сценарій	ALE, тис.	Рівень	Рішення
Вимагач через фішинг	1000	високий	C1 + C2 + C3
Банківський троян	800	високий	C1 + C2
APT через ланцюг постачання	600	середній	C4 + C2
Деструктивна атака (вайпер)	500	критичний	C3 (безумовно) + C2

Висновки до розділу

Моделювання загроз і управління ризиками пов'язують нормативні вимоги й технічні засоби в систему обґрунтованих рішень. Для злоякісного коду загроза реалізується як багатоетапна операція, описувана ланцюгом ураження й базою ATT&CK; ешелонований захист створює асиметрію на користь захисника, бо розрив будь-якого обов'язкового етапу зриває операцію (4.2). Дерева атак дозволяють кількісно аналізувати альтернативні шляхи доставки й ефект заходів, показуючи пріоритет протидії найімовірнішому вектору (фішингу). Кількісне оцінювання ризику через ALE (4.1) має враховувати залежність фактора впливу від типу коду: деструктивні сценарії з EF $\to 1$ пріоритетні попри меншу частоту. Для самопоширюваних загроз оцінку доповнює епідемічна модель, а поєднання CVSS і EPSS уточнює ризик експлуатації вразливостей. Пріоритизація заходів через ROSI (4.3) з урахуванням розміщення вздовж ланцюга ураження, доповнена безумовним захистом від катастрофічних сценаріїв (ізольоване відновлення), становить основу раціональної програми протидії. Модель загроз має будуватися на релевантному профілі порушника, а не на абстрактному ландшафті.

Питання для самоперевірки

1. Опишіть процес управління ризиками щодо ЗПК. Чому визначення релевантного профілю порушника є ключовим на етапі контексту?
2. Опишіть етапи класичного ланцюга ураження (рис. 4.1). Чому розрив будь-якого етапу зриває операцію і яка асиметрія з цього випливає?
3. Чим уніфікований ланцюг ураження доповнює класичний? Опишіть цикл внутрішньомережевої фази.
4. Як база ATT&CK слугує одночасно моделлю загроз, мовою опису поведінки й картою покриття захисту (табл. 4.1)?
5. Побудуйте дерево атак для мети доставки ЗПК і обчисліть ймовірність досягнення кореня. Як моделюють ефект заходів?
6. За деревом рис. 4.2 перерахуйте p_G, якщо навчання знижує запуск вкладення до 0,1, а решта незмінна.

7. Запишіть формулу ALE (4.1). Чому фактор впливу різко залежить від типу ЗПК і чому деструктивні сценарії пріоритетні?
8. Поясніть формулу ймовірності компрометації ланцюга (4.2). Обчисліть $P_{\text{комп}}$ для чотирьох етапів із $p_j^0 = 0{,}7$ і $e_j = 0{,}6$.
9. Як поєднання CVSS і EPSS уточнює пріоритизацію усунення вразливостей? Наведіть приклад.
10. Поясніть показник ROSI (4.3) і результат Гордона–Лоба. Які заходи впроваджують незалежно від ROSI?
11. Чому важливе розміщення заходів уздовж ланцюга ураження? Порівняйте заходи на ранніх і пізніх етапах.
12. Чому для найруйнівніших сценаріїв рішення ухвалюють незалежно від ROSI? Проілюструйте на кейсі підрозділу 4.6.
13. Як епідемічна модель (розділ 3) доповнює оцінку ризику для самопоширюваних загроз?
14. Дослідницьке: побудуйте повну модель загроз для організації обраного сектору з деревами атак, кількісним оцінюванням і реєстром ризиків, та обґрунтуйте пріоритизацію заходів.
15. Дослідницьке: розробіть методику зіставлення наявних засобів виявлення з матрицею ATT&CK для оцінювання покриття захисту й виявлення «білих плям», застосуйте її до модельної інфраструктури.

РОЗДІЛ 5

Внутрішня будова та життєвий цикл шкідливого коду

Ефективна протидія злоякісному коду неможлива без розуміння того, як він улаштований і як виконується. Аналітик, що досліджує зразок, розробник правил виявлення, спеціаліст із реагування — усі спираються на знання форматів виконуваних файлів, механізмів завантаження в пам'ять, способів закріплення в системі й приховування присутності. Це знання має суто захисний характер: воно дозволяє визначати, де шукати ознаки компрометації, які артефакти свідчать про зараження, як відрізнити легітимну поведінку від злоякісної. Розділ розглядає внутрішню будову й життєвий цикл шкідливого коду з позиції аналітика-захисника: формати виконуваних файлів, механізми завантаження й виконання, техніки ін'єкції коду та способи їх виявлення, механізми персистентності, підвищення привілеїв, приховування (руткіти) й модульну архітектуру сучасного ЗПК. Виклад орієнтовано на *розпізнавання* й *аналіз*, а не на створення коду. Інструменти статичного й динамічного аналізу, що спираються на цей матеріал, розглянуто в розділах 6 і 7.

5.1 Формати виконуваних файлів

5.1.1 Структура формату PE

Основним форматом виконуваних файлів Windows є PE (Portable Executable) [155]. Знання його структури є фундаментом статичного аналізу, оскільки більшість ЗПК для найпоширенішої платформи має цей формат. PE-файл складається з (рис. 5.1): заголовка DOS (історична сумісність), сигнатури PE, файлового заголовка COFF (архітектура, кількість секцій, час компіляції), опційного заголовка (точка входу, образ бази, вирівнювання, характеристики), таблиці секцій і самих секцій. Типові секції наведено в табл. 5.1.

Таблиця 5.1 — Типові секції PE-файлу та їхнє значення для аналізу

Секція	Вміст і значення для аналізу
`.text`	виконуваний код; аномально висока ентропія свідчить про пакування
`.data`	ініціалізовані дані; рядки, конфігурація
`.rdata`	константи, таблиця імпортів (підказує функціональність)
`.rsrc`	ресурси; часто містять вкладені навантаження
`.reloc`	релокації; відсутність може вказувати на особливості завантаження
нетипові	нестандартні імена секцій — ознака пакувальника

Для аналітика найцінніша інформація — **таблиця імпортів** (список функцій, які програма викликає з системних бібліотек): вона підказує функціональність ще до виконання. Наприклад, імпорт функцій роботи з криптографією й файловою системою разом характерний для вимагачів; функцій ін'єкції й маніпуляції процесами — для імплантів; мережевих функцій і функцій реєстру — для бекдорів. Набір імпортів утворює характерний відбиток, що використовується для ідентифікації сімейств через **imphash** — геш упорядкованого списку імпортів [149]: зразки одного сімейства часто мають однаковий imphash, що дозволяє їх кластеризувати (розділ 6).

5.1.2 Інші формати

Для Unix-подібних систем (Linux, зокрема сервери й IoT, розділ 14) основним є формат ELF [217] з аналогічною логікою: заголовок, програмні заголовки (сегменти для завантаження), заголовки секцій, таблиці символів і релокацій. Для macOS — Mach-O. Крім нативних форматів, значна частина сучасного ЗПК використовує *проміжні* й *інтерпретовані* форми: байт-код .NET і Java, скрипти (PowerShell, JavaScript, VBA-макроси в документах), що потребують інших підходів до аналізу (розділ 13). Документи офісних форматів із макросами та експлойтами є поширеним носієм для доставки (розділ 1), тому їх структура (OLE, OOXML) також є об'єктом аналізу.

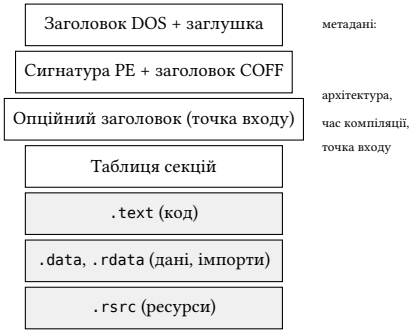


Рис. 5.1 — Структура PE-файлу (за [155])

5.2 Завантаження та виконання

Розуміння процесу завантаження є ключем до виявлення аномалій. Легітимний виконуваний файл завантажується системним завантажувачем: образ відображається в пам'ять процесу за адресою бази, виконуються релокації (коригування адрес), розв'язуються імпорти (адреси функцій заповнюються в таблиці імпортів), налаштовуються права доступу до сторінок пам'яті (код — для виконання, дані — без виконання), і керування передається точці входу. Будь-яке відхилення від цього штатного шляху є потенційним індикатором злоякісності.

Шкідливий код часто завантажується в обхід штатного механізму, щоб уникнути залишення файлу на диску й виявлення. Типові аномалії, що є ознаками для виявлення: ділянки пам'яті з правами «запис і виконання» одночасно (RWX), характерні для коду, згенерованого в пам'яті; виконання коду з ділянок, не підкріплених файлом на диску (приватна, а не відображена пам'ять); невідповідність між секціями на диску й у пам'яті; розв'язання імпортів вручну (через пошук функцій за гешами імен замість таблиці імпортів) — техніка, що приховує справжню функціональність від статичного аналізу. Сучасні засоби захисту кінцевих точок (розділ 10) моніторять саме ці аномалії завантаження, оскільки вони стійкіші до змін коду, ніж сигнатури файлів.

Окремим важливим механізмом є *динамічне розв'язання* системних функцій. Легітимна програма зазвичай оголошує свої залежності в таблиці імпортів, що робить її функціональність прозорою для статичного аналізу. Шкідливий код натомість часто завантажує бібліотеки й отримує адреси функцій під час виконання за іменами або їх гешами, тому таблиця імпортів виглядає «порожньою» або оманливо безневинною. Для аналітика це подвійний сигнал: з одного боку, бідна таблиця імпортів за наявності

складного коду сама собою підозріла; з іншого — щоб відновити справжню функціональність, потрібен динамічний аналіз (розділ 7) або емуляція. Розуміння цього механізму пояснює, чому суто статичний аналіз таблиці імпортів недостатній і чому сучасні засоби поєднують статичні й поведінкові ознаки. Ще однією поширеною технікою є виконання **шелкоду** — позиційно-незалежного коду без структури PE, що ускладнює його розпізнавання як виконуваного; виявлення шелкоду спирається на аналіз вмісту пам'яті й поведінки, а не файлової структури.

5.3 Техніки виконання й ін'єкції коду

Для прихованого виконання й доступу до ресурсів інших процесів ЗПК застосовує **ін'єкцію коду** — виконання власного коду в контексті іншого (зазвичай легітимного) процесу [118, 134, 158]. Мета — маскування (злоякісна активність виглядає як дія довіреної програми), обхід засобів контролю застосунків, доступ до даних цільового процесу (наприклад, браузера для крадіжки сесій). З позиції захисту важливо знати *сліди*, які залишають ці техніки, узагальнені в табл. 5.2 (виклад описує принцип і ознаки виявлення, а не реалізацію).

Таблиця 5.2 — Техніки ін'єкції коду та ознаки їх виявлення

Техніка (принцип)	Характерні ознаки для виявлення
Ін'єкція бібліотеки в процес	завантаження бібліотеки з незвичного шляху в чужий процес
Запис і виконання в чужому процесі	виділення пам'яті RWX у чужому процесі, створення віддаленого потоку
Підміна образу процесу (hollowing)	невідповідність образу в пам'яті файлу на диску; призупинений запуск
Відображення коду без файлу	виконання з приватної пам'яті, ручне розв'язання імпортів
Зловживання механізмами ОС	аномальне використання легітимних системних механізмів розширення

Принципова складність виявлення ін'єкції полягає в тому, що майже кожна її елементарна операція (виділення пам'яті, запис у процес, створення потоку) має легітимні застосування: налагоджувачі, засоби захисту, системи віртуалізації, менеджери вікон законно виконують подібні дії. Тому виявлення спирається не на сам факт операції, а на *нетиповий контекст*: хто саме, в який процес, за яких обставин і в якій послідовності її виконує. Наприклад, текстовий редактор, що виділяє RWX-пам'ять у процесі браузера й створює там потік, — очевидна аномалія, тоді як та сама дія від налагоджувача щодо програми, яку він налагоджує, — норма. Це робить виявлення ін'єкції задачею кореляції послідовності подій, а не розпізнава-

ння окремої дії, і пояснює, чому ефективні детектори потребують повної телеметрії процесів (розділ 10).

Для кількісної пріоритизації підозрілих подій ін'єкції застосовують *скорингові* евристики. Нехай подія характеризується набором бінарних ознак $f_1, \ldots, f_n$ (RWX-пам'ять, крос-процесність, невідповідність диску, ручні імпорти тощо) з вагами w_i. Тоді оцінка підозрілості

$$S = \sum_{i=1}^{n} w_i \, f_i, \tag{5.1}$$

а тривога генерується за $S \geq \theta$. Ваги калібрують так, щоб мінімізувати хибні тривоги на легітимній активності (деякі програми легітимно використовують подібні механізми), що знову приводить до задачі базової частоти (розділ 3): жодна окрема ознака не є достатньою, і рішення ухвалюється за сукупністю. Саме такий підхід реалізують поведінкові детектори EDR (розділ 10).

5.4 Персистентність

Щоб пережити перезавантаження й зберегти доступ, ЗПК забезпечує **персистентність** — автоматичне відновлення виконання [157, 189]. Оскільки механізмів персистентності в сучасних ОС багато, їх систематичний моніторинг є ключовим елементом виявлення. Основні класи: автозапуск через ключі реєстру й теки автозавантаження; реєстрація як системної служби; заплановані завдання; підміна чи «викрадення» порядку завантаження легітимних бібліотек; модифікація механізмів запуску застосунків; на найглибшому рівні — буткіти, що закріплюються в завантажувачі чи прошивці (розглянуто далі).

Ключова властивість для виявлення: персистентність *залишає стійкий артефакт* у системі (запис у реєстрі, файл, завдання, службу), який можна виявити інвентаризацією точок автозапуску й порівнянням із відомим чистим станом. Тому один із найефективніших методів виявлення — регулярний аудит усіх точок автозапуску (існують спеціалізовані утиліти, що перелічують десятки їх категорій) з перевіркою підписів і репутації. Це приклад загального принципу: кожна дія зловмисника, що забезпечує стійкість, створює й стійкий слід, який стає індикатором компрометації (розділ 11). Для безфайлового коду (розділ 13) персистентність реалізується в реєстрі чи через заплановані завдання без файлу на диску, що ускладнює виявлення й вимагає поведінкового моніторингу.

Вибір механізму персистентності зловмисником залежить від компромісу між *надійністю*, *скритністю* й потрібними *привілеями*. Прості механізми (теки автозавантаження, ключі автозапуску реєстру) не потребують

адміністративних прав, але легко виявляються; складніші (служби, драйвери, буткіти) потребують вищих привілеїв, зате стійкіші й менш помітні. Державні угруповання (розділ 13) надають перевагу скритним механізмам і нерідко зловживають легітимними функціями розширення системи, щоб злитися з нормальною активністю. Для захисника це означає, що виявлення персистентності не зводиться до перевірки кількох «популярних» місць: потрібне систематичне покриття всіх класів механізмів і, що важливіше, *базова лінія* нормального стану системи, відносно якої виявляються відхилення. Побудова й підтримка такої базової лінії (відомі легітимні автозапуски, підписані драйвери, звичайні заплановані завдання) є практичною основою як для ручного полювання на загрози, так і для автоматизованого виявлення (розділ 11). Саме відсутність базової лінії робить виявлення персистентності в реальних інфраструктурах складнішим, ніж здається в теорії.

5.5 Підвищення привілеїв

Для повного контролю над системою ЗПК прагне підвищити привілеї від рівня користувача до адміністратора чи ядра. Шляхи: експлуатація вразливостей ядра чи системних служб; зловживання некоректними конфігураціями (надмірні права, вразливі служби); обхід механізмів контролю доступу; крадіжка й повторне використання токенів доступу. З позиції захисту підвищення привілеїв протидіє принцип найменших привілеїв (мінімізація прав), своєчасне усунення вразливостей (розділ 10), моніторинг аномального використання привілеїв і захист облікових даних (запобігання їх вилученню з пам'яті). Виявлення спроб підвищення привілеїв є важливим сигналом, оскільки вказує на перехід атаки від початкового доступу до закріплення й розширення контролю (розділ 4).

5.6 Приховування: руткіти

Руткіт — це компонент, що приховує присутність ЗПК у системі, модифікуючи її так, щоб злоякісні файли, процеси, мережеві з'єднання й записи реєстру не відображалися звичайними засобами [117, 65]. За рівнем дії розрізняють руткіти режиму користувача (перехоплення викликів у застосунках) і режиму ядра (модифікація структур і функцій ядра — найнебезпечніші, бо діють на рівні, що контролює засоби захисту). Концептуальний приклад техніки ядра — пряма маніпуляція об'єктами ядра (приховування процесу вилученням його з переліку), що робить процес невидимим для стандартних засобів переліку, але лишає сліди в інших структурах.

Протидія руткітам ґрунтується на кількох принципах. *Перехресна перевірка* (cross-view): порівняння результатів, отриманих різними способами. Формально, якщо $\mathcal{V}_H$ — множина об'єктів (процесів, файлів, з'єднань), отри-

мана високорівневим інтерфейсом, а $\mathcal{V}_L$ — незалежним низькорівневим, то приховані руткітом об'єкти виявляються як різниця

$$\mathcal{H} = \mathcal{V}_L \setminus \mathcal{V}_H, \tag{5.2}$$

тобто об'єкти, видимі «знизу», але приховані «згори». Непорожня $\mathcal{H}$ є сильним індикатором активного приховування. Ефективність методу залежить від того, наскільки $\mathcal{V}_L$ справді незалежний від руткіта: якщо руткіт діє на рівні, нижчому за обидва інтерфейси, різниця буде порожньою, тому потрібен аналіз із максимально довіреного рівня. *Аналіз із довіреного рівня*: дослідження системи з рівня, нижчого за руткіт (завантаження з чистого носія, аналіз образу пам'яті, апаратна підтримка). *Захист цілісності ядра*: сучасні ОС застосовують обов'язковий підпис драйверів, захист цілісності коду ядра, віртуалізаційну ізоляцію критичних структур, що суттєво ускладнює руткіти режиму ядра. *Довірене завантаження*: механізми Secure Boot і вимірюваного завантаження протидіють буткітам [150], перевіряючи цілісність ланцюга завантаження від апаратного кореня довіри. Попри це, найскладніші загрози (зокрема державні, розділ 13) досягали рівня прошивки, тому аналіз пам'яті й цілісності залишається важливим напрямом криміналістики (розділ 16).

5.7 Модульна архітектура сучасного ЗПК

Сучасний злоякісний код рідко є монолітним. Типова архітектура складного ЗПК (рис. 5.2) є *модульною*: невеликий початковий компонент (dropper/loader) забезпечує доставку й закріплення, завантажує з інфраструктури зловмисника основне ядро (бекдор із C2), а воно за командою підвантажує спеціалізовані модулі (крадіжка даних, рух мережею, шифрування). Така архітектура дає зловмиснику гнучкість (оновлення й заміна модулів), а захиснику створює складність (на диску може не бути повного функціоналу, частина завантажується лише в пам'ять за командою). Водночас модульність створює й можливості для виявлення: кожен модуль має взаємодіяти з іншими, завантажуватися й спілкуватися з C2, залишаючи поведінкові сліди на різних етапах ланцюга ураження (розділ 4).



Рис. 5.2 — Модульна архітектура сучасного складного злоякісного коду

5.8 Кейс: аналіз будови типового імпланта

Розглянемо послідовність аналізу будови зразка з позиції захисника (без виконання, статично — розділ 6).

Крок 1. Формат і метадані. Визначаємо тип файлу (PE/ELF), архітектуру, час компіляції (може бути підробленим), наявність підпису. Обчислюємо криптографічний геш (для порівняння з базами) та imphash (для пошуку сімейства [149]).

Крок 2. Ентропія й пакування. Аналізуємо ентропію секцій (3.3): висока ентропія `.text` і нетипові імена секцій свідчать про пакування (розділ 8), що потребує розпакування перед глибоким аналізом.

Крок 3. Імпорти й рядки. Таблиця імпортів підказує функціональність (мережа, криптографія, ін'єкція). Рядки (URL, шляхи, повідомлення) дають індикатори, хоча часто зашифровані.

Крок 4. Персистентність і виконання. Виявляємо, які механізми закріплення й виконання використовує зразок (за імпортами, рядками, а точніше — динамічним аналізом), щоб визначити артефакти для пошуку в заражених системах.

Крок 5. Оцінка підозрілості. За сукупністю ознак (пакування, підозрілі імпорти, аномальні секції) обчислюємо оцінку (5.1) й ухвалюємо рішення про класифікацію й пріоритет.

Результат. Будова зразка дає: сімейство (за imphash і подібністю), ймовірну функціональність (за імпортами), індикатори компрометації (геші, механізми персистентності, артефакти), пріоритет реагування. Ці результати живлять правила виявлення (розділ 6) і реагування (розділ 16). Аналіз будови реальних імплантів APT-угруповань проти України систематизовано у вітчизняних дослідженнях [3], а засоби автоматизованого аналізу ПЗ на небезпечні конструкції — у [7]. Кейс ілюструє, що знання внутрішньої будови перетворює непрозорий бінарний файл на набір конкретних індикаторів для захисту.

Висновки до розділу

Розуміння внутрішньої будови й життєвого циклу шкідливого коду є фундаментом його аналізу й виявлення. Формати виконуваних файлів (PE, ELF) мають структуру, що розкриває функціональність ще до виконання: таблиця імпортів підказує можливості, а imphash дозволяє кластеризувати сімейства; аномальна ентропія й нетипові секції свідчать про пакування. Процес завантаження має штатний шлях, будь-яке відхилення від якого (RWX-пам'ять, виконання без файлу, ручні імпорти) є стійким індикатором злоякісності, стійкішим за сигнатури файлів. Техніки ін'єкції коду забезпечують маскування й доступ до чужих процесів, але залишають характерні сліди, які виявляють поведінкові детектори за сукупністю ознак (5.1).

Персистентність неминуче створює стійкий артефакт у системі, що робить аудит точок автозапуску ефективним методом виявлення. Руткіти приховують присутність модифікацією системи, і протидія їм ґрунтується на перехресній перевірці, аналізі з довіреного рівня, захисті цілісності ядра й довіреному завантаженні. Сучасний ЗПК є модульним, що дає зловмиснику гнучкість, а захиснику — додаткові поведінкові сліди на різних етапах. Головний висновок: кожна дія зловмисника, спрямована на виконання, закріплення чи приховування, створює артефакт або поведінкову ознаку, і саме знання будови перетворює ці ознаки на конкретні індикатори для захисту.

Питання для самоперевірки

1. Опишіть структуру PE-файлу (рис. 5.1). Яку інформацію дає таблиця імпортів і що таке imphash?
2. Чому висока ентропія секції `.text` і нетипові імена секцій є ознаками пакування (табл. 5.1)?
3. Опишіть штатний процес завантаження виконуваного файлу. Які відхилення є індикаторами злоякісності?
4. Чому ознаки аномалій завантаження (RWX, виконання без файлу) стійкіші за сигнатури файлів?
5. З якою метою ЗПК застосовує ін'єкцію коду? Які сліди вона залишає (табл. 5.2)?
6. Поясніть скорингову евристику (5.1). Чому жодна окрема ознака ін'єкції не є достатньою для тривоги?
7. Назвіть класи механізмів персистентності. Чому персистентність завжди залишає стійкий артефакт і як це використовують для виявлення?
8. Чому аудит точок автозапуску є ефективним методом виявлення? Як ускладнює виявлення безфайлова персистентність?
9. Якими шляхами ЗПК підвищує привілеї? Які заходи протидіють підвищенню привілеїв?
10. Що таке руткіт? Порівняйте руткіти режиму користувача й ядра за рівнем небезпеки.
11. Опишіть принципи протидії руткітам. Як довірене завантаження протидіє буткітам?
12. Опишіть модульну архітектуру сучасного ЗПК (рис. 5.2). Які переваги вона дає зловмиснику й які можливості — захиснику?
13. Опишіть послідовність аналізу будови зразка (кейс підрозділу 5.8). Які індикатори вона дає для захисту?

14. Дослідницьке: розробіть методику автоматичного виявлення аномалій завантаження (RWX-пам'ять, невідповідність диску) у телеметрії кінцевих точок і оцініть її частку хибних тривог з урахуванням легітимних випадків (розділ 3).
15. Дослідницьке: побудуйте систему кластеризації зразків за imphash і подібністю імпортів для автоматичної атрибуції сімейств та оцініть її стійкість до технік зміни імпортів (розділ 8).

РОЗДІЛ 6

Статичний аналіз та сигнатурне виявлення

Статичний аналіз — дослідження зразка без його виконання — є історично першим і досі базовим підходом до виявлення й вивчення злоякісного коду. Він дешевий, швидкий, безпечний (без запуску реального коду) і дозволяє досліджувати всі гілки програми, а не лише виконані. На статичному аналізі ґрунтується сигнатурне виявлення, що залишається першим рубежем більшості антивірусних засобів, а також поглиблене дослідження зразків аналітиками. Водночас статичний аналіз має принципові межі: нерозв'язність точного аналізу коду (розділ 3) і техніки ухилення — пакування, обфускація, шифрування (розділ 8), — що приховують справжній код. Розділ розглядає рівні статичного аналізу, сигнатурне виявлення й мову правил YARA, дизасемблювання й аналіз коду, виявлення за подібністю й кластеризацію сімейств, евристичний статичний аналіз, його межі, і завершується кейсом розроблення правила виявлення для сімейства ЗПК. Динамічний аналіз, що доповнює статичний, розглянуто в розділі 7.

6.1 Місце й рівні статичного аналізу

Статичний аналіз утворює кілька рівнів зростаючої глибини (табл. 6.1), від швидкого поверхневого до трудомісткого аналізу коду. На практиці аналіз є ітеративним: поверхневі ознаки визначають пріоритет і напрям глибшого дослідження, а результати живлять правила виявлення (розділ 11) і реагування (розділ 16). Конвеєр статичного аналізу з ескалацією глибини наведено на рис. 6.1.



Рис. 6.1 — Конвеєр статичного аналізу з ескалацією глибини

Таблиця 6.1 — Рівні статичного аналізу злоякісного коду

Рівень	Що досліджує	Вартість
Поверхневий	геші, рядки, метадані, імпорти, ентропія	дуже низька
Сигнатурний	збіг з відомими патернами	низька
Структурний	секції, ресурси, подібність, imphash	низька
Аналіз коду	дизасемблювання, граф потоку керування	висока
Семантичний	поведінкова модель, абстрактне виконання	дуже висока

6.2 Поверхневий статичний аналіз

Поверхневий аналіз дає швидкі індикатори без розбору коду. *Криптографічні геші* (SHA-256) однозначно ідентифікують відомий зразок і використовуються для пошуку в репутаційних базах; проте, як зазначено в розділі 3, зміна одного байта повністю змінює геш, тому вони непридатні для виявлення варіантів. *Рядки* (текстові фрагменти: URL, шляхи, повідомлення, імена функцій) часто розкривають функціональність та індикатори, хоча в сучасному ЗПК зазвичай зашифровані. *Метадані PE* (час компіляції, компілятор, підпис, версійна інформація) дають контекст і ознаки підробки. *Імпорти й imphash* (розділ 5) підказують можливості й кластеризують сімейства. *Ентропія секцій* (3.3) виявляє пакування. Поєднання цих ознак у ваговій оцінці дає перший фільтр, що відсіює очевидно підозрілі зразки для глибшого аналізу й підвищує ефективну базову частоту на наступних етапах (розділ 3).

Практична цінність поверхневого аналізу полягає не лише в самих ознаках, а в їх *зіставленні* з контекстом. Наприклад, високе значення ентропії секції коду саме собою не є доказом злоякісності: багато легітимних інсталяторів і захищених комерційних програм так само спаковані або зашифровані. Вагомість ознаки визначає відхилення від очікуваного для цього класу файлів: PE-файл, що оголошує себе системною утилітою, але не має цифрового підпису, має час компіляції в майбутньому й імпортує лише `LoadLibrary` та `GetProcAddress` (ознака динамічного розв'язання імпортів для приховування можливостей), значно підозріліший за спакований, але належно підписаний інсталятор. Тому зрілі конвеєри поверхневого аналізу працюють не з абсолютними порогами, а з профілями нормальності й репутаційними даними, накопиченими з потоку мільйонів файлів.

Окремо відзначимо роль рядків як джерела *слідчих* індикаторів. Навіть коли основне корисне навантаження зашифроване, у зразку часто залишаються непрямі сліди: фрагменти шляхів складання (що видають середовище розробника та іноді мову інтерфейсу), назви бібліотек часу виконання, рядки повідомлень про помилки, адреси командних серверів, шаблони мьютексів для запобігання повторному запуску. Ці артефакти живлять як

сигнатури й правила YARA, так і атрибуцію кампаній (розділ 12). Саме тому автори ЗПК докладають зусиль до мінімізації рядків — шифрують константи, збирають рядки на стеку побайтово, завантажують їх з ресурсів під час виконання, — а аналітики, відповідно, розробляють методи їх відновлення (емуляція, деобфускація рядків).

6.3 Сигнатурне виявлення

6.3.1 Принцип і типи сигнатур

Сигнатура — це формалізований опис ознак, наявність яких свідчить про належність зразка до відомої загрози. Сигнатурне виявлення зіставляє досліджуваний об'єкт із базою сигнатур. Типи сигнатур: *байтові* (послідовності байтів, характерні для зразка); *геш-сигнатури* (точні геші файлів чи їх фрагментів); *структурні* (ознаки формату, секцій, імпортів); *правила* (логічні комбінації умов, як у YARA). Класичні антивірусні рушії (наприклад, ClamAV [84]) підтримують кілька форматів сигнатур.

Порівняння типів сигнатур за покриттям і стійкістю наведено в табл. 6.2.

Таблиця 6.2 — Типи сигнатур та їхні властивості

Тип	Принцип	Покриття варіантів
Геш-сигнатура	точний геш файлу	жодного (точний збіг)
Байтова	послідовність байтів коду	низьке (ламається пакуванням)
Структурна	секції, імпорти, формат	помірне
Правило (YARA)	логіка над ознаками	помірне–високе
Семантична	модель поведінки з коду	високе

Принципова перевага сигнатур — висока точність (низька частота хибних тривог) для *відомих* загроз і висока швидкість. Принципова вада — неспроможність виявляти нові й змінені зразки: як показано в розділі 1, за експоненційного зростання кількості зразків (1.1) сигнатурне покриття принципово відстає. Крім того, поліморфізм і пакування (розділ 8) роблять байтові сигнатури неефективними. Тому сигнатури залишаються необхідним, але недостатнім першим рубежем, що доповнюється поведінковими й статистичними методами (розділи 7, 9). Варто підкреслити, що сигнатурний підхід зберігає цінність саме завдяки високій точності: за величезних потоків файлів (розділ 3) дешевий і точний перший рубіж, що впевнено блокує відомі загрози, звільняє дорогі поведінкові й статистичні методи для аналізу решти. Тому раціональна архітектура захисту не протиставляє

сигнатури й поведінку, а вибудовує їх каскадом із зростанням вартості й покриття.

6.3.2 Мова правил YARA

Де-факто стандартом гнучких сигнатур став YARA [234, 233] — мова опису правил, що поєднує текстові й байтові патерни з логічними умовами. Правило YARA складається з розділів: метаданих (опис, автор, посилання), рядків (патерни — текстові, шістнадцяткові з масками, регулярні вирази) й умови (логічний вираз над наявністю патернів, їх кількістю, позицією, а також властивостями файлу). Умова може враховувати формат PE, розмір, ентропію, зміщення патернів. Приклад логіки правила (схематично, без працездатного коду): «файл є PE `AND` містить щонайменше два з трьох характерних рядків `AND` має imphash зі списку `AND` ентропія секції коду > 7». Така комбінація різнорідних ознак дає баланс між покриттям (виявляє варіанти сімейства) і точністю (уникає хибних тривог).

Сила YARA — саме в *модульності* умови. Окреме правило може посилатися на модуль `pe` (щоб перевіряти характеристики заголовка, секцій, таблиці імпортів, цифрового підпису), модуль `math` (для обчислення ентропії довільного діапазону), модуль `hash` (для гешів фрагментів). Це дозволяє висловлювати ознаки різної природи в одному логічному виразі: наприклад, вимагати, щоб точка входу лежала в секції з аномально високою ентропією *й* водночас таблиця імпортів містила характерний для сімейства набір функцій. Така виразність робить YARA мовою не лише антивірусних лабораторій, а й центрів реагування (розділ 16) та платформ аналізу загроз, де правила обмінюються між організаціями як машиночитані індикатори (розділ 12).

Водночас гнучкість YARA покладає відповідальність на автора правила. Поширена помилка початківця — писати правило за *одним* зразком, механічно копіюючи в нього довгі байтові послідовності, що насправді є фрагментами стандартної бібліотеки компілятора або ресурсами. Таке правило або не виявляє жодного варіанта, крім вихідного (надмірна специфічність), або спрацьовує на всіх програмах, зібраних тим самим компілятором (хибні тривоги). Зріле правило спирається на ознаки, що відображають *задум* автора ЗПК (специфічну логіку, унікальні константи алгоритмів, характерні послідовності системних викликів), а не на артефакти інструментарію складання.

Якість правила вимірюють тими самими метриками, що й детектор (розділ 3): повнотою (частка зразків сімейства, що спрацьовують) і точністю (частка істинних серед спрацювань). Ризик надто специфічного правила — пропуск варіантів (низька повнота); надто загального — хибні тривоги на легітимних файлах (низька точність). Для оцінювання ймовірності випад-

кового збігу байтової сигнатури довжиною n байтів на випадкових даних використовують оцінку

$$P_{\text{випадк}} \approx (L - n + 1) \cdot 256^{-n}, \tag{6.1}$$

де L — розмір файлу. Формула (6.1) показує, чому короткі сигнатури (малі n) дають багато хибних збігів: кожен додатковий байт зменшує ймовірність випадкового спрацювання в 256 разів, тому надійні байтові патерни мають довжину щонайменше кількох байтів специфічного коду, а не загальних послідовностей.

6.4 Дизасемблювання та аналіз коду

Глибокий аналіз потребує відновлення логіки коду через **дизасемблювання** — перетворення машинного коду на асемблерні інструкції, а далі — на граф потоку керування (CFG) і, за можливості, псевдокод. Інструменти (IDA Pro [94], Ghidra, Binary Ninja, радару-подібні) автоматизують цей процес, але він має принципові труднощі [53]: розрізнення коду й даних (на архітектурі зі змінною довжиною інструкцій неможливе точно в загальному випадку); непрямі переходи (адреса обчислюється під час виконання); самомодифікований код; антидизасемблерні прийоми (розділ 8). Ці труднощі є проявом нерозв'язності (розділ 3). На практиці застосовують два базові алгоритми дизасемблювання: *лінійний прохід* (послідовне декодування байтів від початку секції), простий, але легко збиваний вставленням «сміттєвих» байтів між інструкціями, і *рекурсивний обхід* (декодування за потоком керування, від точки входу за переходами й викликами), точніший, але безсилий перед непрямими переходами. Сучасні інструменти поєднують обидва з евристиками розпізнавання функцій, тому результат дизасемблювання реального зразка завжди слід розглядати як *обґрунтоване припущення*, а не беззаперечну істину — особливо для навмисно обфускованого коду.

Центральним поданням для аналізу коду є **граф потоку керування** (CFG) — орієнтований граф, вершини якого є базовими блоками (лінійними послідовностями інструкцій), а дуги — можливими переходами (рис. 6.2). CFG дозволяє зрозуміти структуру програми, виявити цикли (наприклад, цикл шифрування у вимагачі), умовні гілки (перевірки середовища антианалізу), недосяжний код. На основі CFG будують граф викликів функцій, що відображає архітектуру програми й використовується для структурної подібності (BinDiff). Обфускація (розділ 8) цілеспрямовано ускладнює CFG (вставлянням зайвих гілок, непрозорими предикатами, перехопленням потоку керування), тому відновлення зрозумілого CFG з обфускованого коду є окремою задачею деобфускації.

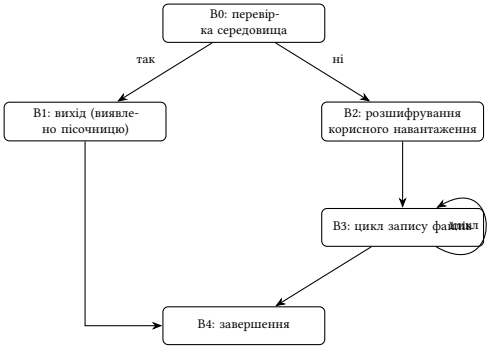


Рис. 6.2 — Спрощений граф потоку керування фрагмента вимагача: умовне розгалуження за результатом антианалізної перевірки (B0) та цикл шифрування файлів (B3) як характерна структурна ознака

Семантичний підхід до виявлення [79] будує абстрактну модель поведінки з коду, що стійкіша до поверхневих змін, ніж байтові сигнатури, бо відображає *що* код робить, а не *як* він записаний. Платформи бінарного аналізу [69] надають проміжні подання (intermediate representation) для автоматизованих перетворень і міркувань про семантику незалежно від конкретної архітектури процесора. Проте повний семантичний аналіз дорогий і обмежений обфускацією, тому застосовується вибірково — для пріоритетних зразків і розроблення стійких правил. Поєднання статичного й динамічного підходів (гібридний аналіз) долає частину обмежень кожного [23].

6.5 Виявлення за подібністю та кластеризація

Оскільки ЗПК існує сімействами, ключовим є виявлення за *подібністю* до відомих зразків, а не точним збігом. Міри подібності (розділ 3): нечіткі геші (ssdeep, TLSH [172]), що дають близькі значення для схожих файлів; подібність множин ознак (імпортів, n-грамів [135]) за коефіцієнтом Жаккара (3.4); структурна подібність графів функцій (BinDiff [93]). Для масштабованого порівняння мільйонів зразків застосовують хешування ознак (BitShred [129]) і локально-чутливе хешування.

Ці міри різняться не лише обчислювальною вартістю, а й *рівнем абстракції*, на якому фіксують подібність, і, відповідно, стійкістю до видів зміни зразка. Нечіткі геші працюють на рівні байтів і добре виявляють дрібні правки (виправлення, перекомпіляцію з незначними змінами), але легко обманюються перепакуванням або зміною секцій. Подібність множин імпортів абстрагується від розташування коду й стійкіша до перекомпіляції, проте нечутлива до внутрішньої логіки. Структурна подібність графів функцій працює на рівні семантики потоку керування й найстійкіша до поверх-

невих перетворень, але найдорожча й сама вразлива до спрямованої обфускації CFG (розділ 8). На практиці системи триажу комбінують кілька мір: дешеві (нечіткі геші, imphash) для грубого попереднього групування мільйонів зразків, дорогі (графова подібність) — лише всередині попередньо сформованих кластерів. Такий багаторівневий підхід робить порівняння за подібністю масштабованим на промислових колекціях.

Кластеризація групує зразки за подібністю, виявляючи сімейства й кампанії. Це має подвійну цінність: автоматична атрибуція нового зразка до відомого сімейства (пришвидшує реагування) і виявлення нових сімейств (як щільних груп раніше невідомих зразків). Якість кластеризації оцінюють мірами чистоти кластерів відносно відомої розмітки. Для оцінювання відстані між зразками, поданими векторами ознак $\mathbf{x}$, $\mathbf{y}$, часто використовують косинусну подібність:

$$\mathrm{sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \, \|\mathbf{y}\|} \in [0; 1], \tag{6.2}$$

що нечутлива до масштабу й добре працює для розріджених ознакових векторів (частоти n-грамів, наявність функцій). Кластеризація й подібність є містком між статичним аналізом і машинним навчанням (розділ 9), де ознаки подібності стають входом класифікаторів.

6.6 Евристичний статичний аналіз та його межі

Евристичний статичний аналіз шукає не конкретні зразки, а *ознаки підозрілості*, характерні для ЗПК узагалі: підозрілі комбінації імпортів, аномальні секції, високу ентропію, ознаки пакування, нетипову структуру. Евристики виявляють нові зразки, яких немає в базі сигнатур, але ціною вищої частоти хибних тривог (легітимні програми теж пакують, шифрують, використовують «підозрілі» функції), що знову приводить до проблеми базової частоти (розділ 3). Тому евристики калібрують і поєднують з іншими методами.

Межі статичного аналізу визначаються двома чинниками. *Теоретичний*: нерозв'язність точного аналізу (розділ 3) — будь-який статичний аналізатор дає наближення з неминучими помилками. *Практичний*: техніки ухилення (розділ 8) — пакування й шифрування приховують справжній код, роблячи статичний аналіз файлу на диску марним без розпакування. Саме тому статичний аналіз доповнюють динамічним (розділ 7): запуск у контрольованому середовищі розгортає справжній код у пам'яті, після чого його можна аналізувати статично (аналіз дампа пам'яті). Порівняння методів тестування безпеки застосунків (статичних і динамічних) систематизовано у вітчизняних дослідженнях [45], а застосування машинного навчання в статичному аналізі — у [31].

6.7 Кейс: розроблення правила виявлення для сімейства

Розглянемо послідовність розроблення правила YARA для виявлення сімейства ЗПК (захисна задача: створення детектора на основі аналізу відомих зразків).

Крок 1. Збір і кластеризація зразків. За подібністю (6.2) та imphash формуємо набір зразків одного сімейства й контрольний набір легітимних і сторонніх файлів.

Крок 2. Виділення спільних ознак. Знаходимо ознаки, спільні для сімейства й відсутні в контрольному наборі: характерні рядки (навіть зашифровані константи), послідовності коду, imphash, структурні особливості. Пріоритет — ознакам, що відображають семантику (вищий рівень «піраміди болю», розділ 2), бо вони стійкіші до варіацій.

Крок 3. Формулювання умови. Будуємо логічну умову, що вимагає збігу кількох незалежних ознак (а не однієї), щоб забезпечити точність. За (6.1) вибираємо достатньо специфічні байтові патерни.

Крок 4. Оцінювання. Вимірюємо повноту на наборі сімейства й точність на контрольному наборі. За потреби коригуємо баланс: послаблюємо умову для підвищення повноти або посилюємо для зниження хибних тривог. Критично перевірити правило на великому наборі легітимних файлів, щоб уникнути хибних спрацювань у продакшні.

Крок 5. Супровід. Правило старіє в міру появи нових варіантів, тому потребує моніторингу ефективності й оновлення. Стійкіші правила, засновані на семантиці, живуть довше за ті, що спираються на поверхневі байти.

Кейс ілюструє ключовий принцип: ефективне сигнатурне виявлення — це не механічний запис байтів одного зразка, а узагальнення спільних, семантично значущих ознак сімейства з балансуванням повноти й точності та постійним супроводом. Такий підхід поєднує переваги сигнатур (точність, швидкість) із частковим покриттям варіантів, але не замінює поведінкових методів для принципово нових загроз.

Висновки до розділу

Статичний аналіз — дешевий, швидкий і безпечний базовий підхід до виявлення й вивчення ЗПК, що охоплює всі гілки коду й утворює кілька рівнів від поверхневого до семантичного. Поверхневий аналіз (геші, рядки, метадані, імпорти, ентропія) дає швидкі індикатори й перший фільтр. Сигнатурне виявлення точне й швидке для відомих загроз, але принципово відстає від нових і безсиле проти поліморфізму й пакування; мова YARA забезпечує гнучкі правила, що поєднують різнорідні ознаки, якість яких балансує повноту й точність (6.1). Дизасемблювання й аналіз коду розкривають семантику зразка, але мають принципові труднощі, зумовлені нерозв'язністю, а семантичне виявлення стійкіше до поверхневих змін. Вияв-

лення за подібністю й кластеризація (6.2) дозволяють працювати із сімействами, автоматично атрибутувати нові зразки й виявляти нові сімейства, слугуючи містком до машинного навчання. Евристичний аналіз виявляє невідомі зразки ціною хибних тривог. Межі статичного аналізу — теоретична нерозв'язність і практичні техніки ухилення — долаються поєднанням із динамічним аналізом. Головний висновок: статичний аналіз є необхідним, але недостатнім; його сила — у поєднанні рівнів і узагальненні семантично значущих ознак, а не в механічному зіставленні байтів.

Питання для самоперевірки

1. Опишіть рівні статичного аналізу (табл. 6.1). Чому аналіз є ітеративним?
2. Які індикатори дає поверхневий аналіз? Чому криптографічні геші непридатні для виявлення варіантів?
3. Що таке сигнатура? Назвіть типи сигнатур і поясніть переваги й вади сигнатурного виявлення.
4. Опишіть структуру правила YARA. Чому поєднання різнорідних ознак дає баланс повноти й точності?
5. За формулою (6.1) поясніть, чому короткі байтові сигнатури дають багато хибних збігів. Обчисліть $P_{\text{випадк}}$ для $n = 8$, $L = 10^6$.
6. Які принципові труднощі має дизасемблювання? Як вони пов'язані з нерозв'язністю (розділ 3)?
7. Що таке семантичне виявлення й чому воно стійкіше до поверхневих змін, ніж байтові сигнатури?
8. Які міри подібності застосовують для виявлення варіантів? Поясніть косинусну подібність (6.2).
9. Яку подвійну цінність має кластеризація зразків? Як її якість оцінюють?
10. У чому перевага й вада евристичного статичного аналізу? Як проблема базової частоти впливає на його застосування?
11. Які теоретичні й практичні чинники обмежують статичний аналіз? Як їх долають поєднанням з динамічним?
12. Опишіть послідовність розроблення правила виявлення для сімейства (кейс підрозділу 6.7). Чому пріоритет надають семантично значущим ознакам?
13. Чому ефективне сигнатурне виявлення не є механічним записом байтів одного зразка?

14. Дослідницьке: розробіть правило виявлення для обраного сімейства ЗПК, оцініть його повноту й точність на незалежних наборах і дослідіть стійкість до змін зразків (розділ 8).
15. Дослідницьке: порівняйте нечіткі геші (ssdeep, TLSH) і структурну подібність (BinDiff) за точністю кластеризації сімейств і обчислювальною вартістю на великій колекції зразків.

РОЗДІЛ 7

Динамічний та поведінковий аналіз шкідливого коду

Динамічний аналіз — дослідження зразка під час його виконання в контрольованому середовищі — доповнює статичний аналіз (розділ 6) там, де той безсилий: проти пакування, шифрування та обфускації (розділ 8), що приховують справжній код на диску. Під час виконання код мусить розгорнути себе в пам'яті й виявити справжню поведінку — саме цю поведінку й спостерігає динамічний аналіз. Його сила — у тому, що він фіксує *що* зразок робить, незалежно від того, *як* цей код записаний і захований; його слабкість — у тому, що спостерігається лише *одна* гілка виконання за даних умов середовища, а не всі можливі (на відміну від статичного аналізу), і в тому, що сучасний ЗПК активно виявляє середовище спостереження й ухиляється від нього. Розділ розглядає принципи й місце динамічного аналізу, архітектуру пісочниць, спостережувані артефакти поведінки, аналіз пам'яті, формалізацію й кластеризацію поведінки, межі методу й протидію антианалізним прийомам, і завершується кейсом дослідження зразка-вимагача. Поведінкове виявлення, що ґрунтується на динамічних ознаках, є основою сучасних засобів захисту кінцевих точок (розділ 10).

7.1 Принципи й місце динамічного аналізу

Ідея динамічного аналізу проста: замість того щоб розбирати код, його *запускають* у середовищі, де кожну дію можна спостерігати й за потреби обернути. Це знімає проблему прихованого коду, бо розпакування й розшифрування відбуваються автоматично — їх виконує сам зразок. Натомість з'являються нові виклики: безпека (виконання реального ЗПК потребує ізоляції), повнота (одне виконання розкриває лише одну гілку), вірогідність (середовище спостереження має бути нерозрізнюваним від реального, інакше зразок не виявить справжньої поведінки). Систематичний огляд методів і засобів динамічного аналізу [95] виокремлює ці компроміси як визначальні для галузі.

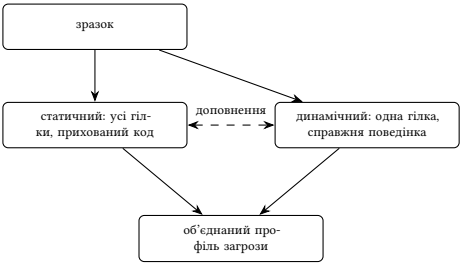


Рис. 7.1 — Взаємодоповнення статичного й динамічного аналізу: кожен долає обмеження іншого

Порівняння двох підходів наведено в табл. 7.1. Головний висновок: вони не конкурують, а доповнюють один одного, і зрілий аналіз є **гібридним** — статичний спрямовує динамічний (визначає точки інтересу, умови запуску), а динамічний розгортає код для подальшого статичного аналізу дампа пам'яті.

Таблиця 7.1 — Статичний та динамічний аналіз: порівняння

Властивість	Статичний	Динамічний
Покриття гілок	усі гілки коду	лише виконані
Прихований код	вразливий до пакування	розгортається самим зразком
Безпека	код не виконується	потрібна ізоляція
Швидкість	висока	обмежена часом виконання
Ухилення	антидизасемблювання	антисендбокс, антиВМ

7.2 Пісочниці та середовища спостереження

Пісочниця (sandbox) — ізольоване інструментоване середовище для виконання й спостереження зразка. Історично першими автоматизованими пісочницями були системи на кшталт CWSandbox [227] та Cuckoo [89], що перехоплювали системні виклики й будували звіт про поведінку. Архітектурно пісочниці різняться за *рівнем* інструментування та *рівнем* ізоляції.

За рівнем спостереження розрізняють: *перехоплення в користувацькому просторі* (хуки API-функцій; просте, але легко виявляється й обходиться прямими системними викликами); *перехоплення в ядрі* (драйвери-монітори; стійкіше, але вразливе до руткітів рівня ядра, розділ 5); *емуляцію та віртуалізацію з інтроспекцією* (аналіз ззовні гостьової системи через гіпервізор — virtual machine introspection, що найважче виявити зсередини). За середовищем виконання — емулятори, віртуальні машини та «голе залізо» (bare-metal): останнє застосовують саме проти зразків, які виявляють віртуалізацію [133].

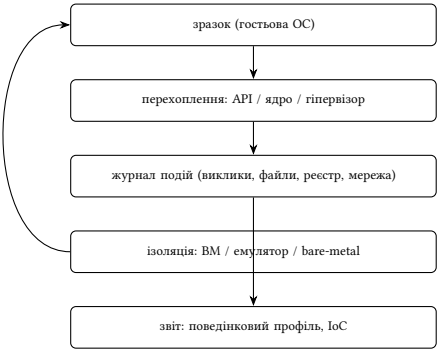


Рис. 7.2 — Узагальнена архітектура пісочниці для динамічного аналізу

Якість пісочниці визначається двома суперечливими вимогами: *прозорістю* (невидимістю для зразка) та *повнотою спостереження*. Що глибше інструментування, то повніший звіт, але й то помітніше втручання. Що краще ізоляція імітує реальну систему (встановлені програми, історія діяльності користувача, реалістична мережа), то ймовірніше зразок виявить справжню поведінку.

Окремо слід розрізняти *інтерактивний* аналіз, який веде аналітик за допомогою зневаджувача (debugger), покроково контролюючи виконання й оглядаючи стан, та *автоматизований* аналіз у пісочниці, що масштабується на тисячі зразків за добу без участі людини. Перший незамінний для глибокого дослідження пріоритетних зразків і подолання складних прийомів антианалізу; другий — основа промислового триажу потоку загроз (розділ 3). На практиці вони утворюють каскад, аналогічний ескалації глибини статичного аналізу (розділ 6): дешевий автоматизований прохід відсіює масу й визначає пріоритети, а дорогий ручний аналіз застосовують до того, що пройшло фільтр або чинить опір автоматиці.

Важливою архітектурною проблемою пісочниці є *керування мережею*. Повна ізоляція без мережі позбавляє спостереження багатьох дій (звернення до командного сервера, завантаження додаткових модулів), бо зразок, не отримавши відповіді, завершується. Повний доступ до інтернету небезпечний (зразок може атакувати сторонні системи, а оператори — виявити аналіз за зверненням з відомої аналітичної мережі й вимкнути сервер). Тому застосовують імітацію мережевих служб (підроблені відповіді DNS, HTTP, служб командного керування), що дозволяє спостерегти наміри зразка, не випускаючи трафік назовні, — компроміс між повнотою й безпекою.

7.3 Спостережувані артефакти поведінки

Поведінка зразка проявляється у взаємодії з операційною системою та мережею. Основні категорії спостережуваних артефактів: *виклики API та системні виклики* (найбагатше джерело — відображають наміри: робота з файлами, процесами, пам'яттю, реєстром, мережею, криптографією); *файлова активність* (створення, зміна, видалення, шифрування файлів — ключова ознака вимагачів); *зміни реєстру* (персистентність, конфігурація — розділ 5); *мережева активність* (звернення до командних серверів, вивантаження даних, завантаження додаткових модулів); *процесна активність* (породження процесів, ін'єкції, маніпуляції з токенами). Зіставлення цих артефактів із тактиками й техніками MITRE ATT&CK [159] дає структурований поведінковий профіль.

Особливу цінність має *послідовність* і *контекст* дій, а не окремі виклики. Один виклик шифрувальної функції нічого не означає; послідовність «перелік файлів → читання → шифрування → перезапис → видалення тіньових копій → створення записки про викуп» однозначно ідентифікує вимагача. Тому поведінкові сигнатури описують не окремі дії, а їх патерни — подібно до того, як правила YARA (розділ 6) описують статичні ознаки.

Виклики API та системні виклики заслуговують окремої уваги як найбагатше й найстійкіше джерело. Будь-яка значуща дія — робота з файлом, мережеве з'єднання, запис у реєстр, виділення виконуваної пам'яті, створення процесу — рано чи пізно звертається до операційної системи через чітко визначений інтерфейс, який неможливо обійти, не відмовившись від самої дії. Це робить рівень системних викликів природним «вузьким місцем» для спостереження: хоч би як був обфускований код, його взаємодія зі світом проходить тут. Саме тому автори ЗПК застосовують прямі системні виклики в обхід перехоплених бібліотечних функцій, а розробники пісочниць переносять перехоплення глибше — у ядро й гіпервізор, — щоб це вузьке місце лишалося під контролем.

Мережева активність має подвійну цінність. По-перше, вона розкриває інфраструктуру зловмисника — адреси й домени командних серверів, протоколи зв'язку, формати маячкових повідомлень (beacon), — що безпосередньо живить мережеві індикатори компрометації (розділ 12) та виявлення на рівні мережі (розділ 10). По-друге, патерни трафіку (регулярність маячків, обсяг вивантажених даних, характерні заголовки) самі є поведінковою ознакою навіть за шифрованого вмісту. Тому мережеве спостереження — невіддільна частина динамічного аналізу, а не окрема дисципліна.

7.4 Аналіз пам'яті

Аналіз пам'яті (memory forensics) досліджує знімок оперативної пам'яті системи й посідає особливе місце: пам'ять містить *розгорнутий* код (пі-

сля розпакування), розшифровані дані, ключі, ін'єктовані модулі, мережеві з'єднання, сліди безфайлового коду, яких немає на диску. Фреймворк Volatility [216, 144] став де-факто стандартом: він відновлює списки процесів (зокрема приховані руткітами через порівняння різних джерел — cross-view, розділ 5), завантажені модулі, мережеві з'єднання, ін'єктовані ділянки пам'яті (з аномальними правами виконання), дескриптори.

Аналіз пам'яті критичний для виявлення *безфайлового* коду (розділ 11), що існує лише в пам'яті й не залишає файлу на диску, а отже невидимий для статичного аналізу файлів. Він також дозволяє перемогти пакування пасивно: замість розпакування вручну аналітик дозволяє зразку розпакуватися самому, після чого знімає дамп і аналізує вже відкритий код статичними методами. Це робить аналіз пам'яті містком між динамічним і статичним аналізом.

Методологічно аналіз пам'яті спирається на реконструкцію структур ядра операційної системи зі «сирого» знімка: списків процесів і потоків, таблиць об'єктів, структур мережевого стеку, кешів файлової системи. Порівняння кількох незалежних джерел тієї самої інформації (cross-view, розділ 5) — наприклад, списку процесів зі службового переліку диспетчера проти обходу структур ядра напряму — викриває приховування: об'єкт, відсутній в одному поданні, але наявний в іншому, майже напевно прихований руткітом. Так аналіз пам'яті перетворює саме зусилля ЗПК зі приховування на детектувальну ознаку. Обмеження методу — миттєвість знімка (він фіксує лише один момент і може не захопити короткочасні події) та потреба в надійному, незіпсованому самим ЗПК механізмі зняття дампа, бо руткіт рівня ядра здатен спотворити знімок, зроблений зсередини системи.

7.5 Формалізація та кластеризація поведінки

Для автоматизованого порівняння поведінку формалізують. Поширене подання — *множина* або *послідовність* поведінкових ознак (викликів API, системних викликів, дій над ресурсами). Подібність поведінки двох зразків A і B, поданих множинами ознак, оцінюють коефіцієнтом Жаккара (3.4), а для послідовностей — мірами на n-грамах викликів. Поведінкова кластеризація [62, 60] групує зразки за схожістю динамічних профілів, що стійкіше до поверхневих статичних змін, бо спирається на семантику дій.

Важливою характеристикою є **покриття** виконання — частка коду (чи гілок), досягнута під час спостереження. Якщо позначити множину можливих семантичних гілок $\mathcal{B}$, а досягнуту за один запуск — $\mathcal{B}_{\text{run}}$, то покриття

$$\text{Cov} = \frac{|\mathcal{B}_{\text{run}}|}{|\mathcal{B}|} \in [0;1] \tag{7.1}$$

фундаментально обмежене: умовний код (зокрема логічні бомби, що спрацьовують за датою чи командою) та залежність від середовища залишають частину поведінки неспостереженою. Для підвищення покриття застосовують багаторазові запуски за різних умов та техніки примусового дослідження кількох гілок (multi-path execution).

Поведінкові профілі, отримані динамічно, мають і самостійну цінність для автоматичної класифікації. На відміну від статичних ознак, вони стійкі до пакування й поверхневих змін коду, бо відображають *дії*, а не запис. Два зразки одного сімейства, скомпільовані по-різному чи запаковані різними пакувальниками (розділ 8), статично виглядають несхоже, але динамічно демонструють ту саму послідовність дій — ту саму роботу з файлами, реєстром, мережею. Саме тому поведінкова кластеризація [62] стала основою масштабного триажу: вона групує функційно подібні зразки попри статичні відмінності, автоматично відносячи нові зразки до відомих сімейств. Обмеження — та сама неповнота: якщо в спостереженому запуску сімейство не проявило характерної поведінки (через ухилення чи невиконану гілку), профіль буде неповним, а класифікація — ненадійною. Тому поведінкову кластеризацію, як і решту динамічних методів, посилюють багаторазовим спостереженням і поєднанням зі статичними ознаками.

Ефективність спроб ухилення зручно описувати через імовірність того, що зразок *не* виявить середовище спостереження. Якщо зразок застосовує k незалежних перевірок, кожна з яких викриває пісочницю з імовірністю p_i, то ймовірність лишитися непоміченим і, отже, спостерегти справжню поведінку:

$$P_{\text{спост}} = \prod_{i=1}^{k} (1 - p_i). \tag{7.2}$$

Формула (7.2) пояснює стратегію вдосконалення пісочниць: зменшення кожного p_i (підвищення прозорості окремих аспектів) та, у разі стійких до ВМ зразків, перехід на bare-metal, де низка перевірок віртуалізації дає $p_i \to 0$.

7.6 Межі методу та протидія ухиленню

Головна вада динамічного аналізу — неповнота: спостерігається лише виконана гілка за конкретних умов. Логічна бомба, що чекає певної дати чи команди, у пісочниці не спрацює й буде визнана нешкідливою. Друга проблема — **антисендбокс** та **антиВМ** прийоми [75, 145]: зразок виявляє ознаки середовища спостереження (артефакти віртуалізації, малий час роботи системи, відсутність активності користувача, характерні імена пристроїв, присутність інструментів аналізу) і, виявивши їх, приховує справжню по-

ведінку — завершується, засинає або виконує безпечний код. Виявлення чутливих до середовища зразків [145] ґрунтується на порівнянні поведінки в різних середовищах: розбіжність сама є ознакою ухилення.

Арсенал антисендбокс-прийомів зручно поділити на кілька груп. *Виявлення артефактів віртуалізації*: характерні рядки в описах пристроїв, специфічні інструкції процесора й розбіжності в їх поведінці, присутність драйверів і служб гіпервізора. *Виявлення інструментування*: наявність перехоплень у функціях, процеси відомих аналітичних засобів, невідповідності в поверненнях API, спричинені хуками. *Аналіз обстановки*: малий обсяг пам'яті чи кількість ядер, мало недавно відкритих документів і збережених облікових записів, «чиста» історія браузера, відсутність руху миші — ознаки штучного середовища, а не реальної робочої станції. *Часові й поведінкові пастки*: тривале присипляння в розрахунку на обмежений час аналізу, очікування перезавантаження чи певної дії користувача, спрацювання лише за конкретної дати (логічна бомба). Кожна група вимагає власних контрзаходів, і саме тому побудова прозорої пісочниці є не разовим рішенням, а безперервним процесом супроводу в міру появи нових перевірок.

Протидія ухиленню — це постійна гонитва: підвищення прозорості пісочниць (інтроспекція через гіпервізор, реалістичні середовища з історією діяльності, емуляція взаємодії користувача), bare-metal-аналіз [133], виявлення самих перевірок ухилення як підозрілої ознаки. Жодне середовище не є ідеально прозорим, тому динамічний аналіз, як і статичний, дає неповну картину й найефективніший у поєднанні з іншими методами. Вітчизняні дослідження систематизують застосування пісочниць [12] та поведінкових методів на основі системних викликів [34] в умовах зростання кількості ухильних зразків.

7.7 Кейс: динамічний аналіз зразка-вимагача

Розглянемо послідовність динамічного аналізу підозрілого зразка (захисна задача: побудова поведінкового профілю для виявлення й реагування).

Крок 1. Підготовка середовища. Готуємо ізольовану пісочницю з реалістичним профілем: встановлені офісні програми, документи-приманки, історія діяльності, контрольована мережа з імітацією інтернету (щоб спостерегти мережеві звернення, не випускаючи зразок назовні). Знімаємо базовий знімок системи.

Крок 2. Запуск і спостереження. Запускаємо зразок, фіксуючи виклики API, файлову й реєстрову активність, мережеві звернення, породження процесів протягом заданого часу з емуляцією дій користувача.

Крок 3. Аналіз поведінки. У журналі виявляємо характерну послідовність вимагача: перелік файлів користувача, читання й перезапис із високою ентропією вмісту (шифрування (3.3)), видалення тіньових копій, звернення

до командного сервера по ключ, створення записок про викуп. Зіставляємо дії з техніками ATT&CK.

Крок 4. Аналіз пам'яті. Знімаємо дамп пам'яті й відновлюємо розшифрований код, рядки конфігурації, адресу командного сервера, можливий ключ — артефакти, відсутні у файлі на диску.

Крок 5. Формування індикаторів і правил. На основі профілю формуємо поведінкові правила виявлення (розділ 11), мережеві та файлові індикатори (розділ 12) для реагування (розділ 16). Перевіряємо стійкість: повторюємо запуск за інших умов (дати, мови системи), щоб виявити приховані гілки.

Кейс ілюструє силу динамічного аналізу — розкриття справжньої поведінки попри пакування — і його межі: висновок про нешкідливість за одним запуском ненадійний, тому критичні рішення спираються на багаторазове спостереження й поєднання зі статичним аналізом.

Висновки до розділу

Динамічний аналіз досліджує зразок під час виконання й долає головне обмеження статичного — прихований пакуванням і шифруванням код, який зразок розгортає сам. Його сила — у фіксації справжньої семантики дій незалежно від запису коду; слабкість — у спостереженні лише однієї гілки за даних умов (7.1) та вразливості до антисендбокс-прийомів. Пісочниці балансують прозорість і повноту спостереження, розрізняючись рівнем перехоплення (API, ядро, гіпервізор) та ізоляції (емулятор, ВМ, bare-metal). Спостережувані артефакти — виклики API, файлова, реєстрова, мережева й процесна активність — найцінніші у вигляді контекстних *послідовностей*, зіставлених із ATT&CK. Аналіз пам'яті розкриває розгорнутий код, ключі та безфайлові загрози, слугуючи містком до статичного аналізу дампа. Формалізація поведінки уможливлює кластеризацію сімейств, стійкішу до поверхневих змін. Межі методу — неповнота покриття й ухилення (7.2) — долаються підвищенням прозорості, bare-metal-аналізом і багаторазовими запусками. Головний висновок: динамічний аналіз необхідний, але недостатній; максимальну цінність він дає в гібридному поєднанні зі статичним аналізом і є фундаментом поведінкового виявлення сучасних засобів захисту.

Питання для самоперевірки

1. У чому полягає принцип динамічного аналізу й яку головну проблему статичного аналізу він долає?
2. Порівняйте статичний і динамічний аналіз (табл. 7.1). Чому зрілий аналіз є гібридним?

3. Що таке пісочниця? Опишіть рівні перехоплення (API, ядро, гіпервізор) та їхні переваги й вади.
4. Поясніть суперечність між прозорістю й повнотою спостереження в пісочниці. Навіщо потрібне реалістичне середовище?
5. Які категорії артефактів поведінки спостерігають динамічно? Чому послідовність дій важливіша за окремі виклики?
6. Чому аналіз пам'яті посідає особливе місце? Що він розкриває з того, чого немає у файлі на диску?
7. Як аналіз пам'яті дозволяє перемогти пакування без ручного розпакування?
8. Що таке покриття виконання (7.1)? Чому воно принципово обмежене й як його підвищують?
9. За формулою (7.2) поясніть стратегію підвищення прозорості пісочниць. Що дає bare-metal-аналіз?
10. Що таке антисендбокс- та антиВМ-прийоми? Наведіть приклади ознак середовища, які виявляє зразок.
11. Як виявляють чутливі до середовища зразки? Чому розбіжність поведінки в різних середовищах є ознакою ухилення?
12. Чому висновок про нешкідливість зразка за одним запуском у пісочниці ненадійний?
13. Опишіть послідовність динамічного аналізу вимагача (кейс підрозділу 7.7). Яку роль відіграє аналіз пам'яті?
14. Дослідницьке: порівняйте повноту поведінкових профілів, отриманих за одного та за багаторазових запусків із варіацією умов, на наборі ухильних зразків.
15. Дослідницьке: оцініть стійкість обраної пісочниці до набору відомих антиВМ-перевірок і запропонуйте заходи підвищення її прозорості.

РОЗДІЛ 8

Техніки ухилення, обфускації та антианалізу

Ефективність методів виявлення й аналізу (розділи 6–7) викликала розвиток технік **ухилення** — засобів, якими автори ЗПК приховують справжній код і поведінку від статичного й динамічного аналізу та від сигнатурних і поведінкових детекторів. Ці техніки — не самостійна функція ЗПК, а оболонка, що захищає корисне навантаження; проте їх розуміння критичне для захисника, бо саме вони визначають межі аналізу (розділ 3) і диктують вибір методів протидії. Парадоксально, техніки захисту коду (обфускація, пакування) походять із легітимної сфери — захисту інтелектуальної власності й протидії зворотному аналізу комерційного ПЗ, — але застосовуються й зловмисниками. Розділ розглядає таксономію ухилення, пакування й шифрування, обфускацію коду, поліморфізм і метаморфізм, антианалізні прийоми, а також методи деобфускації й розпакування, і завершується кейсом підготовки обфускованого зразка до аналізу. Виклад зосереджено на *розпізнаванні* й *подоланні* цих технік, а не на їх створенні.

8.1 Таксономія технік ухилення

Техніки ухилення класифікують за тим, *що* саме вони мають обманути. *Ухилення від статичного аналізу*: пакування, шифрування, обфускація — приховують код на диску. *Ухилення від сигнатур*: поліморфізм, метаморфізм — змінюють байтове подання, зберігаючи функцію. *Ухилення від динамічного аналізу*: антисендбокс, антиВМ, антизневадження, логічні бомби (розділ 7) — приховують поведінку за умов спостереження. *Ухилення від виявлення в системі*: руткіт-приховування (розділ 5), living-off-the-land (розділ 13) — маскують присутність під легітимну активність. Узагальнену таксономію наведено в табл. 8.1.

Таблиця 8.1 — Таксономія технік ухилення за ціллю

Ціль обману	Техніки	Контрзахід
Статичний аналіз	пакування, шифрування, обфускація	розпакування, деобфускація
Сигнатури	поліморфізм, метаморфізм	поведінка, семантика
Динамічний аналіз	антиВМ, антизневадження, бомби	прозорі пісочниці
Виявлення в системі	руткіти, LotL	аналіз пам'яті, аномалії

Ключове спостереження: жодна техніка ухилення не є абсолютною. Пакування долається розпакуванням, обфускація — деобфускацією, антиВМ — прозорими пісочницями. Проте кожна підвищує *вартість* аналізу й знижує ефективність дешевих автоматизованих методів, змушуючи захисника витрачати дорогі ресурси. Тому мета ухилення — не унеможливити аналіз (це недосяжно для виконуваного коду, що мусить зрештою розгорнутися, аби працювати), а зробити його настільки дорогим, щоб відстрочити виявлення й подовжити час присутності (dwell time, розділ 13).

8.2 Пакування та шифрування

Пакувальник (packer) — програма, що перетворює виконуваний файл на стиснений і/або зашифрований образ, доповнений *розпакувальником* (stub), який під час запуску відновлює оригінальний код у пам'яті й передає йому керування. Первісне призначення — стиснення й захист від зворотного аналізу; для ЗПК — приховування коду від статичного аналізу й сигнатур. Масштабне дослідження пакувальників [221] показало широкий спектр складності — від простого стиснення до багатошарового шифрування з віртуалізацією. Споріднена техніка — **криптор** (crypter), що робить наголос на шифруванні для ухилення від детекторів.

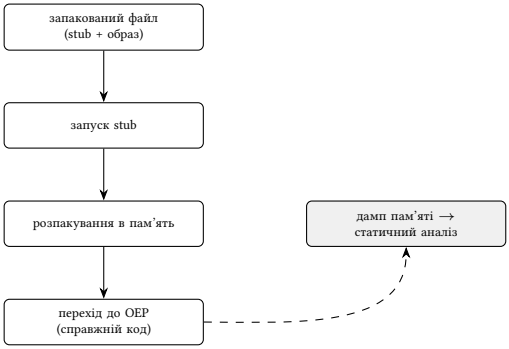


Рис. 8.1 — Життєвий цикл запакованого зразка й точка перехоплення для розпакування (дамп після досягнення OEP)

Пакування майже повністю знищує цінність байтових сигнатур: один і той самий код під різними пакувальниками (або з різними ключами) дає зовсім різні файли. Ознакою пакування слугує висока ентропія секцій (3.3), невідповідність між оголошеним та фактичним розміром, аномальна таблиця імпортів (часто лише кілька функцій для динамічного завантаження решти), нетипові імена секцій. Проте пакування безсиле проти динамічного аналізу й аналізу пам'яті (розділ 7): щоб виконатися, код *мусить* розпакуватися, після чого його можна зняти дампом. Ця фундаментальна властивість — неможливість виконати прихований код, не розкривши його, — є ключем до подолання пакування.

Складність пакувальників утворює спектр [221]. Найпростіші виконують одношарове стиснення зі статичним розпакувальником, який легко розпізнати й автоматично обернути. Складніші додають кілька шарів шифрування, перевірки цілісності (щоб аналітик не міг пропатчити stub), антианалізні перевірки в самому розпакувальнику та відкладене розгортання коду частинами (щоб у пам'яті ніколи не був присутній увесь код одразу). Найскладніші поєднують пакування з віртуалізацією самого розпакувальника. Проте навіть вони підкоряються базовому обмеженню: будь-яка інструкція, що виконується процесором, у момент виконання присутня у відкритому вигляді в пам'яті. Тому стратегія захисника — не намагатися статично «прочитати» запакований файл, а дозволити йому розгорнутися під контролем і зафіксувати відкритий код. Різниця між пакувальниками зводиться до того, *коли* й *наскільки повно* код розкривається, а не до того, чи розкривається взагалі.

Слід відрізняти *стандартні* пакувальники загального призначення від *кастомних*. Стандартні (стискачі та протектори, доступні публічно) мають відомі сигнатури stub-коду й типові розпакувальники, тому часто розпі-

знаються й автоматично обертаються інструментами. Саме тому серйозні зловмисники застосовують власні або суттєво модифіковані пакувальники, унікальні для кампанії: вони позбавлені відомих сигнатур і потребують ручного аналізу. Парадоксально, сам факт застосування рідкісного кастомного пакувальника стає ознакою: легітимне ПЗ зазвичай користується відомими комерційними протекторами, тож нестандартна оболонка підвищує підозрілість зразка. Це приклад загального принципу — зусилля з приховування самі лишають слід, який захисник може використати як ознаку.

8.3 Обфускація коду

Обфускація — перетворення коду, що зберігає його функцію, але ускладнює розуміння [87]. На відміну від пакування (приховує код цілком до розпакування), обфускація лишає код виконуваним і видимим, але заплутаним. Основні прийоми: *непрозорі предикати* (умови, значення яких завжди відоме автору, але важко обчислюване аналізатором, — створюють фіктивні гілки); *сплощення потоку керування* (control-flow flattening — заміна природної структури CFG на диспетчер із великим перемикачем, що приховує справжній порядок блоків); *підстановка еквівалентних інструкцій* та вставлення мертвого коду; *віртуалізація коду* (переклад коду на власну байт-кодову машину з інтерпретатором — найстійкіший і найдорожчий прийом). Інструменти на кшталт Obfuscator-LLVM [130] автоматизують ці перетворення.

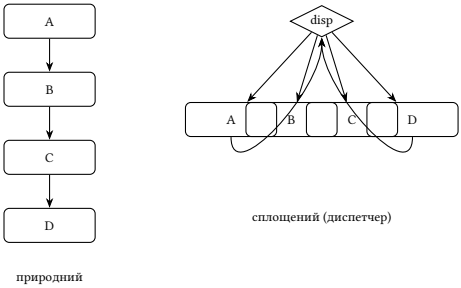


Рис. 8.2 — Сплощення потоку керування: природний CFG (ліворуч) замінюється диспетчером із перемикачем (праворуч), що приховує справжній порядок блоків

Обфускація безпосередньо б'є по аналізу коду (розділ 6): сплощення руйнує зрозумілий CFG, непрозорі предикати породжують фіктивні шляхи, віртуалізація ховає логіку за інтерпретатором. Розгляньмо прийоми докладніше. *Непрозорий предикат* — це вираз, значення якого автор знає наперед (наприклад, тотожність на кшталт $7x^2 - 1 \neq y^2$, що істинна для всіх цілих), але доведення якого вимагає від аналізатора нетривіальних мірку-

вань; вставлений як умова розгалуження, він створює гілку, що насправді ніколи (або завжди) не виконується, роздуваючи CFG фіктивними шляхами. *Сплощення потоку керування* (рис. 8.2) розбиває функцію на базові блоки, нумерує їх і підпорядковує єдиному диспетчеру-перемикачу, який за значенням керованої змінної обирає наступний блок; природна послідовність зникає, і відновити її можна лише, простеживши значення керованої змінної. *Віртуалізація* замінює машинні інструкції байт-кодом власної віртуальної машини, вбудованої у зразок; аналітик мусить спершу зрозуміти саму цю машину, перш ніж декодувати логіку, — що й робить віртуалізацію найдорожчим для подолання прийомом. Проте й вона має теоретичні межі: доведено, що «ідеальна» обфускація неможлива в загальному випадку, а практична обфускація лише підвищує вартість аналізу [193]. Семантичний і динамічний аналіз, що працюють з ефектами коду, а не з його записом, частково нейтралізують обфускацію, бо яким би заплутаним не був код, його спостережувана поведінка лишається тією самою.

8.4 Поліморфізм та метаморфізм

Поліморфізм — здатність коду відтворювати себе в різних байтових формах зі збереженням функції, класично через шифрування тіла змінним ключем і мутацію розшифровувача [214]. **Метаморфізм** — глибша техніка: мутація *самого* коду (перестановка інструкцій, заміна еквівалентними, зміна регістрів) без шифрування, так що кожне покоління має інший код тієї самої функції. Обидві техніки націлені на сигнатури: якщо кожен екземпляр унікальний на рівні байтів, байтова сигнатура безсила.

Ефективність ухилення від сигнатур зручно описати комбінаторно. Якщо генератор варіантів має m незалежних точок мутації, кожна з r рівноможливими варіантами, кількість різних байтових форм

$$N = r^m, \tag{8.1}$$

зростає експоненційно, роблячи перелік усіх форм однією байтовою сигнатурою неможливим. Формула (8.1) пояснює, чому проти поліморфізму й метаморфізму ефективні лише методи, інваріантні до таких мутацій: виявлення за розшифрованим тілом у пам'яті, поведінкові сигнатури (розділ 7), семантичний аналіз (розділ 6), статистичні ознаки (розділ 9). Саме перехід від байтового до поведінкового й семантичного рівня є системною відповіддю на поліморфізм.

Історично поліморфізм і метаморфізм — вершина еволюції самозахисту вірусної епохи [214, 229], коли виявлення спиралося майже виключно на байтові сигнатури. Поліморфний генератор шифрує незмінне тіло вірусу змінним ключем і щоразу по-новому будує розшифровувач (переставляючи інструкції, вставляючи «сміття», змінюючи регістри), тож єдиним спіль-

ним інваріантом лишається поведінка розшифрованого тіла в пам'яті. Відповіддю антивірусної галузі стала емуляція: зразок виконують у вбудованому емуляторі, доки він сам не розшифрує тіло, після чого застосовують сигнатуру до відкритого коду — рання форма динамічного подолання ухилення. Метаморфізм пішов далі, відмовившись від сталого тіла взагалі: мутує весь код, і спільного байтового інваріанта не лишається зовсім. Це остаточно засвідчило вичерпність суто байтового підходу й прискорило розвиток поведінкових, емуляційних і статистичних методів, які й становлять сучасну основу виявлення.

8.5 Антианалізні прийоми

Антианаліз протидіє безпосередньо роботі аналітика та інструментів [67]. *Антизневадження* (anti-debugging): виявлення зневаджувача через прапорці системи, вимірювання часу виконання (у зневаджувачі код повільніший), навмисні винятки, перевірку контрольних точок. *Антидизасемблювання*: вставлення байтів, що збивають лінійний прохід дизасемблера (розділ 6), перекриття інструкцій, непрямі переходи. *АнтиВМ та антисендбокс* (розділ 7): виявлення віртуалізації та інструментування. *Антиемуляція*: використання рідкісних або недокументованих інструкцій, що неточно відтворюються емулятором.

Окрему роль відіграє *часовий* антианаліз. Виконання під зневаджувачем або в емуляторі значно повільніше за виконання на реальному процесорі, тому зразок вимірює тривалість певної послідовності операцій і, виявивши аномальну затримку, робить висновок про спостереження. Споріднений прийом — вимірювання часу між двома точками з очікуванням, що між ними не було зовнішнього втручання: контрольна точка зневаджувача або перехоплення розтягує цей інтервал. Контрзаходи — нормалізація часу (підміна показань таймерів так, щоб вони відповідали «реальній» швидкості) та перехід на апаратне трасування, невидиме для зразка. Часовий антианаліз показово ілюструє загальний принцип: будь-яка спроба ухилення потребує *спостережуваної* відмінності середовищ, і усунення цієї відмінності знімає відповідну перевірку.

Спільна риса антианалізу — опора на *розрізнюваність* середовища аналізу й реальної системи. Доки аналітичне середовище має помітні відмінності (прапорці, часові аномалії, артефакти ВМ), зразок може їх виявити. Контрзахід — підвищення прозорості: приховування зневаджувача, нормалізація часу, bare-metal-аналіз (розділ 7), а також виявлення самих антианалізних перевірок як підозрілої ознаки — наявність численних перевірок середовища сама свідчить про зловмисність. Крім того, антианалізний код підвищує вартість, але не робить аналіз неможливим: досвідчений аналі-

тик обходить перевірки, патчить гілки або працює на рівні, недосяжному для перевірок зразка.

8.6 Деобфускація та розпакування

Подолання ухилення — окрема дисципліна. **Розпакування** спирається на фундаментальну властивість: код мусить розгорнутися, аби виконатися. *Загальне (універсальне) розпакування* [187, 116] не залежить від конкретного пакувальника: зразок виконують, відстежуючи пам'ять, і фіксують момент, коли керування передається в раніше записану (а отже, розпаковану) ділянку. Інваріант «записано-потім-виконано» (write-then-execute) є ознакою завершення розпакування: якщо байт за адресою спершу записано під час виконання, а потім з нього вибирають інструкцію, це майже напевно розпакований код. Формально ознаку досягнення точки входу оригіналу можна виразити умовою

$$\exists a:\ W(a) \prec X(a), \tag{8.2}$$

де $W(a)$ — подія запису за адресою a, $X(a)$ — подія виконання коду за тією самою адресою, а $\prec$ — часове передування. Виявивши таку подію, аналізатор знімає дамп пам'яті й продовжує статичний аналіз уже відкритого коду.

Деобфускація [228] відновлює зрозумілий код з обфускованого: спрощення непрозорих предикатів (через виявлення завжди-істинних умов), відновлення природного CFG зі сплощеного, усунення мертвого коду, девіртуалізація. Загальні підходи спираються на динамічний аналіз потоку даних і символьне виконання, що працюють із семантикою, а не з заплутаним записом. Вітчизняні дослідження систематизують методи обфускації та підходи до їх подолання [21], а також виявлення запакованих зразків за ентропією та структурними ознаками [5].

8.7 Кейс: підготовка обфускованого зразка до аналізу

Розглянемо послідовність подолання ухилення для аналізу запакованого й обфускованого зразка (захисна задача).

Крок 1. Розпізнавання захисту. За ентропією (3.3), структурою секцій та імпортів визначаємо наявність пакування й, за можливості, його тип. Висока ентропія й бідна таблиця імпортів — типові ознаки.

Крок 2. Розпакування. Застосовуємо загальне розпакування: виконуємо зразок у контрольованому середовищі, відстежуючи пам'ять за інваріантом (8.2). Досягнувши точки входу оригіналу, знімаємо дамп і відновлюємо таблицю імпортів.

Крок 3. Оцінювання обфускації. На розпакованому коді оцінюємо рівень обфускації: сплощення CFG, непрозорі предикати, ознаки віртуалізації. Це визначає потрібну глибину деобфускації.

Крок 4. Деобфускація. Спрощуємо непрозорі предикати, відновлюємо природний CFG, усуваємо мертвий код, за потреби девіртуалізуємо. Для стійких до статики прийомів застосовуємо динамічний і семантичний аналіз, що працюють з поведінкою.

Крок 5. Аналіз і сигнатури. На очищеному коді проводимо повноцінний статичний аналіз (розділ 6), виділяємо стійкі семантичні ознаки та будуємо правила виявлення, інваріантні до повторного пакування (орієнтовані на поведінку й семантику, а не на байти оболонки).

Кейс ілюструє стратегічний принцип: ухилення підвищує вартість, але не скасовує аналізу, бо виконуваний код зрештою мусить розгорнути свою справжню логіку. Захисник переносить виявлення на рівні, інваріантні до ухилення, — поведінку, семантику, аномалії, — замість того щоб змагатися з оболонкою на її ж полі байтових форм.

Висновки до розділу

Техніки ухилення — пакування, шифрування, обфускація, поліморфізм, метаморфізм, антианаліз — є захисною оболонкою ЗПК, спрямованою проти конкретних методів виявлення й аналізу. Їх класифікують за ціллю обману: статичний аналіз, сигнатури, динамічний аналіз, виявлення в системі (табл. 8.1). Пакування приховує код на диску, знищуючи байтові сигнатури, але безсиле проти динамічного аналізу й аналізу пам'яті, бо код мусить розпакуватися для виконання. Обфускація лишає код видимим, але заплутаним, б'ючи по аналізу коду, проте має теоретичні межі й нейтралізується семантичним і динамічним аналізом. Поліморфізм і метаморфізм породжують експоненційну кількість байтових форм (8.1), роблячи байтові сигнатури безсилими й змушуючи переходити до поведінкового й семантичного рівня. Антианаліз спирається на розрізнюваність середовища аналізу й долається підвищенням прозорості. Деобфускація й загальне розпакування спираються на фундаментальну властивість виконуваного коду — необхідність розгорнутися (8.2). Головний висновок: ухилення підвищує вартість аналізу, але не скасовує його; системна відповідь захисника — перенесення виявлення на рівні, інваріантні до ухилення (поведінка, семантика, аномалії), замість змагання з оболонкою на рівні байтів.

Питання для самоперевірки

1. За якою ознакою класифікують техніки ухилення (табл. 8.1)? Чому жодна з них не є абсолютною?

2. Яка справжня мета технік ухилення, якщо аналіз виконуваного коду принципово неможливо унеможливити?
3. Що таке пакувальник і криптор? Чому пакування знищує цінність байтових сигнатур?
4. Назвіть ознаки пакування. Чому пакування безсиле проти динамічного аналізу й аналізу пам'яті?
5. Що таке обфускація? Опишіть непрозорі предикати, сплощення CFG та віртуалізацію коду.
6. Чому обфускація має теоретичні межі? Як семантичний і динамічний аналіз її нейтралізують?
7. Порівняйте поліморфізм і метаморфізм. Проти якого методу виявлення вони спрямовані?
8. За формулою (8.1) поясніть, чому байтові сигнатури безсилі проти поліморфізму. Які методи ефективні натомість?
9. Опишіть антизневадження, антидизасемблювання та антиемуляцію. На чому ґрунтується їх спільна риса?
10. Чому наявність численних перевірок середовища сама є ознакою зловмисності?
11. Поясніть принцип загального розпакування та інваріант «записано-потім-виконано» (8.2).
12. Що таке деобфускація? Які підходи застосовують для відновлення зрозумілого коду?
13. Опишіть послідовність підготовки обфускованого зразка до аналізу (кейс підрозділу 8.7).
14. Дослідницьке: оцініть, наскільки різні пакувальники підвищують вартість аналізу й знижують ефективність сигнатурного виявлення на наборі зразків.
15. Дослідницьке: порівняйте стійкість поведінкових та семантичних ознак до повторного пакування й обфускації того самого корисного навантаження.

РОЗДІЛ 9

Машинне навчання та статистичне виявлення

Експоненційне зростання кількості зразків (розділ 1) та вичерпність суто сигнатурного підходу перед поліморфізмом і метаморфізмом (розділ 8) зробили **машинне навчання** (МН) центральним інструментом сучасного виявлення ЗПК. Замість ручного опису ознак кожної загрози МН *навчається* розрізняти злоякісне й легітимне за прикладами, узагальнюючи на небачені зразки. Це дає виявлення нових загроз (на що нездатні сигнатури) і масштабованість на мільйони зразків. Проте МН не є чарівним рішенням: воно успадковує й загострює проблему базової частоти (розділ 3), потребує якісних розмічених даних, вразливе до змагальних атак (розділ 8) і дає рішення, які важко пояснити. Розділ розглядає місце МН у виявленні, ознаки й подання зразків, типові моделі, оцінювання якості, змагальне машинне навчання та практичні обмеження, і завершується кейсом побудови статичного класифікатора PE-файлів. МН не замінює інших методів, а інтегрується з ними (розділи 6, 7, 10).

9.1 Місце машинного навчання у виявленні

МН у протидії ЗПК розв'язує дві основні задачі: *класифікацію* (злоякісний/легітимний — бінарна; або віднесення до сімейства — багатокласова) і *кластеризацію* (виявлення груп схожих зразків без розмітки, розділ 6). За наявністю розмітки розрізняють навчання з учителем (на розмічених прикладах), без учителя (кластеризація, виявлення аномалій) та напівкероване. Фундаментальна ідея — узагальнення: модель, навчена на відомих зразках, має правильно класифікувати *нові*, яких не було в навчальній вибірці. Саме здатність до узагальнення відрізняє МН від сигнатур і дає виявлення небачених загроз [220, 108].

Класична праця [202] застерігає від наївного перенесення МН із задач на кшталт розпізнавання образів у безпеку: тут висока ціна хибних тривог, величезний дисбаланс класів, змінність (concept drift — загрози еволюціонують, тож модель старіє), активний супротивник, що цілеспрямовано обходить модель. Ці особливості визначають, чому МН у безпеці складніше, ніж у типових застосуваннях, і чому воно доповнює, а не замінює детерміновані методи.

Корисно розрізняти два режими застосування МН, що мають різні вимоги й ризики. *Хмарне* виявлення на боці постачальника працює з величезними обчислювальними ресурсами та агрегованою телеметрією з мільйонів вузлів; воно може дозволити собі дорогі моделі й швидке перенавчання,

але вносить затримку й залежність від зв'язку. *Локальне* виявлення на кінцевій точці (розділ 10) працює в реальному часі з обмеженими ресурсами й без гарантованого зв'язку; воно потребує компактних, швидких моделей і особливо чутливе до хибних тривог, бо помилка блокує роботу користувача негайно. Промислові рішення поєднують обидва режими: легка локальна модель дає миттєве рішення, а підозрілі чи невизначені випадки ескалюються в хмару для глибшого аналізу. Таке розшарування — ще один прояв каскадної архітектури із зростанням вартості й покриття, що проходить крізь увесь посібник.

Важливо усвідомлювати межу застосовності узагальнення. Модель надійно впізнає варіації, *подібні* до баченого в навчанні, але принципово нова техніка, не представлена у вибірці, може лишитися невиявленою так само, як і сигнатурою. МН не скасовує гонитви озброєнь, а зміщує її: замість створення нової сигнатури під кожен зразок захисник підтримує актуальність навчальної вибірки й моделі під еволюцію цілих класів загроз. Виграш — у масштабі узагальнення, а не в принциповій повноті, яка, за результатами розділу 3, недосяжна для жодного методу.

9.2 Ознаки та подання зразків

Якість МН визначається насамперед **ознаками** — числовим поданням зразка, що подається моделі. Ознаки поділяють на статичні (з файлу без виконання) і динамічні (з поведінки, розділ 7). Типові статичні ознаки: байтові n-грами, частоти інструкцій, імпортовані функції, властивості PE-заголовка й секцій, ентропія (3.3), рядки, розмір. Піонерська праця [194] уперше застосувала здобуття даних до цих ознак; відкритий набір EMBER [50] стандартизував ознаки PE для досліджень. Динамічні ознаки: послідовності викликів API, дії над ресурсами, мережева активність.

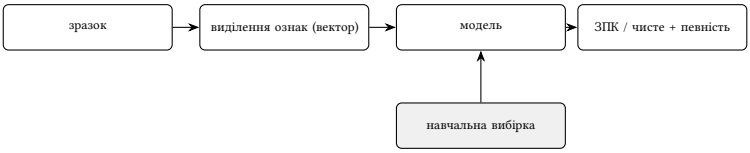


Рис. 9.1 — Конвеєр виявлення на основі машинного навчання: від зразка через виділення ознак до рішення моделі

Окремий напрям — навчання *без* ручного виділення ознак: глибокі моделі приймають «сирі» байти [178] або двовимірне подання файлу [191] й самі навчаються ознак. Це знижує залежність від експертних знань, але потребує більших даних і обчислень та дає менш інтерпретовані моделі.

Вибір між ручними й автоматичними ознаками — компроміс між інтерпретовністю, вартістю й стійкістю до ухилення.

Сирі ознаки зазвичай потребують *нормалізації* та зважування, бо різні ознаки мають різні масштаби й інформативність. Класичний прийом для текстоподібних ознак (рядків, n-грамів інструкцій) — зважування за частотою появи в зразку відносно поширеності в усій колекції (TF–IDF), що підкреслює рідкісні, а отже дискримінативні, ознаки й пригнічує всюдисущі. Для ознаки t у зразку d з колекції D вага

$$w_{t,d} = f_{t,d} \cdot \log \frac{|D|}{|\{d' \in D : t \in d'\}|}, \tag{9.1}$$

де $f_{t,d}$ — частота ознаки в зразку, а логарифмічний множник (зворотна документна частота) тим більший, чим рідкісніша ознака. Формула (9.1) ілюструє загальний принцип інженерії ознак: цінність ознаки визначає не частота сама по собі, а здатність *розрізняти* класи. Погано дібрані або надлишкові ознаки не лише не допомагають, а й підвищують ризик перенавчання та відкривають простір для змагального ухилення.

Окрему увагу приділяють *витоку* інформації між навчанням і тестом. Поширена методологічна помилка — включення до навчальної вибірки варіантів тих самих зразків, що й у тесті, або ознак, які опосередковано кодують мітку (наприклад, артефактів конкретного аналітичного конвеєра). Такий витік дає оптимістично завищену оцінку якості, що не відтворюється в експлуатації. Тому коректне формування вибірок — не технічна дрібниця, а передумова вірогідності всіх подальших висновків.

9.3 Типові моделі та їх властивості

У виявленні ЗПК застосовують широкий спектр моделей, що різняться інтерпретовністю, вартістю й стійкістю (табл. 9.1). *Дерева рішень та їх ансамблі* (випадковий ліс, градієнтний бустинг) — робочий інструмент статичного виявлення РЕ: вони добре працюють із табличними ознаками, відносно інтерпретовані й стійкі. *Лінійні моделі* (логістична регресія, SVM) прості й швидкі, але обмежені в складності меж. *Нейронні мережі*, зокрема глибокі, здатні навчатися складних залежностей і працювати із сирими даними, але потребують великих вибірок, обчислень і важко інтерпретуються. *Методи виявлення аномалій* шукають відхилення від норми, корисні для невідомих загроз, але схильні до хибних тривог.

Таблиця 9.1 — Моделі машинного навчання у виявленні ЗПК

Модель	Сильні риси	Інтерпретовність
Ансамблі дерев	точність на табличних ознаках	помірна
Лінійні (SVM)	швидкість, простота	висока
Нейромережі	складні залежності, сирі дані	низька
Виявлення аномалій	невідомі загрози	помірна

Немає універсально найкращої моделі: вибір залежить від задачі, доступних даних, вимог до швидкості та інтерпретовності й моделі загроз. На практиці ефективні ансамблі та багаторівневі архітектури, що поєднують кілька моделей і типів ознак, а також інтеграція МН у каскад із сигнатурами й поведінковим аналізом.

Особливу увагу в безпеці приділяють *інтерпретовності* моделі. Коли детектор блокує файл, аналітикові центру реагування (розділ 16) потрібно зрозуміти *чому*, щоб підтвердити рішення, розслідувати інцидент і відрізнити справжнє спрацювання від хибного. Інтерпретовані моделі (дерева, лінійні) дають таке пояснення безпосередньо через ваги ознак чи шлях у дереві; для складних моделей застосовують методи пояснення post-hoc, що наближено вказують, які ознаки найбільше вплинули на рішення. Брак інтерпретовності — не лише операційна незручність, а й регуляторний ризик (розділ 2) та перешкода довірі, тому в критичних застосуваннях часто свідомо жертвують часткою точності заради прозорості рішень.

9.4 Оцінювання якості

Оцінювання детектора на основі МН спирається на метрики розділу 3: повноту, точність, частоту хибних тривог, площу під ROC. Критична особливість — величезний дисбаланс і ціна хибних тривог: за мільйонів легітимних файлів навіть мала частка хибних тривог дає неприйнятний абсолютний потік (проблема базової частоти). Тому в безпеці оцінюють якість за дуже низьких рівнів хибних тривог. Для бінарного класифікатора з розмітками «злоякісний» ($+$) і «чистий» ($-$) зведену міру якості дає F_β-міра

$$F_\beta = (1+\beta^2)\cdot \frac{\mathrm{Precision}\cdot \mathrm{Recall}}{\beta^2\cdot \mathrm{Precision} + \mathrm{Recall}}, \tag{9.2}$$

де параметр β задає відносну вагу повноти й точності: $\beta < 1$ наголошує на точності (мінімізації хибних тривог), що відповідає пріоритетам захисту кінцевих точок. Коректне оцінювання вимагає реалістичного складу тестової вибірки (базова частота як у продакшні), часового поділу (навчання на минулому, тест на майбутньому — щоб урахувати дрейф) і незалежності тесту від навчання.

Окрема небезпека — *дрейф концепту*: загрози еволюціонують, тому модель, точна сьогодні, деградує з часом. Це вимагає моніторингу якості в експлуатації та періодичного перенавчання на свіжих даних, а також оцінювання не лише миттєвої, а й *часової* стійкості моделі.

Практичним наслідком пріоритету низьких хибних тривог є вибір *робочої точки* на ROC-кривій (розділ 3). Модель видає не бінарне рішення, а оцінку певності; поріг, за яким оцінку перетворюють на вердикт, визначає компроміс між повнотою й хибними тривогами. У захисті кінцевих точок поріг зсувають так, щоб частота хибних тривог не перевищувала жорсткого операційного ліміту (наприклад, одиниці на мільйони файлів), свідомо жертвуючи частиною повноти. Це означає, що «точність моделі» без указання робочої точки й базової частоти тестового набору є беззмістовною характеристикою: та сама модель за одного порогу придатна до промислового розгортання, а за іншого — генерує неприйнятний потік хибних тривог. Тому в безпеці звітують не одне число, а поведінку детектора в діапазоні низьких хибних тривог, і саме там порівнюють конкурентні моделі.

9.5 Змагальне машинне навчання

Протидія ЗПК — змагальне середовище з активним супротивником, тому МН тут підлягає **змагальним атакам** [64]. *Ухилення* (evasion): зловмисник мінімально змінює зразок так, щоб модель класифікувала його як чистий, зберігаючи зловмисну функцію [113]. На відміну від задач розпізнавання образів, тут зміни обмежені вимогою *працездатності* коду, тому застосовують додавання «нешкідливих» секцій, зміну метаданих, вставлення легітимних ознак. Показано, що навіть навчанням із підкріпленням можна автоматично знаходити модифікації, що обходять статичні моделі PE [49]. *Отруєння* (poisoning): зловмисник впливає на навчальні дані, щоб спотворити модель.



Рис. 9.2 — Змагальне ухилення від класифікатора й напрями протидії

Контрзаходи: змагальне навчання (додавання змагальних прикладів до навчальної вибірки), ансамблі різнорідних моделей і ознак (ухилення від усіх одразу складніше), опора на ознаки, які важко змінити без втрати функції (семантичні, поведінкові), моніторинг аномалій у вхідних даних. Фундаментальний висновок: у змагальному середовищі не буває остаточно

«навченої» моделі — потрібен безперервний цикл оцінювання, оновлення й диверсифікації. Це перегукується зі стратегічним принципом розділу 8: виявлення слід будувати на ознаках, інваріантних до дешевих модифікацій.

Корисно усвідомлювати асиметрію змагальної задачі. Зловмисникові достатньо знайти *одну* модифікацію, що обходить модель, тоді як захисник мусить встояти проти *всіх* можливих. Ця асиметрія — та сама, що й у безпеці загалом, — означає, що абсолютна стійкість моделі недосяжна, а метою є підвищення вартості ухилення до рівня, за якого воно стає невигідним. Практично це досягається поєднанням: по-перше, опорою на ознаки, зміна яких руйнує функцію ЗПК (семантичні, поведінкові), бо тут простір дешевих модифікацій вужчий; по-друге, ансамблюванням різнорідних моделей, що змушує зловмисника обходити кілька незалежних детекторів одночасно; по-третє, таємністю деталей моделі (хоча на неї не варто покладатися як на єдиний захист, бо модель можна досліджувати запитами). Варто також розрізняти ухилення в *білому* ящику (зловмисник знає модель і може обчислювати градієнти) та в *чорному* (лише спостерігає вердикти): перше дає потужніші атаки, друге реалістичніше для більшості сценаріїв і потребує багатьох запитів, що саме може бути виявлене як аномалія.

Отруєння навчальних даних — загроза, специфічна для конвеєрів, що автоматично поповнюють вибірку з відкритих джерел (наприклад, краудсорсингових колекцій зразків). Зловмисник, здатний впливати на ці джерела, може підсунути хибно розмічені або спеціально сконструйовані зразки, що зміщують межу рішення на його користь — створюючи «сліпу пляму» під конкретне сімейство. Контрзаходи — контроль якості й походження даних, стійкі до викидів методи навчання, моніторинг раптових змін у розподілі навчальної вибірки. Отруєння ілюструє, що в безпеці довіряти не можна навіть власним навчальним даним, якщо супротивник здатен на них впливати.

9.6 Практичні обмеження й інтеграція

Практичне застосування МН обмежують кілька чинників. *Дані*: потрібні великі, якісні, актуальні, коректно розмічені вибірки; хибна розмітка й зміщення вибірки псують модель. *Інтерпретовність*: рішення складних моделей важко пояснити, що ускладнює довіру, розслідування й відповідність регуляторним вимогам (розділ 2). *Дрейф і супровід*: моделі старіють і потребують перенавчання. *Змагальність*: активний супротивник. *Ресурси*: навчання й інференс глибоких моделей дорогі. Через це МН у промислових засобах не застосовують ізольовано, а інтегрують у багаторівневий конвеєр із сигнатурами, репутацією та поведінковим аналізом (розділи 6, 7, 10), де кожен рівень компенсує слабкості інших. Вітчизняні дослідження застосува-

ння МН [13] та нейромережевої класифікації за статичними ознаками [30] підтверджують ці висновки на українських даних.

9.7 Кейс: статичний класифікатор PE-файлів

Розглянемо побудову статичного класифікатора «злоякісний/чистий» для PE-файлів (захисна задача).

Крок 1. Дані. Формуємо збалансований за часом набір розмічених зразків (злоякісних і легітимних) з реалістичним складом, із часовим поділом на навчання й тест.

Крок 2. Ознаки. Виділяємо статичні ознаки: властивості заголовка й секцій, ентропію (3.3), імпорти, байтові гістограми, рядки (за зразком EMBER [50]).

Крок 3. Модель. Навчаємо ансамбль дерев (градієнтний бустинг) як компроміс точності, швидкості й інтерпретовності.

Крок 4. Оцінювання. Вимірюємо повноту за фіксованого низького рівня хибних тривог, ROC та F_β (9.2) з $\beta < 1$. Оцінюємо часову стійкість на пізніших зразках (дрейф).

Крок 5. Стійкість і супровід. Перевіряємо стійкість до змагального ухилення (додавання нешкідливих ознак), за потреби застосовуємо змагальне навчання й ансамблювання. Плануємо моніторинг якості й перенавчання.

Кейс демонструє, що ефективний класифікатор — не лише вдала модель, а дисциплінований процес: реалістичні дані, коректне оцінювання за низьких хибних тривог, урахування дрейфу й змагальності, інтеграція в загальну архітектуру захисту.

Висновки до розділу

Машинне навчання стало центральним інструментом виявлення ЗПК, бо дає узагальнення на нові загрози й масштабованість, яких бракує сигнатурам. Воно розв'язує задачі класифікації й кластеризації, спираючись на статичні та динамічні ознаки, причому якість визначається насамперед ознаками, а не лише моделлю. Спектр моделей — від інтерпретованих ансамблів дерев до глибоких мереж на сирих даних — не має універсального лідера; вибір залежить від задачі, даних і вимог (табл. 9.1). Оцінювання в безпеці підпорядковане проблемі базової частоти: якість міряють за дуже низьких рівнів хибних тривог (9.2), з урахуванням дрейфу концепту й часової стійкості. Протидія ЗПК — змагальне середовище, тому МН вразливе до ухилення й отруєння, що вимагає змагального навчання, ансамблів, опори на стійкі ознаки й безперервного циклу оновлення. Практичні обмеження — дані, інтерпретовність, дрейф, змагальність, ресурси — роблять ізольоване застосування МН недоцільним. Головний висновок: МН — потужний, але не самодостатній компонент; його цінність розкривається в інтеграції з

сигнатурами, репутацією та поведінковим аналізом у багаторівневому конвеєрі з дисциплінованим супроводом.

Питання для самоперевірки

1. Чому машинне навчання стало центральним інструментом виявлення ЗПК? Яку властивість сигнатур воно долає?
2. Які задачі розв'язує МН у протидії ЗПК? Поясніть ідею узагальнення.
3. Від чого застерігає праця [202]? Назвіть особливості безпеки, що ускладнюють застосування МН.
4. Що таке ознаки? Порівняйте статичні й динамічні ознаки та ручне й автоматичне їх виділення.
5. Опишіть типові моделі (табл. 9.1) та їх компроміси. Чому немає універсально найкращої моделі?
6. Чому в безпеці якість оцінюють за дуже низьких рівнів хибних тривог? Як це пов'язано з базовою частотою (розділ 3)?
7. Поясніть F_β-міру (9.2). Чому для захисту кінцевих точок обирають $\beta < 1$?
8. Що таке дрейф концепту? Як він впливає на оцінювання й супровід моделі?
9. Що таке змагальне ухилення? Чим воно обмежене у випадку ЗПК порівняно з розпізнаванням образів?
10. Що таке отруєння навчальних даних? Назвіть контрзаходи проти змагальних атак.
11. Чому в змагальному середовищі не буває остаточно «навченої» моделі?
12. Які практичні чинники обмежують застосування МН? Чому МН інтегрують у багаторівневий конвеєр?
13. Опишіть побудову статичного класифікатора PE (кейс підрозділу 9.7). Чому важливий часовий поділ даних?
14. Дослідницьке: оцініть деградацію якості обраної моделі в часі (дрейф) на наборі зразків, розподілених за датами, і запропонуйте стратегію перенавчання.
15. Дослідницьке: порівняйте стійкість моделей на ручних та на сирих ознаках до автоматичного змагального ухилення.

РОЗДІЛ 10

Захист кінцевих точок, моніторинг і телеметрія

Методи виявлення (розділи 6–9) реалізуються на практиці як **системи захисту кінцевих точок** — вузлів, де ЗПК зрештою виконується: робочих станцій, серверів, мобільних пристроїв. Кінцева точка — остання й водночас найбагатша на інформацію лінія: саме тут поведінка коду спостережувана безпосередньо. Еволюція від простого антивірусу до засобів виявлення й реагування на кінцевих точках (EDR) і розширеного виявлення (XDR) відображає зміщення парадигми: від запобігання окремому файлу до безперервного моніторингу, кореляції подій і реагування в масштабі всієї інфраструктури. Розділ розглядає архітектуру захисту кінцевих точок, телеметрію й джерела подій, виявлення й реагування (EDR/XDR), профілактичні заходи (зниження поверхні атаки, контроль застосунків, найменші привілеї) та обмеження, і завершується кейсом виявлення безфайлової атаки за телеметрією. Зібрані тут дані живлять інженерію виявлення (розділ 11), розвідку загроз (розділ 12) і реагування на інциденти (розділ 16).

10.1 Еволюція й архітектура захисту кінцевих точок

Класичний **антивірус** будувався навколо запобігання: сканування файлів за сигнатурами (розділ 6) під час доступу й за розкладом, блокування відомих загроз. Зростання кількості й витонченості загроз, зокрема безфайлових і living-off-the-land (розділ 13), виявило межі такого підходу: що не має файлу або маскується під легітимний процес, те сигнатура файлу не ловить. Відповіддю став **EDR** (Endpoint Detection and Response) [80] — безперервний запис активності кінцевої точки, виявлення підозрілих патернів поведінки й засоби реагування (ізоляція вузла, зупинення процесу, відкочування змін). **XDR** [107] розширює цю ідею, корелюючи телеметрію кінцевих точок, мережі, пошти й хмари в єдину картину.

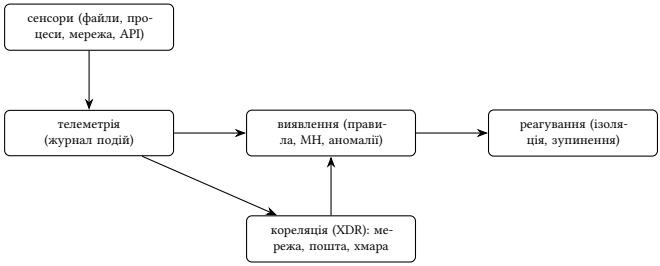


Рис. 10.1 — Архітектура EDR/XDR: від сенсорів кінцевої точки через телеметрію й виявлення до реагування й кореляції

Сучасна кінцева точка поєднує кілька рівнів захисту (табл. 10.1): запобіжний (сигнатури, репутація, блокування), виявлювальний (поведінка, МН, аномалії), реагувальний (ізоляція, відкочування) і слідчий (запис телеметрії для розслідування). Ця багаторівневість — пряме втілення принципу ешелонованого захисту [165]: жоден рівень не ідеальний, тому вони компенсують слабкості один одного.

Принципова новизна EDR порівняно з антивірусом — зміщення акценту з *запобігання* на *видимість* і *реагування*. Класичний антивірус працював за бінарною логікою «заблокувати або пропустити» в момент доступу до файлу й не зберігав історії: якщо загроза пройшла, слідів не лишалося. EDR натомість виходить із припущення, що частина загроз неодмінно обійде запобіжний рівень (припущення компрометації, assume breach), і тому безперервно записує активність, аби уможливити виявлення *постфактум*, розслідування й відновлення перебігу атаки. Це змінює саму роль засобу: він стає не лише бар'єром, а й «чорною скринькою» кінцевої точки, без якої розслідування інциденту (розділ 16) спиралося б лише на уцілілі артефакти диска. Нормативні настанови з обробки інцидентів ЗПК [205] прямо наголошують на цінності такого запису для реагування.

Архітектурно агент EDR працює на кількох рівнях привілеїв. Компоненти в *користувацькому* просторі перехоплюють виклики бібліотек і скриптові рушії; компоненти в *ядрі* (драйвери-монітори, нотифікації про створення процесів і завантаження образів) дають глибшу й важчу для обходу видимість. Сучасні операційні системи надають для цього захищені інтерфейси, щоб агент отримував події надійно й водночас сам був захищений від втручання ЗПК. Ця архітектурна вимога — наслідок того, що EDR є пріоритетною ціллю атаки: якщо зловмисник здатен вимкнути або обманути агента, уся подальша телеметрія втрачає довіру. Тому самозахист агента (цілісність, стійкість до вивантаження, виявлення спроб втручання) — невід'ємна частина його архітектури, а не додаткова функція.

Таблиця 10.1 — Рівні захисту сучасної кінцевої точки

Рівень	Функція	Приклад
Запобіжний	блокування відомого	сигнатури, репутація
Виявлювальний	розпізнавання підозрілого	поведінка, МН
Реагувальний	зупинення й відновлення	ізоляція, відкочування
Слідчий	запис для розслідування	телеметрія EDR

10.2 Телеметрія й джерела подій

Основа EDR — **телеметрія**: безперервний запис подій кінцевої точки. Ключові джерела в екосистемі Windows: *трасування подій* (ETW [154]) — вбудований механізм, що постачає події процесів, потоків, мережі, реєстру, завантаження модулів; Sysmon [188] — доповнення, що дає докладні, зручні для виявлення події (створення процесів із командним рядком і гешами, мережеві з'єднання, зміни реєстру, створення віддалених потоків — ознака ін'єкцій, розділ 5); *інтерфейс сканування* (AMSI [153]) — точка перехоплення скриптів і коду в пам'яті *після* деобфускації, що критично для виявлення безфайлових атак і обфускованих скриптів (розділ 8). Аналоги в інших ОС — аудит системних викликів, фреймворки спостереження ядра.

Цінність телеметрії визначається трьома чинниками: *повнотою* (чи фіксуються потрібні події), *достовірністю* (чи не може ЗПК її підробити або вимкнути) і *придатністю до аналізу* (структурованість, збагачення контекстом). Критична подія — створення процесу з повним командним рядком і батьківським процесом: саме ланцюги породження процесів («офісний застосунок → інтерпретатор команд → завантажувач») є характерним слідом атак. AMSI особливо цінний, бо перехоплює код *після* розгортання в пам'яті, обходячи обфускацію на рівні джерела.

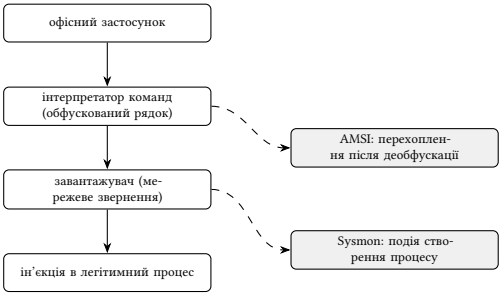


Рис. 10.2 — Підозрілий ланцюг породження процесів безфайлової атаки та точки спостереження телеметрії

Телеметрію слід розглядати не як простий журнал, а як *джерело сигналу*, з якого виявлення видобуває рідкісні ознаки серед переважно нормальної активності. Тому сучасні сенсори не лише фіксують подію, а й *збагачують* її контекстом: гешем виконуваного файлу й репутацією, повним шляхом і цифровим підписом, батьківським ланцюгом, обліковим записом, геоприв'язкою мережевого з'єднання. Збагачена подія різко підвищує якість подальшого виявлення й пришвидшує розслідування, бо аналітикові не доводиться добирати контекст вручну. Водночас обсяг збагаченої телеметрії величезний, що ставить задачу балансу між глибиною запису, вартістю зберігання й швидкістю пошуку — інженерний компроміс, який визначає практичну придатність EDR у великій інфраструктурі.

10.3 Виявлення й реагування

На зібраній телеметрії EDR застосовує методи виявлення: *правила* поведінки (розділ 11) — наприклад, Sigma-правила, що описують підозрілі патерни подій; *машинне навчання* (розділ 9) — класифікація й виявлення аномалій; *зіставлення з ATT&CK* [159] — мапування подій на техніки для структурованого розуміння. Ключова перевага перед файловим скануванням — виявлення за *поведінкою* незалежно від наявності файлу, що ловить безфайлові атаки, зловживання легітимними інструментами й невідомі зразки.

Виявлена загроза запускає **реагування**. Автоматичне: ізоляція вузла від мережі (зупинення поширення й зв'язку з командним сервером), зупинення й карантин процесу, відкочування змін. Напівавтоматичне: сповіщення аналітика центру безпеки з контекстом для рішення. Швидкість реагування безпосередньо впливає на збитки: що менший час від виявлення до стримування, то менше встигне зробити загроза. Цей показник пов'язаний із часом присутності (dwell time, розділ 13): EDR скорочує обидва — час до виявлення й час до стримування.

Сукупний час присутності загрози зручно розкласти на складові, що відповідають фазам реагування:

$$T_{\text{dwell}} = T_{\text{detect}} + T_{\text{triage}} + T_{\text{respond}}, \tag{10.1}$$

де T_{detect} — час від компрометації до появи сигналу, T_{triage} — час на підтвердження й пріоритезацію аналітиком, T_{respond} — час на стримування. Формула (10.1) показує, що EDR скорочує dwell time не лише кращим виявленням (T_{detect}), а й автоматизацією триажу та реагування (T_{triage}, T_{respond}): навіть за ідеального виявлення ручний триаж і повільне реагування залишають загрозі час на дію. Саме тому автоматичне стримування й збагачення подій контекстом — не додаткова зручність, а прямий важіль зниження збитків.

Окремо варто наголосити на ролі *кореляції* в XDR. Одинична подія на кінцевій точці часто неоднозначна: запуск інтерпретатора команд сам собою законний. Але та сама подія в поєднанні з попереднім підозрілим листом, аномальним мережевим з'єднанням і наступною спробою бокового переміщення складається в однозначну картину атаки. Кореляція різнорідних джерел підвищує і повноту (видно те, що кожне джерело окремо пропускає), і точність (контекст відсікає хибні тривоги), частково пом'якшуючи проблему базової частоти саме через вимогу збігу кількох незалежних сигналів — той самий принцип, що лежить в основі надійних правил YARA (розділ 6) та багатоознакового МН (розділ 9).

10.4 Профілактичні заходи й зниження поверхні атаки

Найдешевша протидія — не допустити виконання ЗПК узагалі. **Зниження поверхні атаки** зменшує кількість можливих векторів: вимкнення непотрібних функцій (макроси з інтернету, застарілі протоколи), обмеження запуску скриптів, блокування типових ланцюгів експлуатації. **Контроль застосунків** (application control, whitelisting [196]) дозволяє виконання лише явно дозволеним програмам — радикально ефективний проти невідомого коду, бо ЗПК просто не отримує дозволу на запуск, але вимагає дисциплінованого супроводу переліку. Принцип **найменших привілеїв** [190] обмежує права процесів і користувачів мінімально необхідними, зменшуючи вплив скомпрометованого коду.

Профілактика має принципову перевагу перед виявленням: вона діє *до* виконання й не залежить від здатності розпізнати конкретну загрозу. Контроль застосунків не потребує знати, чим саме є невідомий файл, — достатньо, що його немає в переліку дозволених. Найменші привілеї не запобігають запуску коду, але обмежують його наслідки: завантажувач, що виконується з правами звичайного користувача, не може встановити драйвер ядра чи змінити системні області, а отже позбавлений цілого класу технік персистентності й приховування (розділ 5). Ця властивість робить профілактичні заходи особливо цінними проти *нових* загроз, яких жоден детектор ще не знає. Їх недолік — операційне тертя: суворий whitelisting ускладнює легітимні зміни, а жорсткі обмеження привілеїв — адміністрування. Тому на практиці профілактику калібрують за критичністю активу: максимально сувору на серверах і спеціалізованих вузлах (розділ 14), поміркованішу на робочих станціях.

Окремий профілактичний рубіж — своєчасне *оновлення* та усунення вразливостей, що закриває вектори початкового доступу (експлойти), якими ЗПК потрапляє у систему. Жоден захист кінцевої точки не компенсує повністю невиправлену критичну вразливість, через яку код виконується з високими привілеями ще до того, як поведінкове виявлення встигне спра-

цювати. Тому гартування конфігурації, керування вразливостями й захист кінцевих точок утворюють взаємодоповнювану трійцю: перші два зменшують імовірність і наслідки компрометації, третій виявляє й стримує те, що все ж відбулося.

Ефективність цих заходів зручно оцінити через модель зниження ризику. Якщо базова ймовірність успішного ураження за один вектор становить p, а незалежні профілактичні заходи блокують вектори з ефективностями $e_1, \ldots, e_m$, то залишкова ймовірність ураження

$$p_{\text{зал}} = p \cdot \prod_{j=1}^{m} (1 - e_j), \tag{10.2}$$

показує мультиплікативне зниження ризику за ешелонованого захисту: кожен додатковий незалежний рівень множить залишковий ризик на $(1 - e_j) < 1$. Формула (10.2) формалізує принцип ешелонування й пояснює, чому поєднання кількох недосконалих заходів дає високу сукупну стійкість — за умови *незалежності* рівнів (спільна вразливість руйнує добуток).

10.5 Обмеження й протидія ухиленню

Захист кінцевих точок не є абсолютним. EDR сам стає ціллю: ЗПК намагається виявити й вимкнути агента, підробити або заглушити телеметрію (зокрема обходом ETW та AMSI), діяти в прогалинах видимості. Емпіричні оцінки стійкості EDR проти технік APT [131] показують, що проти цілеспрямованого супротивника жоден продукт не ловить усього. Крім того, EDR генерує величезний потік подій, що загострює проблему базової частоти (розділ 3): надмірна чутливість топить аналітиків у хибних тривогах, недостатня — пропускає загрози. Тому EDR потребує налаштування, кваліфікованих аналітиків і процесів (розділ 16).

Проблема потоку подій заслуговує окремого розгляду, бо вона визначає практичну придатність EDR не менше, ніж якість алгоритмів виявлення. Велика інфраструктура генерує мільярди подій на добу, з яких переважна більшість — нормальна активність. Виявлення мусить знайти серед них рідкісні зловмисні патерни, не затопивши аналітиків хибними тривогами. Звідси випливає кілька практичних наслідків. По-перше, правила й моделі налаштовують під конкретне середовище: те, що в одній організації є аномалією, в іншій — буденна робота адміністраторів. По-друге, застосовують багаторівневу фільтрацію й агрегацію: дешеві правила відсівають очевидно нормальне, дорогі методи й ручний аналіз застосовують до залишку — та сама каскадна логіка, що й у статичному аналізі (розділ 6). По-третє, сигнали пріоритезують за ризиком активу й достовірністю, щоб обмежений ресурс аналітиків спрямувати на найважливіше. Без цих захо-

дів навіть технічно досконалий EDR стає марним через «втому від тривог», коли реальні загрози тонуть у шумі.

Нарешті, EDR не існує ізольовано, а є сенсором і виконавчим органом ширшої системи керування безпекою. Його телеметрія стікається в системи кореляції й аналітики (SIEM/XDR), збагачується розвідкою загроз (розділ 12), а рішення про реагування вбудовані в організаційні процеси (розділ 16) та стратегію (розділ 17). Технічна досконалість агента — необхідна, але недостатня умова: без зрілих процесів, кваліфікованих людей і актуальної розвідки найкращий EDR реалізує лише частину свого потенціалу.

Фундаментальне обмеження — залежність від *видимості*: EDR бачить лише те, що фіксує телеметрія, тому прогалини в покритті (невстановлений агент, непідтримувана платформа, вимкнене джерело подій) є сліпими зонами. Звідси випливає принцип: захист кінцевих точок доповнюють мережевим моніторингом і сегментацією (щоб бачити й стримувати те, що уникло кінцевої точки), а також гартуванням конфігурації, щоб зменшити залежність від реактивного виявлення. Вітчизняні дослідження розглядають архітектуру й можливості EDR у корпоративних мережах [4] та моніторинг аномальної активності на вузлах [25].

10.6 Кейс: виявлення безфайлової атаки за телеметрією

Розглянемо виявлення безфайлової атаки, що не залишає файлу на диску й тому невидима файловому скануванню (захисна задача).

Крок 1. Збір телеметрії. EDR фіксує подію: офісний документ породжує інтерпретатор команд із підозрілим, обфускованим командним рядком — нетиповий ланцюг породження процесів.

Крок 2. Перехоплення в пам'яті. AMSI [153] перехоплює скрипт *після* деобфускації, розкриваючи справжній код завантажувача, попри обфускацію джерела (розділ 8).

Крок 3. Кореляція. Система зіставляє події: батьківський процес (офісний застосунок), командний рядок, мережеве звернення до зовнішнього вузла, спробу ін'єкції в легітимний процес — і мапує їх на техніки ATT&CK, формуючи цілісну картину атаки.

Крок 4. Реагування. За правилом спрацьовує автоматичне стримування: ізоляція вузла від мережі, зупинення ланцюга процесів, збереження артефактів пам'яті для розслідування (розділ 16).

Крок 5. Збагачення й поширення. З інциденту видобувають індикатори (адреси, геші, патерни команд), передають у розвідку загроз (розділ 12) і правила виявлення (розділ 11) для захисту решти вузлів.

Кейс демонструє силу поведінкового моніторингу: безфайлова атака, невидима для файлового сканування, виявляється за ланцюгом подій і перехопленням у пам'яті, а кореляція й швидке реагування мінімізують збитки.

Водночас успіх залежить від повноти телеметрії — прогалина в будь-якій ланці стала б сліпою зоною.

Висновки до розділу

Захист кінцевих точок реалізує методи виявлення там, де ЗПК виконується, і еволюціонував від файлового антивірусу до EDR і XDR — безперервного моніторингу, кореляції подій і реагування в масштабі інфраструктури. Сучасна кінцева точка поєднує запобіжний, виявлювальний, реагувальний і слідчий рівні (табл. 10.1) за принципом ешелонованого захисту. Основа EDR — телеметрія (ETW, Sysmon, AMSI та аналоги), цінність якої визначають повнота, достовірність і придатність до аналізу; ключовим слідом атак є ланцюги породження процесів, а AMSI перехоплює код після деобфускації. На телеметрії застосовують правила, MH і зіставлення з ATT&CK, виявляючи загрози за поведінкою незалежно від наявності файлу, із швидким автоматичним реагуванням. Профілактика — зниження поверхні атаки, контроль застосунків і найменші привілеї — дає мультиплікативне зниження ризику (10.2) за незалежності рівнів. Обмеження — EDR сам є ціллю, залежить від видимості й генерує потік подій, що загострює проблему базової частоти. Головний висновок: захист кінцевих точок — центральна, але не самодостатня ланка; його сила у поєднанні поведінкового моніторингу, профілактики, кореляції й швидкого реагування, доповнених мережевим рівнем і кваліфікованими процесами.

Питання для самоперевірки

1. Чому кінцева точка є найбагатшою на інформацію лінією захисту? Як еволюціонував захист від антивірусу до EDR/XDR?
2. У чому межі класичного файлового антивірусу? Які загрози він принципово не ловить?
3. Опишіть рівні захисту сучасної кінцевої точки (табл. 10.1). Як вони втілюють ешелонований захист?
4. Що таке телеметрія? Назвіть ключові джерела подій (ETW, Sysmon, AMSI) та їхню роль.
5. Чому ланцюг породження процесів є характерним слідом атак? Чим особливо цінний AMSI?
6. Які методи виявлення застосовує EDR на телеметрії? У чому перевага виявлення за поведінкою перед файловим скануванням?
7. Опишіть види реагування EDR. Як швидкість реагування пов'язана з часом присутності (dwell time)?

8. Що таке зниження поверхні атаки, контроль застосунків і найменші привілеї? Чому whitelisting радикально ефективний?
9. Поясніть формулу залишкового ризику (10.2). Чому незалежність рівнів критична для ешелонування?
10. Якими способами ЗПК протидіє EDR? Чому жоден продукт не ловить усього проти APT?
11. Як проблема базової частоти проявляється в потоці подій EDR? Що потрібно для її подолання?
12. Поясніть залежність EDR від видимості. Чому захист кінцевих точок доповнюють мережевим рівнем?
13. Опишіть виявлення безфайлової атаки за телеметрією (кейс підрозділу 10.6). Яку роль відіграють AMSI та кореляція?
14. Дослідницьке: оцініть покриття технік ATT&CK телеметрією обраної конфігурації Sysmon і запропонуйте доповнення для усунення сліпих зон.
15. Дослідницьке: дослідіть вплив налаштування чутливості правил EDR на баланс повноти й частоти хибних тривог на реалістичному потоці подій.

РОЗДІЛ 11

Інженерія виявлення: правила, індикатори та безфайлові загрози

Методи виявлення (розділи 6–10) стають дієвими лише тоді, коли втілені в конкретні, супроводжувані, вимірювані **правила виявлення**. **Інженерія виявлення** (detection engineering) — це систематична дисципліна створення, тестування, розгортання й супроводу логіки виявлення, що розглядає правила як код: із контролем версій, тестуванням, метриками якості й життєвим циклом. Цей розділ поєднує попередні методи в практику: він розглядає мови й рівні правил, «піраміду болю» та ієрархію індикаторів, виявлення безфайлових атак і зловживання легітимними інструментами (living-off-the-land), життєвий цикл і метрики виявлення, а також покриття технік ATT&CK, і завершується кейсом розроблення виявлення для конкретної техніки. Результати живлять захист кінцевих точок (розділ 10), розвідку загроз (розділ 12), полювання за загрозами (розділ 13) і реагування (розділ 16).

11.1 Правила виявлення як код

Інженерія виявлення запозичує практики розробки ПЗ: правило має версію, автора, опис, тести, метрики й життєвий цикл. Це протиставляється стихійному накопиченню несупроводжуваних правил, що з часом деградують (хибні тривоги, застарілість). Правила класифікують за рівнем даних, на якому вони працюють: *файлові* (YARA, розділ 6) — над вмістом і структурою файлів; *подієві/хостові* (Sigma [176]) — над журналами подій кінцевих точок (розділ 10); *мережеві* (Snort [185], Suricata [173]) — над мережевим трафіком. Кожен рівень бачить свій зріз активності, тому повне покриття вимагає правил на всіх рівнях.

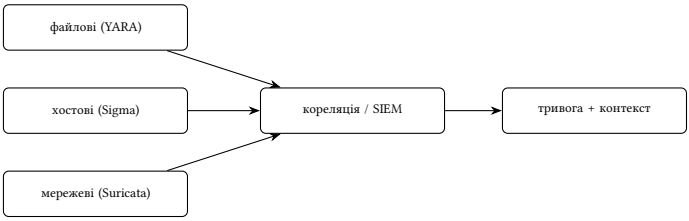


Рис. 11.1 — Рівні правил виявлення й їх злиття в кореляційній системі

Якість правила визначають метриками детектора (розділ 3): повнотою, точністю, частотою хибних тривог. Інженерія виявлення додає й операційні характеристики: вартість обчислення (чи встигає правило в реальному часі), супроводжуваність (чи зрозуміле воно, чи легко оновити), стійкість до ухилення (наскільки легко обійти). Зріле правило документує не лише *що* воно шукає, а й *чому*, на яку техніку спрямоване та які відомі способи обходу.

Перенесення практик розробки ПЗ на виявлення (detection-as-code) має конкретні наслідки. *Контроль версій* зберігає історію змін правила, дозволяє відкотити невдале оновлення й пов'язати зміну з причиною (нова загроза, хибні тривоги). *Автоматизоване тестування* прогонить правило проти набору злоякісних зразків (перевірка повноти) і проти «чистого» еталонного трафіку (перевірка на хибні тривоги) перед кожним розгортанням — так само, як модульні тести захищають код від регресій. *Рецензування* (code review) правил іншими інженерами виявляє логічні помилки, надмірну специфічність і прогалини. *Безперервна інтеграція* автоматизує цей конвеєр. Без цих практик бібліотека правил великої організації швидко стає некерованою: ніхто не пам'ятає, навіщо існує те чи інше правило, які з них дублюються, а які давно не спрацьовують, — і сукупна якість виявлення непомітно деградує.

Важливо також розрізняти *атомарні* та *кореляційні* правила. Атомарне правило спрацьовує на одній події чи ознаці (підозрілий командний рядок, звернення до відомого шкідливого домену). Кореляційне правило поєднує кілька подій у часі та за сутностями (той самий вузол, той самий користувач) — наприклад, «фішинговий лист, а згодом на тому ж вузлі аномальний процес і вихідне з'єднання». Атомарні правила прості, швидкі, але шумні; кореляційні — точніші й стійкіші, але складніші в розробленні й потребують якісної, збагаченої телеметрії (розділ 10). Зріла програма виявлення поєднує обидва типи.

11.2 Піраміда болю та ієрархія індикаторів

Ключова концепція інженерії виявлення — **піраміда болю** [63], що впорядковує індикатори за тим, наскільки *болісно* зловмиснику їх змінити (а отже, наскільки стійке виявлення на їх основі). Знизу вгору: геші файлів (змінити тривіально — один байт), IP-адреси (легко — змінити інфраструктуру), доменні імена (трохи складніше), мережеві й хостові артефакти (складніше), інструменти (ще складніше — потрібно переписати чи замінити), і на вершині — **тактики, техніки й процедури** (TTP), змінити які найважче, бо це вимагає змінити сам спосіб дій. Цю ієрархію узагальнено в табл. 11.1.

Таблиця 11.1 — Піраміда болю: ієрархія індикаторів за стійкістю

Рівень	Індикатор	Вартість зміни
Нижній	геші файлів	тривіальна
	IP-адреси, домени	низька
Середній	артефакти хоста/мережі	помірна
	інструменти	висока
Вершина	TTP (техніки, процедури)	дуже висока

Практичний висновок: виявлення на нижніх рівнях (геші, адреси) дешеве й точне, але нестійке — зловмисник обходить його миттєво. Виявлення на вершині (поведінкові патерни TTP) дороге в розробленні, але стійке, бо змушує зловмисника змінити спосіб дій. Тому зріла стратегія виявлення інвестує у *поведінкові* правила над техніками, тоді як індикатори нижніх рівнів використовує для швидкого, але тимчасового блокування відомого. Це пряме продовження принципу семантичної стійкості (розділи 6, 8).

Піраміда болю має й стратегічний вимір: вона описує, як виявлення *впливає на економіку* атаки. Блокування гешу змушує зловмисника перекомпілювати зразок — справа хвилин. Блокування інфраструктури (адрес, доменів) коштує дорожче, але теж посильне. А ось виявлення, що спирається на саму техніку, ставить зловмисника перед вибором: або відмовитися від перевіреного способу дій і розробити новий (дорого, ризиковано), або щоразу спрацьовувати на виявленні. Саме тому мета зрілого виявлення — не стільки зупинити конкретний зразок, скільки *підняти вартість* усієї операції настільки, щоб вона стала невигідною або помітною. Це перегукується з економічним поглядом на безпеку: абсолютний захист недосяжний, але можна зробити атаку дорожчою за її очікувану вигоду.

Ефективність набору правил зручно характеризувати очікуваним числом виявлених атак. Якщо техніка реалізується з імовірністю q_i у типовій атаці, а правило на неї має повноту r_i, то очікувана частка атак, у яких спрацює хоча б одне правило з набору незалежних технік,

$$P_{\text{виявл}} = 1 - \prod_i \bigl(1 - q_i r_i\bigr), \tag{11.1}$$

зростає з розширенням покриття технік. Формула (11.1) формалізує цінність *широти* покриття: виявлення, що спирається на кілька незалежних технік атаки, надійніше за будь-яке одиничне правило, бо зловмисник рідко може уникнути одразу всіх поведінкових слідів.

11.3 Безфайлові загрози та living-off-the-land

Сучасні атаки дедалі частіше **безфайлові** [138]: код виконується в пам'яті, не залишаючи файлу на диску, а отже невидимий файловому скануванню (розділ 6). Спорідена тактика — **living-off-the-land** (LotL): зловживання легітимними, вбудованими в систему інструментами (інтерпретаторами команд і сценаріїв, системними утиліами адміністрування), занесеними в проєкт LOLBAS [146]. Привабливість для зловмисника: немає стороннього файлу для виявлення, дії зливаються з легітимною адміністративною активністю, обхід контролю застосунків (бо інструменти дозволені).

Виявлення цих загроз неможливе на рівні файлів і спирається на поведінку (розділи 7, 10): аномальні ланцюги породження процесів, підозрілі командні рядки, нетипове використання легітимних інструментів (наприклад, системна утиліта, що звертається до інтернету й запускає код), перехоплення коду в пам'яті (AMSI, розділ 10). Виявлення обфускованих сценаріїв застосовує статистичний аналіз самого коду сценарію [66]: висока ентропія, аномальна структура, рідкісні конструкції. Ключ — не наявність файлу, а *невідповідність* поведінки нормальному профілю: легітимний інструмент, застосований нетиповим чином і в нетиповому контексті, є сигналом.

Безфайловість і LotL ілюструють глибший зсув у ландшафті загроз: межа між «шкідливим» і «легітимним» дедалі частіше пролягає не у *природі* коду, а в *контексті* його застосування. Інтерпретатор сценаріїв, утиліта віддаленого керування, засіб адміністрування самі по собі законні й потрібні; зловмисними їх робить те, *хто*, *звідки*, *в якій послідовності* й *з якою метою* їх запускає. Це принципово ускладнює виявлення, бо не існує «поганого файлу», який можна занести до чорного списку, — є лише підозрілі *патерни використання* звичайних засобів. Тому виявлення зміщується від аналізу об'єктів до аналізу *поведінки суб'єктів* і потребує моделі нормальної діяльності, відхилення від якої й породжує сигнал. Саме тут стають незамінними поведінкова аналітика та кореляція (розділ 10), а також знання типових дій адміністраторів у конкретному середовищі, без якого неможливо відрізнити легітимну адміністративну активність від замаскованої під неї атаки.

Обфускація сценаріїв — супутня проблема. Зловмисники приховують справжні команди шарами кодування, конкатенації, динамічного складання рядків (розділ 8), щоб обійти прості правила над текстом команди. Відповідь двоїста: по-перше, перехоплення коду *після* деобфускації на рівні рушія сценаріїв (AMSI, розділ 10), коли справжні команди вже розгорнуті; по-друге, статистичне виявлення самого факту обфускації [66] — аномально висока ентропія, нетипова довжина лексем, рідкісні конструкції, — бо

сильно обфускований адміністративний сценарій сам по собі підозрілий, незалежно від того, що саме він робить.

11.4 Життєвий цикл і метрики виявлення

Правило виявлення має **життєвий цикл**: гіпотеза (яку загрозу й за якими ознаками виявляти) → розроблення → тестування (на зразках загрози й на легітимному трафіку) → розгортання → моніторинг якості → налаштування або виведення з експлуатації. Правила старіють: загрози еволюціонують (дрейф, розділ 9), середовище змінюється, тому несупроводжуване правило з часом або перестає виявляти (низька повнота), або починає шуміти (низька точність).

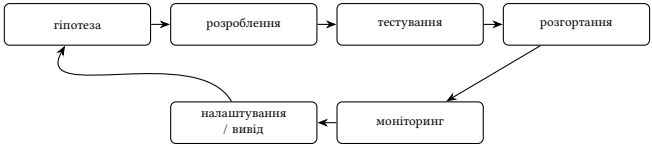


Рис. 11.2 — Життєвий цикл правила виявлення як безперервний інженерний процес

Метрики інженерії виявлення поділяють на якість окремих правил (повнота, точність, хибні тривоги) і характеристики програми виявлення загалом: *покриття* (яку частку технік ATT&CK покрито правилами), *глибину* (скільки незалежних сигналів на техніку), *час виявлення* (складова dwell time (10.1)), *операційне навантаження* (потік тривог на аналітика). Для оцінювання співвідношення корисного сигналу й шуму застосовують показник на кшталт частки істинних спрацювань серед усіх:

$$\mathrm{PPV} = \frac{TP}{TP + FP}, \tag{11.2}$$

де TP — істинні, FP — хибні спрацювання. Низький PPV означає, що аналітики витрачають час на хибні тривоги; інженерія виявлення прагне максимізувати PPV за прийнятної повноти, що знову повертає до проблеми базової частоти (розділ 3) й необхідності кореляції кількох ознак.

Практичний наслідок — потреба у *зворотному зв'язку* з експлуатації до розроблення. Кожна тривога, яку аналітик позначив як хибну або істинну, — це розмічений приклад, що дозволяє виміряти реальний PPV правила й скоригувати його. Без такого замкнутого контуру інженер виявлення працює наосліп, оцінюючи правила лише на штучних наборах, які не відображають справжнього розподілу подій у середовищі. Тому зрілі програми інтегрують виявлення з робочим процесом центру реагування (розділ 16): вердикти аналітиків повертаються як метрики якості правил, живлять їх

налаштування й навіть навчальні вибірки для МН-компонентів (розділ 9). Так інженерія виявлення стає не одноразовим написанням правил, а безперервним циклом, у якому експлуатація й розроблення взаємно живлять одне одного, а якість виявлення вимірюється й покращується систематично, а не інтуїтивно.

11.5 Покриття технік і баланс

Зрілі програми виявлення оцінюють і візуалізують **покриття** матриці ATT&CK [159], виявляючи сліпі зони — техніки, для яких немає виявлення. Проте покриття не самоціль: важливіша якість виявлення критичних технік, ніж формальне «замальовування» матриці. Оцінки зрілості виявлення [210] та настанови з організації центрів безпеки [231] наголошують на балансі широти (покриття), глибини (надійність на техніку) та операційної придатності (прийнятний рівень шуму). Українські практики систематизують розроблення правил для SIEM [44] та формування й застосування індикаторів компрометації [47].

Фундаментальний баланс інженерії виявлення — між повнотою й точністю під операційним обмеженням потоку тривог. Надто агресивні правила дають високу повноту, але топлять аналітиків у хибних тривогах (руйнуючи PPV (11.2)); надто консервативні — точні, але пропускають. Розв'язання — не одне «ідеальне» правило, а *набір* правил різної специфічності, кореляція кількох сигналів (розділ 10) та пріоритезація за ризиком, що дозволяє поєднати широке покриття з керованим потоком тривог.

11.6 Кейс: розроблення виявлення для техніки

Розглянемо розроблення виявлення для техніки зловживання легітимним інструментом (захисна задача, за життєвим циклом підрозділу 11.4).

Крок 1. Гіпотеза. На основі розвідки загроз (розділ 12) обираємо техніку ATT&CK (наприклад, виконання коду через системну утиліту, не призначену для цього) і формулюємо, за якими спостережуваними ознаками вона проявляється.

Крок 2. Розроблення. Пишемо хостове правило (Sigma) над телеметрією створення процесів (розділ 10): легітимна утиліта з нетиповими аргументами, нетиповим батьківським процесом чи мережевою активністю. Орієнтуємося на поведінкові ознаки (вершина піраміди болю), а не на геші.

Крок 3. Тестування. Перевіряємо повноту на зразках техніки (зокрема варіаціях) і точність на великому обсязі легітимного трафіку, щоб оцінити хибні тривоги. Налаштовуємо умови для прийнятного PPV (11.2).

Крок 4. Розгортання й моніторинг. Розгортаємо правило, стежимо за якістю в експлуатації, збираємо зворотний зв'язок від аналітиків, коригуємо за появи хибних тривог чи обходів.

Крок 5. Супровід. Оновлюємо правило в міру еволюції техніки, документуємо відомі обходи, за втрати ефективності виводимо з експлуатації або замінюємо. Оцінюємо внесок правила в загальне покриття (11.1).

Кейс ілюструє інженерний характер виявлення: не разовий запис, а керований життєвий цикл із гіпотезою, тестуванням, метриками й супроводом, орієнтований на стійкі поведінкові ознаки та баланс повноти, точності й операційного навантаження.

Висновки до розділу

Інженерія виявлення перетворює методи виявлення на конкретні, супроводжувані, вимірювані правила, розглядаючи їх як код із версіями, тестами, метриками й життєвим циклом. Правила працюють на різних рівнях — файловому (YARA), хостовому (Sigma), мережевому (Suricata), — і повне покриття вимагає їх поєднання та кореляції. Піраміда болю (табл. 11.1) впорядковує індикатори за стійкістю: виявлення на вершині (поведінкові TTP) дороге, але стійке, тоді як геші й адреси дешеві, але нестійкі, тому зріла стратегія інвестує в поведінкові правила (11.1). Безфайлові й living-off-the-land загрози невидимі на рівні файлів і виявляються лише за поведінкою — аномальними ланцюгами процесів, командними рядками, перехопленням у пам'яті. Правило має життєвий цикл від гіпотези до виведення з експлуатації й оцінюється метриками якості (повнота, точність, PPV (11.2)) та програми (покриття, глибина, час виявлення, навантаження). Фундаментальний баланс — між повнотою й точністю під обмеженням потоку тривог — розв'язують набором правил, кореляцією й пріоритезацією, а не пошуком одного ідеального правила. Головний висновок: ефективне виявлення — це інженерна дисципліна, орієнтована на стійкі поведінкові ознаки, широке й глибоке покриття технік та кероване операційне навантаження.

Питання для самоперевірки

1. Що таке інженерія виявлення? Чому правила розглядають як код?
2. Назвіть рівні правил виявлення (файлові, хостові, мережеві) та приклади мов. Чому потрібне поєднання рівнів?
3. Які операційні характеристики правила додає інженерія виявлення до метрик детектора?
4. Опишіть піраміду болю (табл. 11.1). Чому виявлення на вершині стійкіше за виявлення за гешами?
5. Поясніть формулу покриття (11.1). Чому широта покриття технік підвищує надійність виявлення?
6. Що таке безфайлові атаки та living-off-the-land? Чому вони привабливі для зловмисника?

7. Як виявляють безфайлові та LotL-загрози? Чому ключем є невідповідність поведінки нормальному профілю?
8. Опишіть життєвий цикл правила виявлення. Чому несупроводжуване правило деградує?
9. Які метрики характеризують програму виявлення загалом? Поясніть PPV (11.2) та його зв'язок з навантаженням на аналітиків.
10. Що таке покриття матриці ATT&CK? Чому формальне покриття не є самоціллю?
11. Як розв'язують баланс повноти й точності під обмеженням потоку тривог?
12. Опишіть розроблення виявлення для техніки (кейс підрозділу 11.6). Чому орієнтуються на поведінкові ознаки?
13. Як інженерія виявлення пов'язана з розвідкою загроз (розділ 12) і реагуванням (розділ 16)?
14. Дослідницьке: оцініть покриття обраного набору правил за матрицею ATT&CK, виявіть сліпі зони й запропонуйте пріоритети їх усунення.
15. Дослідницьке: порівняйте стійкість виявлення на рівні гешів та на рівні поведінкових TTP до варіацій одного сімейства ЗПК.

РОЗДІЛ 12

Розвідка загроз, індикатори компрометації та атрибуція

Виявлення й аналіз окремих зразків (розділи 6–11) дають фрагменти, які **розвідка загроз** (threat intelligence) об'єднує в цілісне, контекстуалізоване знання про супротивника: хто атакує, якими засобами, з якою метою, за якими ознаками їх розпізнати й передбачити. Розвідка загроз перетворює реактивне виявлення на проактивний захист: знаючи тактики, техніки й інфраструктуру ймовірних супротивників, захисник готується заздалегідь. Цей розділ розглядає поняття й рівні розвідки загроз, індикатори компрометації та їх життєвий цикл, стандарти обміну, моделі аналізу й атрибуції, її можливості й межі, а також роль CERT-UA в Україні, і завершується кейсом на основі кампаній WannaCry та NotPetya. Розвідка загроз живить інженерію виявлення (розділ 11), полювання за загрозами (розділ 13), реагування (розділ 16) і стратегічне управління (розділ 17).

12.1 Поняття й рівні розвідки загроз

Розвідка загроз — це засноване на доказах знання про наявні чи ймовірні загрози, придатне для прийняття рішень [151]. Ключові слова — *доказовість* (спирається на спостереження, а не припущення) і *придатність для рішень* (не дані заради даних, а знання, що змінює дії). Розрізняють три рівні за горизонтом і аудиторією (табл. 12.1): *тактичний* (технічні індикатори й техніки для негайного виявлення — для аналітиків і засобів захисту); *оперативний* (кампанії, інструменти, інфраструктура конкретних угруповань — для реагування й полювання); *стратегічний* (мотиви, можливості, тенденції супротивників — для керівництва й стратегії, розділ 17).

Таблиця 12.1 — Рівні розвідки загроз

Рівень	Зміст	Аудиторія
Тактичний	індикатори, техніки	аналітики, засоби
Оперативний	кампанії, інфраструктура	реагування, полювання
Стратегічний	мотиви, тенденції	керівництво

Цінність розвідки визначається не обсягом, а *релевантністю* й *своєчасністю*: індикатор, актуальний для моделі загроз організації (розділ 4) і отриманий вчасно, вартий більше за гори нерелевантних даних. Тому зріла розвідка починається з визначення пріоритетних розвідувальних вимог — які саме загрози важливі для цієї організації.

Розвідка загроз проходить класичний *розвідувальний цикл*: планування (визначення вимог) → збір (із власної телеметрії, обміну, відкритих і закритих джерел) → оброблення (нормалізація, дедублікація, збагачення) → аналіз (перетворення даних на знання) → поширення (доведення до споживачів у придатній формі) → зворотний зв'язок (оцінка корисності й уточнення вимог). Пропуск будь-якого етапу знижує цінність: збір без планування дає нерелевантні гори даних; аналіз без поширення лишає знання невикористаним; поширення без зворотного зв'язку не дозволяє вдосконалюватися. Саме дисциплінований цикл, а не доступ до «стрічок індикаторів», відрізняє зрілу розвідкову функцію від простого споживання чужих даних.

Важливо розрізняти *споживання* та *виробництво* розвідки. Більшість організацій починають зі споживання: підписуються на зовнішні канали індикаторів і звітів. Зрілі організації додають виробництво: аналізують власні інциденти (розділ 16), видобувають із них індикатори й TTP, збагачують і повертають у спільноту. Виробництво цінне тим, що власна телеметрія відображає саме ті загрози, які атакують цю організацію, — найрелевантніше джерело. Поєднання споживання й виробництва замикає контур: зовнішня розвідка спрямовує увагу, а внутрішній аналіз уточнює й доповнює її під власний контекст.

12.2 Індикатори компрометації та їх життєвий цикл

Індикатор компрометації (IoC) — спостережувана ознака, наявність якої свідчить про ймовірну компрометацію: геш шкідливого файлу, адреса чи домен командного сервера, ключ реєстру персистентності, патерн мережевого трафіку, рядок у пам'яті. IoC — робочий матеріал тактичної розвідки: ними наповнюють правила виявлення (розділ 11), їх перевіряють у телеметрії (розділ 10).

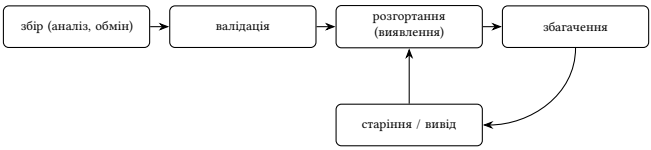


Рис. 12.1 — Життєвий цикл індикатора компрометації

IoC мають **життєвий цикл**: збір (з власного аналізу, обміну, відкритих джерел) → валідація (перевірка достовірності й релевантності) → розгортання → збагачення контекстом → старіння й виведення. Критична властивість — *старіння*: адреси й домени зловмисник швидко змінює (нижні рівні піраміди болю, розділ 11), тому старий IoC дає хибні тривоги (адресу

перепризначено легітимному сервісу) або пропуски. Тому IoC без терміну придатності й контексту — радше шум, ніж розвідка. Це знову підкреслює перевагу стійких поведінкових індикаторів (TTP) над ефемерними атомарними.

Старіння індикаторів зручно моделювати спаданням їх розвідувальної цінності в часі. Якщо початкову цінність свіжого індикатора позначити v_0, а темп старіння — сталою λ, характерною для типу індикатора, то цінність через час t після появи

$$v(t) = v_0\, e^{-\lambda t}, \tag{12.1}$$

де велике λ (швидке старіння) відповідає ефемерним індикаторам нижніх рівнів піраміди болю (адресам, доменам), а мале λ — стійким поведінковим ознакам (TTP). Формула (12.1) формалізує інтуїцію про «термін придатності» й обґрунтовує диференційовану політику: атомарні індикатори потребують частого оновлення й швидкого виведення, тоді як поведінкові зберігають цінність надовго. Половинний час життя індикатора $t_{1/2} = \ln 2/\lambda$ слугує практичним орієнтиром для планування перегляду бази індикаторів.

12.3 Стандарти обміну розвідкою

Цінність розвідки зростає з обміном: загроза, виявлена в одній організації, застерігає решту. Для машиночитаного обміну розроблено стандарти: **STIX** [169] — структурований формат опису загроз (індикатори, техніки, угруповання, кампанії та зв'язки між ними); **TAXII** [170] — протокол їх передавання. Платформи обміну, зокрема відкрита MISP [226], автоматизують накопичення, кореляцію й поширення розвідки між спільнотами й галузевими центрами.

Обмін має бар'єри: довіра (чи достовірне джерело), конфіденційність (індикатор може розкрити факт компрометації його джерела), швидкість (застарілий індикатор марний), якість (без контексту й валідації обмін поширює шум). Для керування чутливістю застосовують протоколи маркування (на кшталт TLP — traffic light protocol), що визначають, кому можна передавати інформацію. Попри бар'єри, колективний обмін є силовим множником: він зміщує асиметрію на користь захисників, дозволяючи вчитися на чужому досвіді, перш ніж зазнати тієї самої атаки.

Структуроване машиночитане подання розвідки має й самостійну цінність поза передаванням. Коли індикатори, техніки, інструменти, угруповання й кампанії описані в єдиній формальній моделі (як у STIX) зі зв'язками між сутностями, розвідка стає *графом знань*, що піддається автоматизованому аналізу: можна запитати, які кампанії застосовували даний інструмент,

яку інфраструктуру поділяють два угруповання, які жертви зазнали схожих TTP. Це перетворює розрізнені звіти на пов'язану базу, у якій нове спостереження автоматично зіставляється з накопиченим і або підтверджує відому кампанію, або сигналізує про нову. Без такої структуризації розвідка лишається купою текстів, корисність яких обмежена здатністю окремого аналітика все це пам'ятати.

12.4 Моделі аналізу й атрибуція

Для структурування аналізу застосовують моделі. **Діамантова модель** [70] описує подію через чотири вершини — супротивник, можливості (інструменти), інфраструктура, жертва — та зв'язки між ними, що допомагає від відомих елементів виводити невідомі (наприклад, від інфраструктури — до інших жертв). У поєднанні з ланцюгом ураження й ATT&CK (розділ 4) вона дає каркас для накопичення знань про угруповання.

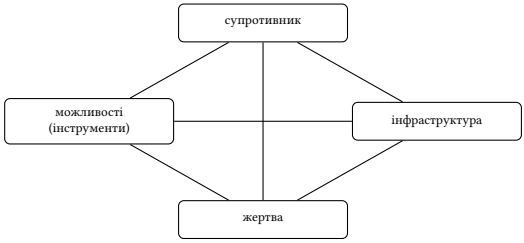


Рис. 12.2 — Діамантова модель вторгнення: чотири вершини та зв'язки, що дозволяють виводити невідомі елементи від відомих

Атрибуція — віднесення атаки до конкретного актора — є одним із найскладніших і найделікатніших завдань [184]. Вона спирається на сукупність свідчень: технічні (інфраструктура, повторно використаний код, артефакти збірки, мовні сліди, робочий час за часовим поясом), оперативні (вибір цілей, TTP) і контекстуальні (геополітичний мотив). Жодне свідчення саме не є доказом, тому атрибуція — це оцінка ймовірності за сукупністю, а не однозначне твердження. Кластеризацію активності в угруповання позначають умовними іменами (APT-номери, кодові назви) ще до встановлення реального актора.

Доказовий характер атрибуції зручно виразити через накопичення свідчень. Якщо кожне незалежне свідчення i підтримує гіпотезу про актора з вагою (інформативністю) w_i та спостерігається (значення індикатора $x_i \in \{0, 1\}$), то сукупну впевненість можна подати нормованою зваженою сумою

$$C = \frac{\sum_i w_i x_i}{\sum_i w_i} \in [0; 1], \tag{12.2}$$

що зростає з кількістю й вагомістю узгоджених свідчень. Формула (12.2) підкреслює дві речі: жодне окреме свідчення ($x_i = 1$ за малого w_i) не дає впевненості, а висока впевненість потребує сукупності незалежних свідчень; крім того, вагу w_i слід занижувати для ознак, які легко підробити (хибні прапори), — саме тому мовні сліди чи чужі інструменти важать менше за складно підроблювані оперативні патерни.

Межі атрибуції принципові. По-перше, *хибні прапори* [61]: зловмисник навмисно залишає сліди, що вказують на іншого (чужу мову, чужі інструменти, чужу інфраструктуру), аби ввести в оману. По-друге, повторне використання інструментів і витоки коду розмивають зв'язок «інструмент → актор». По-третє, атрибуція має політичні наслідки, тому потребує особливо високого стандарту доказовості. Для захисту точна атрибуція до держави чи особи часто й не потрібна: достатньо стійко розпізнавати *угруповання* за його TTP, щоб виявляти й випереджати його дії, незалежно від того, хто за ним стоїть.

12.5 Розвідка загроз в Україні: CERT-UA

Україна — полігон і ціль найактивніших у світі кампаній ЗПК, тому національна розвідка загроз має критичне значення. **CERT-UA** [17] — урядова команда реагування на комп'ютерні надзвичайні події у складі Держспецзв'язку (розділ 2) — збирає, аналізує й поширює індикатори та аналітичні звіти про атаки на українські організації, координує реагування й міжнародний обмін. Через безпрецедентну інтенсивність атак на критичну інфраструктуру (розділ 14) український досвід і розвідка загроз мають світове значення: багато кампаній, уперше виявлених в Україні, згодом загрожували всьому світу. Вітчизняні дослідження систематизують застосування розвідки загроз для проактивного захисту [14] та діяльність APT-угруповань проти критичної інфраструктури України [2].

Український контекст вирізняється кількома особливостями, важливими для розуміння розвідки загроз. По-перше, тут спостерігається рідкісна концентрація діяльності найспроможніших державних угруповань, що застосовують повний спектр — від шпигунства до деструктивних атак на енергетику (розділ 14). По-друге, саме в Україні вперше спостерігали техніки, які згодом поширилися світом (деструктивні вайпери під виглядом вимагачів, атаки на промислову автоматику), що робить українську розвідку випереджальним індикатором для глобальної спільноти. По-третє, висока інтенсивність атак прискорила розвиток практик швидкого обміну індикаторами між державним сектором, критичною інфраструктурою та міжна-

родними партнерами. Для магістранта-дослідника це означає, що українські дані — не локальний окремий випадок, а один із найбагатших у світі емпіричних матеріалів для вивчення сучасних загроз і методів протидії.

12.6 Кейс: від зразка до розвідки (WannaCry, NotPetya)

Розглянемо, як окремі спостереження перетворюються на розвідку, на прикладі двох кампаній 2017 року, що зачепили Україну й світ (захисна/аналітична задача).

Крок 1. Технічний аналіз. Статичний і динамічний аналіз (розділи 6–7) зразків WannaCry [163] і NotPetya [112] дає технічні деталі: механізм поширення (експлуатація мережевої вразливості), шифрування, індикатори. Виявляється, що NotPetya, попри вигляд вимагача, був деструктивним вайпером без реальної можливості відновлення — висновок, можливий лише через аналіз, а не з самозаявленої поведінки.

Крок 2. Індикатори. Формуються IoC: геші, мережеві патерни, домени, ознаки поведінки — для негайного виявлення (розділ 11).

Крок 3. Кампанія й контекст. Діамантова модель [70] і ATT&CK зводять спостереження в картину кампанії: вектор початкового доступу (для NotPetya — компрометація оновлення бухгалтерського ПЗ, тобто ланцюг постачання, розділ 15), цілі, географія, TTP.

Крок 4. Атрибуція. За сукупністю технічних і контекстуальних свідчень активність кластеризують і відносять до угруповання, з урахуванням ризику хибних прапорів [61]. Для NotPetya вибір цілей (переважно українські організації) і контекст є вагомими оперативними свідченнями.

Крок 5. Поширення й застосування. Розвідку (STIX/TAXII, через CERT-UA й спільноти) поширюють, щоб решта організацій застосувала її проактивно: оновила вразливе ПЗ, розгорнула виявлення, перевірила телеметрію ретроспективно.

Кейс показує повний шлях від зразка до дієвої розвідки й ілюструє ключові уроки: самозаявлена поведінка (вимагач) може маскувати справжню мету (деструкція); вектор ланцюга постачання робить навіть оновлення недовіреним; а своєчасний обмін розвідкою міг би обмежити глобальне поширення.

Висновки до розділу

Розвідка загроз об'єднує фрагменти виявлення й аналізу в контекстуалізоване знання про супротивника, перетворюючи реактивний захист на проактивний. Вона має три рівні — тактичний, оперативний, стратегічний (табл. 12.1), — а її цінність визначають доказовість, релевантність і своєчасність, а не обсяг. Індикатори компрометації — робочий матеріал тактичної розвідки — мають життєвий цикл зі старінням, через що ефемерні атомар-

ні IoC поступаються стійким поведінковим TTP. Обмін розвідкою (STIX, TAXII, MISP) є силовим множником, що зміщує асиметрію на користь захисників, попри бар'єри довіри, конфіденційності та якості. Моделі аналізу (діамантова, ATT&CK) структурують накопичення знань про угруповання; атрибуція спирається на сукупність свідчень як оцінка ймовірності й обмежена хибними прапорами та повторним використанням інструментів, тому для захисту часто достатньо розпізнавати угруповання за TTP. В Україні CERT-UA виконує критичну роль національної розвідки загроз, а український досвід має світове значення. Кейс WannaCry/NotPetya демонструє повний шлях від зразка до дієвої розвідки й уроки маскування мети, вектора ланцюга постачання та цінності своєчасного обміну. Головний висновок: розвідка загроз — це міст від технічного аналізу до стратегічних рішень, а її сила — у релевантності, контексті, стійких індикаторах і колективному обміні.

Питання для самоперевірки

1. Що таке розвідка загроз? Поясніть ключові ознаки доказовості й придатності для рішень.
2. Опишіть три рівні розвідки загроз (табл. 12.1) за горизонтом і аудиторією.
3. Чому цінність розвідки визначається релевантністю й своєчасністю, а не обсягом?
4. Що таке індикатор компрометації? Наведіть приклади й опишіть їх життєвий цикл.
5. Чому старіння IoC є критичною проблемою? Як це пов'язано з пірамідою болю (розділ 11)?
6. Які стандарти обміну розвідкою існують (STIX, TAXII, MISP)? Яку задачу розв'язує кожен?
7. Назвіть бар'єри обміну розвідкою. Чому попри них обмін є силовим множником?
8. Опишіть діамантову модель аналізу вторгнень. Як вона допомагає виводити невідомі елементи?
9. Що таке атрибуція? На які групи свідчень вона спирається й чому є оцінкою ймовірності?
10. Які принципові межі має атрибуція? Що таке хибні прапори? Чому для захисту достатньо розпізнавання угруповання за TTP?
11. Яку роль відіграє CERT-UA? Чому український досвід розвідки загроз має світове значення?

12. Опишіть шлях від зразка до розвідки на прикладі WannaCry/NotPetya (кейс підрозділу 12.6). Чому самозаявлена поведінка може маскувати мету?
13. Як розвідка загроз пов'язана з інженерією виявлення, реагуванням і стратегічним управлінням?
14. Дослідницьке: побудуйте профіль обраного APT-угруповання за відкритими джерелами, застосувавши діамантову модель і ATT&CK, та оцініть повноту й суперечливість атрибуції.
15. Дослідницьке: оцініть швидкість старіння різних типів IoC (гешів, доменів, TTP) на реальному потоці розвідки й обґрунтуйте політику їх виведення з експлуатації.

РОЗДІЛ 13

Цілеспрямовані сталі загрози та полювання за загрозами

Масові автоматизовані загрози (розділ 1) протистоять автоматизованим засобам захисту (розділи 6–11). Проте найнебезпечніший супротивник — **цілеспрямована стала загроза** (APT, advanced persistent threat): добре ресурсоване угруповання, що наполегливо, тривало й адаптивно досягає конкретних цілей попри захист. Проти APT реактивне виявлення недостатнє: такий супротивник уникає відомих сигнатур, зловживає легітимними інструментами (LotL, розділ 11) і може роками лишатися непоміченим. Відповіддю є **полювання за загрозами** (threat hunting) — проактивний, керований гіпотезами пошук прихованих загроз, що *не* спрацювали на автоматичному виявленні. Розділ розглядає феномен APT і його життєвий цикл, поняття часу присутності (dwell time), парадигму полювання за загрозами, його методологію й гіпотези, бокове переміщення й living-off-the-land як об'єкти полювання, зрілість і метрики, і завершується кейсом полювання за прихованим угрупованням. Полювання спирається на телеметрію (розділ 10), розвідку загроз (розділ 12) і живить реагування (розділ 16).

13.1 Феномен APT та його життєвий цикл

APT характеризують три ознаки: *advanced* (високі можливості — власні інструменти, експлойти, ресурси); *persistent* (наполегливість — угруповання не відступає після невдачі, а адаптується й повертається); *threat* (цілеспрямованість — за атакою стоїть організація з конкретними цілями, часто державна або пов'язана з державою). Перші задокументовані кейси кіберспіонажу [211] та відкриття діяльності державних угруповань [148] показали, що такі актори діють методично, за життєвим циклом ланцюга ураження (розділ 4): розвідка цілі, початковий доступ, закріплення, підвищення привілеїв, бокове переміщення, досягнення мети (шпигунство, саботаж), приховування слідів.

Відмінність APT від масового ЗПК — не стільки технічна витонченість кожного кроку, скільки *цілеспрямованість і наполегливість*. Угруповання вивчає конкретну жертву, адаптує інструменти під її середовище, діє повільно й обережно, зливаючись із легітимною активністю, щоб уникнути виявлення. Саме тому ранні аналізи [224] наголошували: типовий периметровий захист і сигнатурне виявлення систематично пропускають APT, бо ті не застосовують відомого шкідливого коду там, де його чекають. Відмінності APT від масових загроз узагальнено в табл. 13.2.

Таблиця 13.2 — APT та масовий ЗПК: порівняння

Ознака	Масовий ЗПК	APT
Ціль	будь-хто вразливий	конкретна жертва
Мотив	миттєва вигода	стратегічна мета
Наполегливість	разова спроба	адаптація, повернення
Інструменти	типові, відомі	власні, LotL
Виявлення	сигнатури ловлять	потребує полювання

Варто застерегти від спрощеного ототожнення APT з надскладними технологіями. Угруповання застосовує рівно стільки витонченості, скільки потрібно: якщо ціль беруть фішингом і викраденими паролями, дорогі експлойти нульового дня притримують. Саме економність і прагматизм роблять APT небезпечним — він не «світить» рідкісними інструментами там, де вистачає буденних, а отже й не лишає помітних слідів. Це пояснює, чому наголос у протидії зміщується з пошуку екзотичного шкідливого коду на виявлення *аномальної поведінки* цілком звичайних засобів.

13.2 Час присутності (dwell time)

Ключовий показник ефективності захисту проти APT — **час присутності** (dwell time): проміжок від компрометації до виявлення. Велике dwell time означає, що супротивник довго діяв непоміченим, встигнувши досягти цілей. Складові часу реагування формалізовано у (10.1); стратегічна мета захисту — мінімізувати dwell time, бо збитки від APT зростають із часом присутності.



Рис. 13.1 — Час присутності (dwell time) як проміжок від компрометації до виявлення

Зв'язок збитків із часом присутності часто монотонно зростаючий: що довше супротивник діє, то більше вузлів компрометує, то більше даних викрадає чи руйнує. Спрощену залежність очікуваних збитків D від dwell time T можна подати як

$$D(T) = D_0\big(e^{\gamma T} - 1\big), \tag{13.1}$$

де D_0 й $\gamma > 0$ — параметри, що відображають початковий темп і прискорення шкоди (бокове переміщення розширює доступ, збільшуючи й темп). Експоненційний характер (13.1) пояснює, чому навіть помірне скорочення

dwell time дає непропорційно велике зниження збитків, і робить раннє виявлення — а отже полювання — економічно виправданим попри його вартість.

Автоматичне виявлення скорочує dwell time для *відомих* загроз, але APT навмисно їх уникає. Тому для скорочення dwell time проти невідомого потрібен проактивний пошук — полювання. Емпірично організації з розвиненим полюванням виявляють APT значно раніше, ніж ті, що покладаються лише на реактивні сповіщення.

13.3 Парадигма полювання за загрозами

Полювання за загрозами [208] — проактивний, ітеративний, керований людиною пошук ознак компрометації, що уникли автоматичного виявлення. Його відмінність від реагування на сповіщення: полювання починається не з тривоги, а з *гіпотези* («якщо угруповання X діє в нашій мережі, ми побачимо ознаку Y»), яку аналітик перевіряє в даних. Полювання виходить із припущення компрометації (assume breach): замість питати «чи спрацювала тривога?», аналітик питає «що ми могли пропустити?».

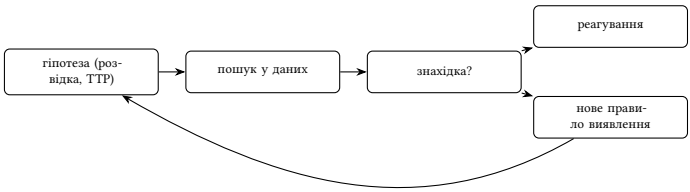


Рис. 13.2 — Цикл полювання за загрозами: від гіпотези до реагування й нового автоматизованого виявлення

Цінний продукт полювання — не лише знайдені загрози, а й *автоматизація*: успішна гіпотеза стає новим правилом виявлення (розділ 11), тож одноразовий ручний пошук перетворюється на постійне автоматичне виявлення. Так полювання й інженерія виявлення утворюють контур: полювання знаходить нове, виявлення автоматизує знайдене, звільняючи мисливців для пошуку наступного невідомого.

Полювання принципово спирається на *людину*, і це не тимчасове обмеження, а сутнісна риса. Автоматика чудова в розпізнаванні *відомих* патернів, але погано справляється з нечіткими, контекстуальними, уперше баченими ознаками, де потрібні інтуїція, доменне знання й творче формулювання гіпотез. Аналітик-мисливець привносить саме це: розуміння, що *мало б* статися за даної гіпотези, навіть якщо точного правила ще не існує. Тому полювання не конкурує з автоматизацією, а працює на її межі — там,

де сигнали занадто слабкі чи незвичні для наявних правил, — і поступово розширює автоматизовану зону, перетворюючи вчорашні відкриття на сьогоднішні правила. Ця взаємодія людини й машини — визначальна риса зрілого кіберзахисту проти мислячого супротивника.

13.4 Методологія й гіпотези

Гіпотези полювання походять із трьох джерел: *розвідки загроз* (розділ 12) — TTP відомих угруповань, релевантних моделі загроз; *аномалій* — статистичних відхилень у телеметрії; *обізнаності про середовище* — знання, що є нормою, і що — ні. Методично полювання спирається на багату телеметрію (розділ 10) й аналітику; репозиторії аналітик на кшталт MITRE CAR [161] надають готові запити для виявлення технік. Модель «хто, що, де, коли, чому, як» [142] структурує планування полювання.

Практично гіпотези полювання доцільно формулювати так, щоб вони були *спростовними* й *перевірними* наявними даними. Добра гіпотеза вказує конкретну очікувану ознаку та джерело, де її шукати: не розпливчасте «чи є в нас APT?», а «якщо застосовано техніку T, то в телеметрії створення процесів за останні 30 діб ми побачимо батьківсько-дочірній ланцюг виду P». Така постановка дозволяє або знайти підтвердження, або впевнено його відкинути, або виявити, що потрібних даних немає (прогалина видимості). Останній результат особливо цінний: він перетворює полювання на інструмент оцінювання якості телеметрії, спрямовуючи інвестиції в ті джерела подій, яких бракує для перевірки пріоритетних гіпотез. Відсутність даних — тиха, але небезпечна форма сліпоти, яку реактивний захист не помічає, бо не ставить питань, на які немає відповіді.

Окремий методологічний принцип — зосередження на техніках, стійких у піраміді болю (розділ 11). Полювання за ефемерними індикаторами (адресами, гешами) малоцінне: їх і так покриває автоматика, а зловмисник легко змінює. Натомість полювання за поведінковими TTP — закономірностями, які угруповання змушене повторювати через обмеженість способів досягнення мети, — дає стійкі результати й породжує довговічні правила. Тому зрілі програми пріоритезують гіпотези навколо технік, від яких супротивникові найважче відмовитися: механізмів закріплення, бокового переміщення, вивантаження даних.

Ефективність полювання зручно оцінити через приріст виявлюваності. Якщо автоматичне виявлення покриває техніки з повнотою r_{auto}, а полювання додає виявлення технік, що уникли автоматики, з повнотою r_{hunt} на решті, то сукупна повнота

$$r_{\Sigma} = r_{\text{auto}} + (1 - r_{\text{auto}})\, r_{\text{hunt}}, \tag{13.2}$$

перевищує будь-яку окремо. Формула (13.2) формалізує взаємодоповнення: полювання цінне саме там, де автоматика безсила (r_{auto} мала для нових TTP), тому інвестиція в полювання дає найбільший приріст проти витончених, невідомих загроз — тобто проти APT.

13.5 Бокове переміщення та living-off-the-land як об'єкти полювання

Найхарактерніші й найважчі для автоматичного виявлення дії APT — **бокове переміщення** (поширення мережею від початкового вузла до цілей) та зловживання легітимними інструментами (LotL, розділ 11; в Unix-середовищах — GTFOBins [115]). Бокове переміщення використовує легітимні механізми (віддалене виконання, викрадені облікові дані, адміністративні протоколи), тому зливається з нормальною активністю й погано ловиться сигнатурами. Його виявлення [201] спирається на аномалії: нетипові зв'язки між вузлами, незвичні часи, аномальне використання облікових записів.

Полювання за цими діями — показовий приклад переваги гіпотезокерованого підходу: замість чекати сигнатури аналітик висуває гіпотезу («скомпрометований обліковий запис використовується для доступу до нетипових вузлів») і шукає її підтвердження в графі автентифікацій і з'єднань. Ключ — модель норми: що є типовим для цього користувача, вузла, часу. Відхилення від норми, контекстуалізоване розвідкою й знанням середовища, і є здобиччю полювання. Українські практики розглядають полювання як складову стратегії кіберзахисту [36] та методи виявлення цілеспрямованих атак на критичну інфраструктуру [6].

13.6 Зрілість і метрики полювання

Зрілість полювання описують моделлю [141]: від нульового рівня (покладання лише на автоматичні сповіщення) через ситуативне й кероване розвідкою полювання до найвищого — автоматизованого, де успішні гіпотези систематично перетворюються на виявлення. Прогрес залежить від якості телеметрії, навичок аналітиків і процесів. Рівні зрілості узагальнено в табл. 13.1.

Таблиця 13.1 — Модель зрілості полювання за загрозами

Рівень	Характеристика	Основа
Початковий	лише автоматичні сповіщення	немає полювання
Мінімальний	включення індикаторів розвідки	зовнішня розвідка
Процедурний	регулярні процедури полювання	телеметрія, аналітики
Інноваційний	власні аналітики й гіпотези	розвинені навички
Провідний	автоматизація успішних гіпотез	контур із виявленням

Метрики полювання відрізняються від метрик виявлення: важлива не лише кількість знайдених загроз (успішне полювання може нічого не знайти, але підтвердити відсутність загрози й виявити прогалини видимості), а й *приріст покриття* (скільки нових правил породжено), *виявлені сліпі зони* (прогалини телеметрії), *скорочення dwell time*. Парадокс полювання: «порожнє» полювання не є провалом — воно або підтверджує безпеку, або виявляє, що бракує даних для перевірки гіпотези, що саме є цінним результатом. Тому полювання оцінюють за систематичністю й покриттям гіпотез, а не лише за «уловом».

13.7 Кейс: полювання за прихованим угрупованням

Розглянемо полювання за ознаками APT, що уникло автоматичного виявлення (захисна задача).

Крок 1. Гіпотеза. З розвідки загроз (розділ 12) відомо, що релевантне угруповання застосовує бокове переміщення через викрадені адміністративні облікові дані. Формулюємо гіпотезу: «якщо угруповання діє, ми побачимо нетипові віддалені входи адміністративним обліковим записом».

Крок 2. Дані. Збираємо телеметрію автентифікацій, створення процесів і мережевих з'єднань (розділ 10) за достатній період.

Крок 3. Пошук аномалій. Будуємо модель норми (типові вузли, часи, облікові записи) і шукаємо відхилення: вхід адміністратора на вузли, куди він ніколи не заходив, у неробочий час, з подальшим запуском інтерпретатора команд.

Крок 4. Підтвердження й розширення. Знайшовши підозрілий патерн, розширюємо розслідування діамантовою моделлю (розділ 12): від скомпрометованого вузла — до інфраструктури, інших уражених вузлів, часу початкової компрометації (оцінка dwell time).

Крок 5. Реагування й автоматизація. Передаємо підтверджену загрозу на реагування (розділ 16), а гіпотезу перетворюємо на правило виявлення (розділ 11), щоб надалі ловити цей патерн автоматично (13.2).

Кейс ілюструє силу проактивного, гіпотезо-керованого пошуку проти супротивника, який навмисно уникає автоматичного виявлення: полювання скорочує dwell time, виявляє приховане й перетворює знахідки на постійне автоматизоване виявлення.

Висновки до розділу

Цілеспрямована стала загроза (APT) — добре ресурсоване, наполегливе, адаптивне угруповання, що досягає конкретних цілей попри захист, уникаючи відомих сигнатур і зловживаючи легітимними інструментами. Проти APT реактивне виявлення недостатнє, бо такий супротивник діє приховано й може роками лишатися непоміченим, збільшуючи час присутності (dwell time), з яким зростають збитки. Відповіддю є полювання за загрозами — проактивний, ітеративний, керований гіпотезами пошук прихованого, що виходить із припущення компрометації. Гіпотези походять із розвідки загроз, аномалій і знання середовища; цінний продукт полювання — не лише знахідки, а й автоматизація успішних гіпотез у правила виявлення, що замикає контур із інженерією виявлення (13.2). Найхарактерніші об'єкти полювання — бокове переміщення й LotL, що зливаються з нормою й виявляються за аномаліями відносно моделі норми. Зрілість полювання зростає від реактивності до автоматизації, а його метрики оцінюють систематичність і покриття гіпотез, а не лише «улов», бо «порожнє» полювання теж цінне. Головний висновок: проти витончених невідомих загроз проактивне полювання є необхідним доповненням автоматичного виявлення, що найбільше скорочує dwell time саме там, де автоматика безсила.

Питання для самоперевірки

1. Якими трьома ознаками характеризують APT? Чим APT відрізняється від масового ЗПК?
2. Чому типовий периметровий захист і сигнатурне виявлення систематично пропускають APT?
3. Що таке час присутності (dwell time)? Чому його мінімізація є стратегічною метою?
4. Чому для скорочення dwell time проти невідомих загроз потрібен проактивний пошук?
5. Що таке полювання за загрозами? Чим воно відрізняється від реагування на сповіщення?

6. Поясніть припущення компрометації (assume breach) як основу полювання.
7. Чому автоматизація успішних гіпотез є цінним продуктом полювання? Як утворюється контур із інженерією виявлення?
8. З яких джерел походять гіпотези полювання? Наведіть приклад гіпотези.
9. Поясніть формулу сукупної повноти (13.2). Чому полювання дає найбільший приріст проти APT?
10. Що таке бокове переміщення й LotL? Чому вони важкі для автоматичного виявлення?
11. Як виявляють бокове переміщення? Яку роль відіграє модель норми?
12. Опишіть модель зрілості полювання. Чому «порожнє» полювання не є провалом?
13. Опишіть полювання за прихованим угрупованням (кейс підрозділу 13.7). Як оцінюють dwell time?
14. Дослідницьке: розробіть і перевірте набір гіпотез полювання за технікою бокового переміщення на реалістичній телеметрії, оцініть виявлені сліпі зони.
15. Дослідницьке: оцініть вплив розвиненості полювання на dwell time, порівнявши сценарії суто реактивного та проактивного захисту.

РОЗДІЛ 14

Протидія злоякісному коду у спеціалізованих середовищах

Методи й засоби попередніх розділів орієнтовані переважно на класичні обчислювальні системи (робочі станції, сервери загального призначення). Проте ЗПК дедалі частіше націлюється на *спеціалізовані* середовища — промислові системи керування, Інтернет речей, мобільні платформи, хмару, — кожне з яких має власну модель загроз, обмеження й вектори, через що прямий перенос «настільних» підходів часто неможливий. Для України це питання має критичне значення: атаки на промислову автоматику енергетики (Industroyer) і руйнівні кампанії проти критичної інфраструктури показали, що спеціалізовані середовища є пріоритетними цілями. Розділ розглядає спільні принципи протидії в спеціалізованих середовищах, захист промислових систем (ICS/SCADA), безпеку Інтернету речей, мобільних платформ і хмарних та контейнерних середовищ, і завершується кейсом захисту промислового об'єкта. Виклад спирається на моделювання загроз (розділ 4), виявлення (розділи 6–11) і розвідку загроз (розділ 12), адаптуючи їх до специфіки кожного середовища.

14.1 Спільні принципи й виклики

Попри відмінності, спеціалізовані середовища поділяють спільні виклики, що відрізняють їх від класичних систем (табл. 14.1): *обмежені ресурси* (неможливість встановити повноцінний агент захисту); *специфічні ОС і протоколи* (сигнатури й засоби для класичних систем незастосовні); *довгий життєвий цикл і складність оновлення* (пристрої працюють роками без патчів); *висока ціна збою* (у промисловості й медицині — фізична безпека людей); *масовість і слабка видимість* (мільярди пристроїв IoT поза моніторингом). Ці чинники зміщують акцент із виявлення на кінцевій точці (часто неможливого) на мережевий рівень, сегментацію й профілактику.

Таблиця 14.1 — Виклики протидії ЗПК у спеціалізованих середовищах

Виклик	Наслідок	Відповідь
Обмежені ресурси	немає агента	мережевий моніторинг
Специфічні протоколи	сигнатури незастосовні	спеціалізовані правила
Складність оновлення	незакриті вразливості	сегментація, компенсація
Ціна збою	пріоритет доступності	захист без втручання

Спільним стратегічним принципом стає **ешелонований захист без покладання на кінцеву точку**: оскільки сам пристрій часто незахищуваний, основним рубежем стають сегментація мережі (ізоляція спеціалізованого сегмента), контроль і моніторинг трафіку на межах сегментів, гартування конфігурації й сувора профілактика. Там, де не можна виявити ЗПК *на* пристрої, його виявляють *навколо* пристрою — за аномаліями мережевої взаємодії.

Ще один спільний принцип — *передбачуваність* спеціалізованих середовищ, яку захисник може обернути собі на користь. На відміну від універсальної робочої станції, де користувач запускає довільне ПЗ, спеціалізований пристрій зазвичай виконує вузький, сталий набір функцій: контролер циклічно опитує датчики й керує виконавчими механізмами, камера передає відеопотік на сервер, а не сканує сусідів. Ця вузькість означає, що нормальна поведінка добре окреслена, а отже будь-яке відхилення — нетипове з'єднання, нова команда, сплеск трафіку — підозріле з високою питомою вагою. Там, де на універсальній системі аномалія тоне в розмаїтті легітимної активності (проблема базової частоти, розділ 3), у спеціалізованому середовищі та сама аномалія різко виділяється. Тому **білий список дозволеної поведінки** (allow-listing) — як мережевої, так і процесної — тут значно дієвіший, ніж на настільних системах, і становить природний фундамент захисту.

14.2 Промислові системи керування (ICS/SCADA)

Промислові системи керування (ICS, зокрема SCADA) керують фізичними процесами — енергетикою, водопостачанням, виробництвом. Їх безпека [212, 120] має принципову відмінність від IT: пріоритет — *доступність і цілісність* процесу (а не конфіденційність даних), бо збій означає фізичні наслідки. Специфіка: застарілі системи без вбудованої безпеки, спеціалізовані протоколи, неперервність роботи (неможливість зупинити для оновлення), конвергенція IT/OT (поєднання офісних і технологічних мереж створює нові вектори).

ЗПК для ICS — вершина складності загроз. Stuxnet [140] уперше показав код, що фізично руйнував обладнання (центрифуги), маніпулюючи логікою контролерів і приховуючи це від операторів. В Україні Industroyer/Industroyer2 [76] — спеціалізований код для атак на енергосистему — знав промислові протоколи й безпосередньо керував підстанціями, спричинивши вимкнення електропостачання. Ці кейси засвідчили: ЗПК для ICS потребує глибокого знання конкретного процесу й обладнання, діє цілеспрямовано (APT, розділ 13) і має на меті фізичний вплив.

Протидія: сувора сегментація IT/OT (демілітаризовані зони, односпрямовані шлюзи), моніторинг промислових протоколів на аномалії (нетипові команди контролерам), гартування й контроль цілісності логіки контролерів, фізичні запобіжники, незалежні від керуючого коду. Оскільки встановити агент на контролер зазвичай неможливо, ключовим є пасивний мережевий моніторинг OT-сегмента. Українські дослідження систематизують захист АСК ТП критичної інфраструктури [15].

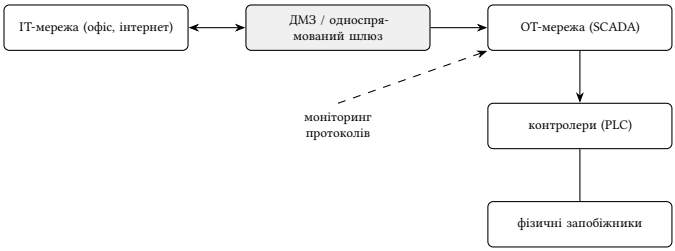


Рис. 14.2 — Сегментація IT/OT із демілітаризованою зоною, мережевим моніторингом і фізичними запобіжниками

Специфіку ризику в ICS формалізує врахування *фізичних* наслідків. На відміну від IT, де збиток вимірюють компрометацією даних, в OT очікуваний збиток зважують на тяжкість фізичного наслідку:

$$R_{\text{OT}} = \sum_{s} P(s) \cdot I_{\text{safety}}(s), \tag{14.1}$$

де $P(s)$ — імовірність сценарію s, а $I_{\text{safety}}(s)$ — його наслідок із урахуванням загрози людям і довкіллю. Формула (14.1) пояснює, чому в ICS навіть малоймовірні сценарії з катастрофічним I_{safety} домінують у ризику й виправдовують дорогі незалежні фізичні запобіжники: жодне програмне виявлення не замінює захисту, що діє, навіть коли керуючий код повністю скомпрометовано.

14.3 Інтернет речей (IoT)

Інтернет речей — мільярди під'єднаних пристроїв (камери, маршрутизатори, датчики) — став масовою ціллю через слабку безпеку: типові паролі, незакриті вразливості, відсутність оновлень, мінімальні ресурси для захисту [175]. Ботнет Mirai [54] продемонстрував масштаб загрози: заразивши сотні тисяч пристроїв простим перебором типових паролів, він здійснив одні з найпотужніших DDoS-атак в історії. Привабливість IoT для зловмисника: масовість (величезна поверхня атаки), слабкий захист, цілодобова доступність, слабка видимість (власники не моніторять).

Швидкість захоплення IoT-ботнетом описують епідемічними моделями (розділ 3). На ранній стадії, коли вразлива популяція ще велика, кількість заражених пристроїв зростає майже експоненційно:

$$\frac{dN}{dt} = \beta N \left(1 - \frac{N}{N_{\max}}\right), \tag{14.2}$$

де N — заражені пристрої, $N_{\max}$ — розмір уразливої популяції, β — темп поширення (залежить від швидкості сканування й частки пристроїв із типовими паролями). Логістична форма (14.2) пояснює вибуховий характер Mirai: за великого $N_{\max}$ (мільйони пристроїв із типовими паролями) і високого β ботнет досягає масштабу, достатнього для рекордних DDoS, за години. Вона ж підказує важелі протидії: зниження $N_{\max}$ (зміна типових паролів, оновлення) і β (обмеження сканування сегментацією) збивають епідемію ще на старті.

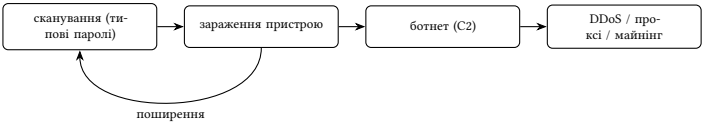


Рис. 14.1 — Життєвий цикл IoT-ботнету на прикладі моделі Mirai

Протидія ускладнена обмеженнями пристроїв: повноцінний захист на пристрої неможливий. Тому акцент — на мережевому рівні (виявлення аномального трафіку, сегментація IoT в окремий сегмент, обмеження зовнішнього доступу), профілактиці (зміна типових паролів, оновлення, вимкнення зайвих служб) і безпеці за задумом (secure-by-design — вимоги до виробників). Виявлення зараження спирається на аномалії поведінки (сплеск вихідного трафіку, звернення до командних серверів), а не на аналіз коду на пристрої. Вітчизняні роботи пропонують моделі виявлення шкідливої активності в IoT [24].

14.4 Мобільні платформи

Мобільні ОС (Android, iOS) мають принципово іншу модель безпеки, ніж настільні: **пісочниця застосунків** (ізоляція кожного застосунку), дозвільна модель (контроль доступу до ресурсів), централізовані магазини з перевіркою, оновлення платформи [109]. Це ускладнює класичний ЗПК, але не унеможливлює: шкідливі застосунки обходять перевірку магазинів, зловживають дозволами, застосовують соціальну інженерію, експлуатують вразливості. Дослідження безпеки Android [96] показали поширеність витоку даних і зловживання дозволами навіть у легітимних застосунках.

Специфіка мобільних загроз: шпигунський код (стеження, перехоплення повідомлень — особливо небезпечний для цільових осіб), банківські трояни (накладні вікна для крадіжки облікових даних), зловживання дозволами. Протидія: модель безпеки платформи (пісочниця, дозволи), перевірка магазинів, мобільні засоби захисту, контроль у корпоративному середовищі (MDM), аналіз застосунків (статичний і динамічний, розділи 6–7, адаптовані до байт-коду й специфіки платформи). Ключова лінія — дозвільна модель і пісочниця, що обмежують вплив навіть зловмисного застосунку. Порівняння моделей захисту чотирьох середовищ наведено в табл. 14.2.

Таблиця 14.2 — Спеціалізовані середовища: модель захисту

Середовище	Основний рубіж	Пріоритет
ICS/SCADA	сегментація, монітор OT	доступність, безпека
IoT	мережевий рівень, профілактика	масштаб, видимість
Мобільні	пісочниця, дозволи	ізоляція застосунків
Хмара/контейнери	цілісність образів, незмінність	динаміка, масштаб

14.5 Хмарні та контейнерні середовища

Хмарні [152] й контейнерні [203] середовища зміщують модель загроз: спільна відповідальність (безпека розподілена між провайдером і клієнтом), ефемерність (контейнери живуть короткий час), масштаб і динамічність, нові вектори (компрометація образів, оркестрації, викрадення обчислень для майнінгу). ЗПК у хмарі: заражені образи контейнерів (ланцюг постачання, розділ 15), код для криптомайнінгу на викрадених ресурсах, компрометація керуючої площини.

Протидія адаптує знайомі принципи до динаміки хмари: контроль цілісності й сканування образів *до* розгортання (а не лише під час виконання), найменші привілеї для робочих навантажень, сегментація й мікросегмен-

тація, моніторинг поведінки контейнерів (аномальні процеси, мережа), незмінна інфраструктура (immutable — заміна замість зміни, що ускладнює персистентність). Ефемерність контейнерів — палиця з двома кінцями: з одного боку, ускладнює персистентність ЗПК; з іншого, ускладнює розслідування (контейнер зникає разом зі слідами), що вимагає завчасного збору телеметрії.

Модель *спільної відповідальності* заслуговує окремого наголосу, бо її нерозуміння — типове джерело компрометацій. Провайдер відповідає за безпеку *хмари* (фізичну інфраструктуру, гіпервізор, керуючі служби), клієнт — за безпеку *у хмарі* (конфігурацію, дані, застосунки, права доступу). Значна частина хмарних інцидентів спричинена не проломом захисту провайдера, а помилками клієнта: відкритими сховищами, надмірними правами, скомпрометованими ключами доступу, невиправленими застосунками. Для ЗПК це означає нові вектори, специфічні для хмари: викрадення хмарних облікових даних відкриває доступ до керуючої площини, звідки можна розгортати власні навантаження (майнінг, проксі) коштом жертви. Тому захист у хмарі зміщує акцент із «антивірусу» на керування ідентичністю, правами й конфігурацією, а виявлення — на аномалії в журналах керуючої площини, а не лише в самих робочих навантаженнях.

14.6 Кейс: захист промислового об'єкта критичної інфраструктури

Розглянемо побудову захисту промислового об'єкта від цілеспрямованого ЗПК на кшталт Industroyer (захисна задача, за принципами IEC 62443 [120]).

Крок 1. Модель загроз. Визначаємо активи (контролери, інженерні станції, керуючий процес), ймовірного супротивника (APT державного рівня, розділ 13), наслідки (фізичний збій, загроза людям) та вектори (конвергенція IT/OT, ланцюг постачання, знімні носії).

Крок 2. Сегментація. Розділяємо IT- та OT-мережі демілітаризованою зоною й суворо контрольованими (ідеально — односпрямованими) шлюзами, щоб компрометація офісної мережі не давала прямого доступу до OT.

Крок 3. Моніторинг OT. Розгортаємо пасивний моніторинг промислових протоколів, що виявляє аномальні команди контролерам (нетипові для нормального процесу) — ключовий рубіж там, де агент на контролері неможливий.

Крок 4. Гартування й цілісність. Гартуємо конфігурацію інженерних станцій, контролюємо цілісність логіки контролерів, обмежуємо знімні носії й віддалений доступ, застосовуємо найменші привілеї.

Крок 5. Фізичні запобіжники й реагування. Передбачаємо незалежні від керуючого коду фізичні запобіжники (щоб код не міг спричинити катастрофу) і план реагування (розділ 16) з урахуванням неперервності процесу.

Кейс ілюструє адаптацію загальних принципів до ICS: оскільки виявлення на контролері неможливе, захист будують навколо нього — сегментацією, мережевим моніторингом, гартуванням і фізичними запобіжниками, з пріоритетом доступності й безпеки процесу.

Висновки до розділу

Спеціалізовані середовища — ICS/SCADA, IoT, мобільні платформи, хмара — мають власні моделі загроз і обмеження, через що прямий перенос «настільних» підходів часто неможливий (табл. 14.1). Спільний виклик — неможливість повноцінного захисту на самому пристрої — зміщує акцент на мережевий рівень, сегментацію й профілактику: там, де не можна виявити ЗПК на пристрої, його виявляють навколо нього. У ICS пріоритет доступності й фізичні наслідки роблять код на кшталт Stuxnet та Industroyer вершиною складності загроз; протидія — сувора сегментація IT/OT, моніторинг промислових протоколів, гартування й фізичні запобіжники. IoT через масовість і слабкий захист став базою ботнетів (Mirai); протидія — мережевий рівень, профілактика й безпека за задумом. Мобільні платформи мають сильну вбудовану модель безпеки (пісочниця, дозволи), що обмежує, але не усуває загрози. Хмарні й контейнерні середовища вимагають адаптації принципів до ефемерності й динаміки з акцентом на цілісність образів і незмінну інфраструктуру. Для України захист критичної інфраструктури є питанням національної безпеки. Головний висновок: протидія в спеціалізованих середовищах будується не на кінцевій точці, а навколо неї — сегментацією, мережевим моніторингом аномалій, профілактикою й архітектурними запобіжниками, адаптованими до специфіки кожного середовища.

Питання для самоперевірки

1. Які спільні виклики відрізняють спеціалізовані середовища від класичних систем (табл. 14.1)?
2. Поясніть принцип ешелонованого захисту без покладання на кінцеву точку. Чому він стає спільним для спеціалізованих середовищ?
3. Чим безпека ICS/SCADA принципово відрізняється від IT? Чому пріоритетом є доступність і цілісність?
4. Опишіть кейси Stuxnet та Industroyer. Чому ЗПК для ICS є вершиною складності загроз?
5. Які заходи протидії застосовують у ICS? Чому ключовим є пасивний мережевий моніторинг OT?
6. Чому IoT став масовою ціллю? Опишіть життєвий цикл ботнету Mirai (рис. 14.1).

7. Як протидіють загрозам IoT за умов обмежених ресурсів пристроїв?
8. Чим модель безпеки мобільних платформ відрізняється від настільних? Як пісочниця й дозволи обмежують загрози?
9. Які специфічні мобільні загрози існують? Які заходи протидії застосовують?
10. Як хмарні й контейнерні середовища змінюють модель загроз? Що таке незмінна інфраструктура та чому вона ускладнює персистентність?
11. Чому ефемерність контейнерів є палицею з двома кінцями для захисту?
12. Опишіть побудову захисту промислового об'єкта (кейс підрозділу 14.6). Чому захист будують навколо контролера, а не на ньому?
13. Чому захист критичної інфраструктури є питанням національної безпеки України?
14. Дослідницьке: розробіть модель загроз і схему сегментації для обраного типу об'єкта критичної інфраструктури за IEC 62443 та оцініть залишковий ризик.
15. Дослідницьке: порівняйте ефективність мережевого виявлення аномалій для IoT-ботнету за різних наборів ознак трафіку на реалістичних даних.

РОЗДІЛ 15

Атаки на ланцюг постачання програмного забезпечення

Класичні загрози (розділи 1, 5) приходять ззовні, тому захист будують на межі довіри: недовірене блокують, довіреному дозволяють. **Атака на ланцюг постачання** підриває саму цю межу: ЗПК проникає *через довірений канал* — легітимне оновлення, підписаний інсталятор, популярну бібліотеку, — яким захист довіряє за визначенням. Це робить такі атаки особливо небезпечними: вони обходять периметровий захист, уражають багато жертв одночасно (усіх користувачів скомпрометованого компонента) й зловживають найціннішим ресурсом безпеки — довірою. Для України тема має особливу вагу: NotPetya, одна з найруйнівніших кібератак в історії, поширилася саме через скомпрометоване оновлення широко вживаного бухгалтерського ПЗ. Розділ розглядає поняття й анатомію атак на ланцюг постачання, їх типологію та кейси, механізми довіри й цілісності (підпис, походження, SLSA, SBOM, відтворювані збірки) та стратегії протидії, і завершується кейсом захисту конвеєра розробки. Тема поєднує моделювання загроз (розділ 4), розвідку (розділ 12) і реагування (розділ 16).

15.1 Поняття й анатомія атаки

Ланцюг постачання ПЗ — сукупність усіх компонентів, людей, процесів та інструментів, що беруть участь у створенні й доставці програмного продукту до користувача: вихідний код, сторонні бібліотеки, інструменти збірки, конвеєри CI/CD, репозиторії, механізми оновлення. Атака на ланцюг постачання [98] компрометує будь-яку ланку цього ланцюга так, щоб зловмисний код потрапив у кінцевий продукт і був доставлений жертвам через довірений канал.

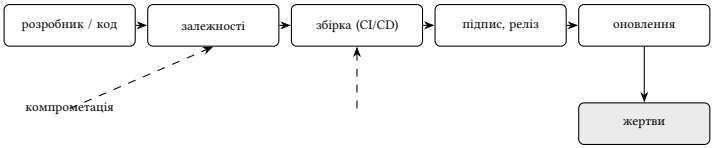


Рис. 15.1 — Ланцюг постачання ПЗ і точки можливої компрометації

Анатомія атаки має спільну логіку: зловмисник компрометує точку з високим *коефіцієнтом множення* (один скомпрометований компонент уражає тисячі жертв) і високою *довірою* (жертви не перевіряють довірений канал). Саме поєднання множення й довіри робить ланцюг постачання стра-

тегічно привабливою ціллю, особливо для APT (розділ 13): замість атакувати кожну жертву окремо, достатньо скомпрометувати спільного постачальника.

Коефіцієнт множення зручно виразити кількісно. Якщо скомпрометований компонент використовують U прямих споживачів, а кожен, своєю чергою, постачає продукт далі в середньому k наступним, то за d рівнів поширення потенційна кількість уражених

$$V \approx U \sum_{j=0}^{d} k^{j} = U \, \frac{k^{d+1} - 1}{k - 1} \quad (k > 1), \tag{15.1}$$

зростає геометрично з глибиною ланцюга. Формула (15.1) пояснює, чому компрометація популярної базової бібліотеки чи інструмента з великим U і транзитивним поширенням ($k > 1$) дає лавиноподібний «радіус ураження», непорівнянний із прямою атакою на окрему жертву, — і чому саме вузли з найбільшим U є пріоритетними як для зловмисника, так і для захисту.

15.2 Типологія та кейси

Атаки класифікують за ураженою ланкою (табл. 15.1): компрометація *оновлень* (підміна легітимного оновлення — SolarWinds [83], CCleaner [56], NotPetya); компрометація *залежностей* (зловмисний код у сторонніх бібліотеках відкритих репозиторіїв [171] — typosquatting, захоплення покинутих пакетів, впровадження в популярні); компрометація *інструментів збірки* й CI/CD (впровадження коду під час складання); компрометація *підпису* (викрадення ключів для підписання зловмисного коду як легітимного).

Таблиця 15.1 — Типологія атак на ланцюг постачання

Уражена ланка	Механізм	Приклад
Оновлення	підміна релізу	SolarWinds, NotPetya
Залежності	зловмисний пакет	typosquatting
Збірка (CI/CD)	код під час складання	компрометація конвеєра
Підпис	викрадені ключі	підробка довіри

Кейс SolarWinds [83] — взірець витонченості: APT впровадило бекдор у легітимне оновлення широко вживаного ПЗ мережевого керування, яке потрапило до тисяч організацій, зокрема урядових, через механізм оновлення з коректним підписом. NotPetya [1] — взірець руйнівності: через оновлення бухгалтерського ПЗ код поширився лавиною й завдав мільярдних

збитків світовій економіці. Ці кейси засвідчили: довірений підписаний канал не гарантує безпеки, якщо скомпрометоване саме джерело довіри.

Окремо варто виділити атаки на *відкриті репозиторії залежностей* [171], бо вони дешевші й масовіші за компрометацію великого вендора. *Typosquatting* — публікація зловмисного пакета з іменем, схожим на популярний (різниця в одну літеру), у розрахунку на помилку розробника при встановленні. *Захоплення покинутих пакетів* — перебирання контролю над занедбаним, але все ще вживаним пакетом через покинутий обліковий запис супровідника. *Впровадження в легітимний пакет* — компрометація облікового запису супровідника популярної бібліотеки й випуск отруєної версії. Спільна риса — зловживання автоматизмом сучасної розробки: залежності підтягуються транзитивно, часто без ревізії, тож один отруєний пакет глибоко в дереві залежностей тихо потрапляє в тисячі продуктів. Це робить *гігієну залежностей* — фіксацію версій, перевірку походження, мінімізацію дерева — базовою, а не факультативною практикою.

15.3 Довіра, цілісність та походження

Основою протидії є забезпечення **цілісності** й **походження** (provenance) артефактів — гарантій того, що компонент не змінено й що він походить із довіреного процесу. Цифровий підпис гарантує цілісність і автентичність *за умови* захищеності ключа; його недостатньо, якщо скомпрометовано збірку (підписується вже заражений артефакт) чи ключ. Тому сучасні підходи забезпечують цілісність *усього* процесу: in-toto [218] фіксує й перевіряє кожен крок конвеєра; SLSA [174] визначає рівні зрілості гарантій цілісності складання; Sigstore [167] спрощує підписання й прозорий облік артефактів; відтворювані збірки [139] дозволяють незалежно перевірити, що бінарний артефакт відповідає вихідному коду.

Довіру до ланцюга зручно оцінювати як добуток довір до окремих ланок. Якщо ланцюг складається з n ланок, кожна з надійністю (ймовірністю бути нескомпрометованою) t_i, то цілісність кінцевого артефакту за незалежних ланок

$$T_{\text{ланцюг}} = \prod_{i=1}^{n} t_i, \tag{15.2}$$

не перевищує надійності найслабшої ланки й спадає зі зростанням довжини ланцюга. Формула (15.2) формалізує інтуїцію «ланцюг міцний настільки, наскільки міцна його найслабша ланка» й пояснює, чому подовження ланцюга (більше залежностей, посередників, автоматичних кроків) системно знижує довіру, якщо кожну нову ланку не забезпечено гарантіями цілісності.

Гарантії цілісності складання прийнято впорядковувати за рівнями зрілості (табл. 15.2), що дозволяє поступово підвищувати надійність t_i кожної ланки з (15.2).

Таблиця 15.2 — Рівні гарантій цілісності складання (за моделлю SLSA)

Рівень	Гарантія	Механізм
Базовий	документована збірка	опис процесу
Середній	походження артефакту	підписана provenance
Високий	ізольована, керована збірка	захищений конвеєр
Найвищий	незалежна перевірюваність	відтворювані збірки

15.4 Прозорість: SBOM та інвентаризація

Неможливо захистити те, про існування чого не відомо. Сучасне ПЗ містить десятки й сотні сторонніх компонентів, тому першою умовою захисту є **прозорість** складу. **SBOM** (Software Bill of Materials [166]) — машиночитаний перелік усіх компонентів продукту та їх версій — дозволяє швидко відповісти на критичне питання: «чи містить наш продукт уражений компонент?». Коли виявляють вразливість чи компрометацію популярної бібліотеки, організації зі SBOM миттєво визначають, що саме зачеплено, тоді як інші витрачають дні на ручний аудит.

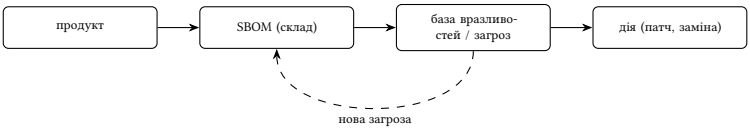


Рис. 15.2 — SBOM як основа швидкого реагування на вразливості й компрометації компонентів

SBOM — не самоціль, а основа процесів: безперервного зіставлення складу з базами вразливостей і розвідкою загроз (розділ 12), керування оновленнями, оцінювання ризику залежностей. Він перетворює реагування на компрометацію компонента з хаотичного пошуку на кероване, швидке зіставлення, прямо скорочуючи час реагування (10.1).

Повнота SBOM вимагає врахування *транзитивних* залежностей, а не лише прямих. Сучасний продукт прямо залучає десятки бібліотек, але кожна з них тягне власні, ті — свої, тож реальне дерево залежностей сягає сотень вузлів, більшість із яких розробник ніколи свідомо не обирав. Саме в цих глибоких, «невидимих» вузлах і ховаються отруєні пакети (15.1). Тому цін-

ний SBOM рекурсивно розгортає все дерево й фіксує точні версії (а не діапазони), бо лише так зіставлення з розвідкою загроз дає однозначну відповідь, уражений продукт чи ні. Без транзитивної повноти SBOM створює оманливе відчуття контролю, покриваючи лише верхівку дерева.

15.5 Стратегії протидії

Протидія атакам на ланцюг постачання багаторівнева. *Зменшення й контроль залежностей*: мінімізація сторонніх компонентів, перевірка їх репутації й походження, фіксація версій, використання довірених дзеркал. *Захист конвеєра збірки*: ізоляція, найменші привілеї, контроль цілісності (in-toto, SLSA), захист ключів підпису (апаратні модулі, короткоживучі ключі). *Перевірка артефактів*: підпис і його верифікація, відтворювані збірки, прозорий облік. *Прозорість*: SBOM, інвентаризація, моніторинг складу. *Виявлення*: аномалії в поведінці оновлень і компонентів (навіть довірений компонент, що раптом звертається до нових серверів, підозрілий — тут знову працюють поведінкові методи розділів 7, 10).

Фундаментальний зсув — від *сліпої* довіри до *перевірюваної*: замість «довіряти, бо підписано» — «довіряти, бо перевірено цілісність усього процесу». Це втілення принципу нульової довіри (zero trust) у ланцюгу постачання: жодна ланка не є довіреною за замовчуванням, кожна підлягає перевірці. Повністю усунути ризик неможливо (сучасне ПЗ не будують без сторонніх компонентів), тому мета — зробити компрометацію важчою, а наслідки — виявлюваними й обмеженими. Українські дослідження систематизують загрози ланцюга постачання й методи забезпечення цілісності [11].

Важливо усвідомлювати, що захист ланцюга постачання стикається з принциповим обмеженням виявлення: скомпрометований компонент *за визначенням* проходить через довірений канал і часто має коректний підпис, тому сигнатурні й репутаційні методи (розділ 6) безсилі — вони саме й призначені довіряти підписаному. Єдине, що лишається виявленню, — поведінкові аномалії *після* розгортання: довірений компонент, що раптом поводиться незвично (нові мережеві з'єднання, нетипові дії), викриває себе не походженням, а поведінкою. Це повертає до наскрізного принципу посібника: там, де статична довіра підведена, останнім рубежем лишається поведінковий моніторинг (розділи 7, 10). Проте й він обмежений: витончена атака на кшталт SolarWinds навмисно діяла повільно й обережно, імітуючи легітимну активність, аби не викликати поведінкових сигналів, — що знову підкреслює незамінність *запобіжної* цілісності процесу над реактивним виявленням.

15.6 Кейс: захист конвеєра розробки

Розглянемо побудову захисту конвеєра розробки ПЗ від компрометації ланцюга постачання (захисна задача, за моделлю SLSA [174]).

Крок 1. Інвентаризація й SBOM. Формуємо повний перелік компонентів і залежностей (SBOM [166]), встановлюємо безперервне зіставлення з базами вразливостей і розвідкою загроз.

Крок 2. Контроль залежностей. Мінімізуємо й фіксуємо залежності, перевіряємо їх походження й репутацію, використовуємо довірені джерела, автоматично скануємо на відомі загрози й typosquatting.

Крок 3. Захист збірки. Ізолюємо конвеєр CI/CD, застосовуємо найменші привілеї, фіксуємо й перевіряємо кожен крок (in-toto [218]), захищаємо ключі підпису апаратно й робимо їх короткоживучими.

Крок 4. Перевірка артефактів. Підписуємо артефакти, забезпечуємо верифікацію підпису споживачами, за можливості впроваджуємо відтворювані збірки [139] для незалежної перевірки відповідності коду.

Крок 5. Моніторинг і реагування. Моніторимо поведінку власних оновлень і компонентів на аномалії, готуємо план швидкого реагування на компрометацію компонента (завдяки SBOM — миттєве визначення ураженого) з повідомленням споживачів.

Кейс демонструє зсув від сліпої довіри до перевірюваної цілісності всього процесу: захист ланцюга постачання — це не окремий засіб, а дисципліна цілісності, прозорості й перевірки на кожній ланці, доповнена поведінковим моніторингом і готовністю до реагування.

Висновки до розділу

Атака на ланцюг постачання підриває саму межу довіри, проводячи ЗПК через довірений канал — оновлення, підписаний інсталятор, популярну бібліотеку, — чим обходить периметровий захист і уражає багато жертв одночасно. Її сила — у поєднанні множення (один компонент — тисячі жертв) і довіри, що робить ланцюг стратегічною ціллю APT. Атаки класифікують за ураженою ланкою — оновлення, залежності, збірка, підпис (табл. 15.1); кейси SolarWinds і NotPetya засвідчили, що підписаний канал не гарантує безпеки, якщо скомпрометоване джерело довіри. Протидія забезпечує цілісність і походження *усього* процесу (in-toto, SLSA, Sigstore, відтворювані збірки), бо довіра ланцюга не перевищує надійності найслабшої ланки (15.2). Прозорість складу (SBOM) є основою швидкого реагування на компрометацію компонента. Стратегія протидії — зсув від сліпої довіри до перевірюваної (нульова довіра в ланцюгу постачання): мінімізація й контроль залежностей, захист конвеєра й ключів, перевірка артефактів, прозорість і поведінковий моніторинг. Для України NotPetya є гірким прикладом руйнівного потенціалу таких атак. Головний висновок: повністю усунути

ризик ланцюга постачання неможливо, але його знижують дисципліною цілісності, прозорості й перевірки на кожній ланці, роблячи компрометацію важчою, а наслідки — виявлюваними й обмеженими.

Питання для самоперевірки

1. Чому атака на ланцюг постачання особливо небезпечна? Як вона підриває межу довіри?
2. Що таке ланцюг постачання ПЗ? Назвіть його ланки й точки можливої компрометації (рис. 15.1).
3. Поясніть роль множення й довіри в анатомії атаки. Чому ланцюг постачання привабливий для APT?
4. Опишіть типологію атак за ураженою ланкою (табл. 15.1).
5. Опишіть кейси SolarWinds і NotPetya. Який спільний урок вони дають щодо підписаного каналу?
6. Що таке цілісність і походження артефактів? Чому цифрового підпису недостатньо?
7. Які підходи забезпечують цілісність усього процесу (in-toto, SLSA, Sigstore, відтворювані збірки)?
8. Поясніть формулу довіри ланцюга (15.2). Чому подовження ланцюга знижує довіру?
9. Що таке SBOM? Яку критичну задачу він розв'язує при виявленні компрометації компонента?
10. Чому прозорість складу є першою умовою захисту? Як SBOM скорочує час реагування?
11. Назвіть рівні стратегії протидії атакам на ланцюг постачання.
12. Поясніть зсув від сліпої довіри до перевірюваної. Як принцип нульової довіри застосовується до ланцюга постачання?
13. Опишіть побудову захисту конвеєра розробки (кейс підрозділу 15.6). Чому захист — це дисципліна, а не окремий засіб?
14. Дослідницьке: оцініть ризик залежностей обраного проєкту за SBOM, побудуйте граф залежностей і визначте найслабші ланки (15.2).
15. Дослідницьке: дослідіть ефективність відтворюваних збірок для виявлення впровадження коду на етапі складання на модельному конвеєрі.

РОЗДІЛ 16

Реагування на інциденти та цифрова форензика шкідливого коду

Виявлення загрози (розділи 6–13) — не кінець, а початок роботи: за ним має настати **реагування на інцидент** — організований процес стримування, усунення й відновлення, а також **цифрова форензика** — дослідження артефактів для встановлення фактів. Зрілість захисту визначається не лише здатністю виявити, а й здатністю швидко й правильно відреагувати: невміле реагування може завдати більшої шкоди, ніж сам інцидент (знищити докази, не усунути присутність, спричинити повторне зараження). Реагування скорочує час присутності й збитки (розділ 13), а форензика встановлює повну картину, живить розвідку загроз (розділ 12) і, за потреби, доказову базу. Розділ розглядає життєвий цикл реагування, його фази, принципи цифрової форензики (докази, їх цілісність, порядок летючості), аналіз інциденту ЗПК, антифорензику й протидію їй, а також нормативні й організаційні аспекти в Україні, і завершується наскрізним кейсом реагування на зараження. Виклад спирається на телеметрію (розділ 10) й аналіз зразків (розділи 6–7).

16.1 Життєвий цикл реагування на інцидент

Реагування — не хаотична реакція, а структурований процес. Класична модель [81, 162] виділяє фази: *підготовка* (процеси, інструменти, навички, телеметрія — *до* інциденту); *виявлення й аналіз* (підтвердження інциденту, оцінка масштабу); *стримування* (зупинення поширення); *усунення* (видалення присутності загрози); *відновлення* (повернення до нормальної роботи); *уроки* (аналіз після інциденту, вдосконалення). Ці фази утворюють цикл, у якому уроки живлять підготовку до наступних інцидентів.

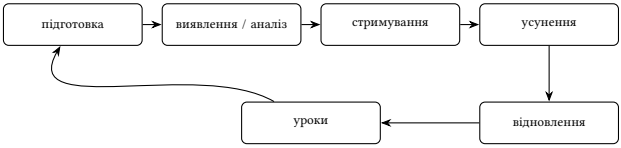


Рис. 16.1 — Життєвий цикл реагування на інцидент як безперервний цикл

Критична, часто недооцінена фаза — *підготовка*: саме вона визначає успіх решти. Без завчасно зібраної телеметрії (розділ 10), налагоджених процесів, визначених ролей, інструментів і навичок реагування перетворю-

ється на імпровізацію під тиском, коли ціна помилки найвища. Тому зрілі організації інвестують у підготовку й регулярні навчання (кіберполігони, настільні вправи), а не лише в засоби виявлення. Основну мету кожної фази узагальнено в табл. 16.1.

Таблиця 16.1 — Фази реагування та їх основна мета

Фаза	Основна мета	Ключовий ризик
Підготовка	готовність процесів і даних	недооцінка
Виявлення й аналіз	підтвердження, масштаб	хибна оцінка
Стримування	зупинити поширення	втрата доказів
Усунення	видалити присутність	неповнота
Відновлення	повернути роботу	повторне зараження
Уроки	вдосконалення	формальність

16.2 Стримування, усунення, відновлення

Стримування — першочергове зупинення поширення й шкоди: ізоляція уражених вузлів (розділ 10), блокування зв'язку з командними серверами, відключення скомпрометованих облікових записів. Стримування балансує швидкість і збереження доказів: різке вимкнення зупиняє загрозу, але знищує летючі докази (пам'ять), тому рішення залежить від пріоритетів (негайна зупинка шкоди проти повноти розслідування).

Усунення видаляє присутність загрози: шкідливі файли, механізми персистентності (розділ 5), створені облікові записи, бекдори. Ключова вимога — *повнота*: якщо лишити хоч один механізм персистентності, загроза повернеться. Тому усунення спирається на повне розуміння інциденту (що саме зробив ЗПК), отримане аналізом. **Відновлення** [204] повертає системи до нормальної роботи з контролем, що загрозу усунуто й уразливість, яка дала доступ, закрито (інакше — повторне зараження). Відновлення з резервних копій вимагає впевненості, що копія не містить тієї самої закладки.

16.3 Цифрова форензика: докази та їх цілісність

Цифрова форензика досліджує цифрові артефакти для встановлення фактів інциденту: що сталося, коли, як, які системи й дані зачеплено, хто (за можливості). Її принципи [73, 72] забезпечують, щоб висновки були обґрунтованими, відтворюваними й (за потреби) юридично прийнятними. Міжнародний стандарт ISO/IEC 27037 [122] визначає настанови з ідентифікації, збору, отримання й збереження цифрових доказів.

Ключові принципи: *цілісність доказів* (докази не повинні змінюватися при зборі — працюють із копіями, фіксують геші); *ланцюг збереження* (chain

of custody — документований облік, хто, коли й як поводився з доказом); *порядок летючості* (order of volatility [68] — спершу збирають найлетючіші дані). Порядок летючості критичний: регістри й кеш, оперативна пам'ять, мережеві з'єднання зникають за секунди-хвилини, тоді як диск і резервні копії зберігаються довго.

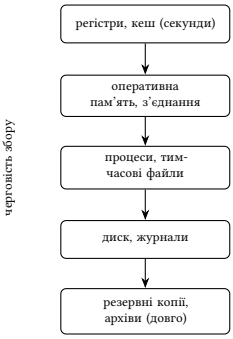


Рис. 16.2 — Порядок летючості: найлетючіші дані збирають першими

Для ЗПК особливо цінний аналіз пам'яті (розділ 7): він розкриває розгорнутий код, ключі, ін'єкції, безфайлові загрози, яких немає на диску. Тому за можливості пам'ять знімають *до* вимкнення вузла — що входить у суперечність зі швидким стримуванням і вимагає зваженого рішення.

Принцип цілісності доказів заслуговує практичного пояснення. Форензичний аналіз ніколи не ведуть на оригіналі: з носія знімають побітову копію (образ), обчислюють її криптографічний геш і працюють із копією, періодично звіряючи геш, щоб довести незмінність. Будь-яка дія на оригіналі — навіть просте відкриття файлу — змінює часові мітки й може знищити докази, тому використовують апаратні чи програмні блокувальники запису. Ланцюг збереження документує кожен перехід доказу з рук у руки з часом, особою й метою, аби в суді чи при службовому розслідуванні можна було довести, що доказ не підмінено й не спотворено. Ці формальності не бюрократія: саме вони відрізняють доказ, на який можна спертися в юридично значущому рішенні (розділ 2), від припущення, яке опонент легко оскаржить. Порушення цілісності на будь-якому етапі знецінює доказ незворотно — відновити довіру до нього вже неможливо.

Варто також розрізняти форензику *живої* та *мертвої* системи. Аналіз живої системи (до вимкнення) дає летючі дані — пам'ять, активні з'єднання, запущені процеси, — але сам факт роботи аналітика на живій системі вносить зміни, які треба документувати. Аналіз мертвої системи (з образу вимкненого носія) гарантує незмінність, але втрачає летюче. Для ЗПК, осо-

бливо безфайлового (розділ 11), вибір часто на користь живого аналізу з ретельним документуванням, бо втрата пам'яті означає втрату головного доказу. Це рішення — частина завчасної підготовки, а не імпровізації під час інциденту.

16.4 Аналіз інциденту ЗПК

Форензичний аналіз інциденту ЗПК [22] відновлює повну картину: вектор початкового доступу (як потрапив), механізми персистентності (як закріпився), дії (що робив — бокове переміщення, збір і вивантаження даних, шкода), індикатори (для виявлення решти уражень і розвідки), масштаб (які системи й дані зачеплено). Аналіз поєднує дослідження зразка (розділи 6–7), артефактів системи (журнали, реєстр, файлова система, пам'ять) і телеметрії (розділ 10) в єдину часову лінію подій (timeline). Основні джерела форензичних артефактів і факти, які вони розкривають, наведено в табл. 16.2.

Таблиця 16.2 — Джерела форензичних артефактів при розслідуванні ЗПК

Джерело	Що розкриває
Оперативна пам'ять	розгорнутий код, ключі, ін'єкції, безфайлові загрози
Файлова система	зразки, часові мітки, видалені файли
Реєстр / автозапуск	механізми персистентності
Журнали подій	дії, автентифікації, бокове переміщення
Мережеві записи	зв'язок із C2, вивантаження даних

Повноту аналізу зручно характеризувати відновленою часткою ланцюга ураження. Якщо ланцюг має K етапів (розділ 4), а аналіз достовірно відновив k із них, то повнота розслідування

$$\mathrm{Compl} = \frac{k}{K} \in [0;1], \tag{16.1}$$

причому неповне розслідування (пропущений етап) залишає ризик: невиявлений механізм персистентності чи уражений вузол спричинить повторний інцидент. Формула (16.1) підкреслює, чому мета форензики — відновити *весь* ланцюг, а не лише очевидний фрагмент: саме прогалини, а не знайдене, визначають залишковий ризик.

16.5 Антифорензика та протидія їй

Зловмисники застосовують **антифорензику** [106] — приховування й знищення слідів: видалення журналів, затирання файлів, підробку часових міток, безфайлове виконання (немає файлу для аналізу), шифрування. Мета

— ускладнити або унеможливити розслідування, збільшивши час присутності й уникнувши атрибуції.

Протидія антифорензиці спирається на кілька принципів. *Надмірність джерел*: сліди лишаються в багатьох місцях (централізована телеметрія поза ураженим вузлом, мережеві записи, резервні копії), тож знищення одного джерела не знищує всіх. *Завчасний збір*: централізований збір телеметрії (розділ 10) *до* інциденту зберігає докази поза досяжністю зловмисника на ураженому вузлі. *Аналіз пам'яті*: безфайлові загрози, невидимі на диску, проявляються в пам'яті. *Виявлення самої антифорензики*: ознаки затирання (прогалини в журналах, аномальні часові мітки) самі є індикаторами. Надмірність джерел зручно обґрунтувати кількісно. Якщо один і той самий факт лишає слід у m незалежних джерелах, а зловмисник знищує кожне з імовірністю q, то ймовірність, що факт *не* вціліє в жодному,

$$P_{\text{втрати}} = q^{m}, \tag{16.2}$$

спадає експоненційно зі зростанням кількості незалежних джерел. Формула (16.2) формалізує, чому надмірність — найдієвіша протидія антифорензиці: навіть за високого q (зловмисник майстерно затирає сліди на вузлі) кілька незалежних джерел, зокрема централізована телеметрія поза його досяжністю, роблять повну втрату доказів малоймовірною. Ключовий висновок: централізована, захищена від втручання телеметрія — найкраща протидія антифорензиці, бо переносить докази туди, куди зловмисник на ураженому вузлі не дістане.

16.6 Нормативні й організаційні аспекти в Україні

Реагування на кіберінциденти в Україні регулюється законодавством (розділ 2): обов'язок повідомляти про інциденти, взаємодія з CERT-UA (розділ 12), вимоги до критичної інфраструктури. Стандарти ISO/IEC 27035 (управління інцидентами) та ISO/IEC 27037 (цифрові докази) задають міжнародно узгоджені практики. Організаційно реагування потребує визначеної команди (CSIRT/SOC), ролей, планів, каналів ескалації й взаємодії з правоохоронними органами, коли інцидент має ознаки злочину (ст. 361-1 КК України). Вітчизняні дослідження систематизують організацію реагування відповідно до вимог законодавства [33]. Юридична прийнятність доказів вимагає суворого дотримання цілісності й ланцюга збереження з самого початку, бо порушення на етапі збору знецінює докази незворотно.

16.7 Кейс: наскрізне реагування на зараження

Розглянемо наскрізне реагування на підтверджене зараження (захисна задача, за моделлю NIST [81]).

Крок 1. Підготовка (до інциденту). Налагоджено централізовану телеметрію (розділ 10), визначено команду, ролі, план, інструменти; проведено навчання.

Крок 2. Виявлення й аналіз. Сигнал (розділи 10–11) підтверджено як інцидент; оцінено попередній масштаб, зафіксовано летючі докази (пам'ять) із дотриманням порядку летючості й цілісності.

Крок 3. Стримування. Уражені вузли ізольовано, зв'язок із командними серверами заблоковано, скомпрометовані облікові записи відключено — із балансом між зупинкою шкоди й збереженням доказів.

Крок 4. Аналіз і усунення. Форензичний аналіз відновлює ланцюг ураження (16.1): вектор доступу, персистентність, дії, масштаб. На основі повного розуміння усуваємо *всі* механізми присутності й закриваємо вразливість доступу.

Крок 5. Відновлення та уроки. Відновлюємо системи з перевірених копій, переконуємося у відсутності повторного зараження, видобуваємо індикатори для розвідки (розділ 12) і нові правила виявлення (розділ 11), документуємо уроки й удосконалюємо підготовку.

Кейс демонструє, що реагування — це дисциплінований цикл, у якому підготовка визначає успіх, стримування балансує швидкість і докази, усунення вимагає повноти, а уроки замикають контур удосконалення. Форензика перетворює інцидент на знання, що зміцнює захист проти наступних загроз.

Висновки до розділу

Виявлення загрози — початок роботи: за ним настає реагування (стримування, усунення, відновлення) і форензика (встановлення фактів). Зрілість захисту визначається здатністю швидко й правильно відреагувати, бо невміле реагування завдає більшої шкоди, ніж сам інцидент. Реагування — структурований цикл фаз (рис. 16.1), у якому недооцінена підготовка визначає успіх решти, а уроки живлять підготовку до наступних інцидентів. Стримування балансує швидкість і збереження доказів, усунення вимагає повноти (один лишений механізм персистентності повертає загрозу), відновлення — впевненості в усуненні й закритті вразливості. Цифрова форензика спирається на цілісність доказів, ланцюг збереження й порядок летючості (рис. 16.2; ISO/IEC 27037), а для ЗПК особливо цінний аналіз пам'яті. Повнота розслідування (16.1) визначає залишковий ризик: прогалини в ланцюгу ураження спричиняють повторні інциденти. Антифорензику долають надмірністю джерел, завчасним централізованим збором і аналізом пам'яті. В Україні реагування регулюється законодавством і взаємодією з CERT-UA. Головний висновок: реагування й форензика перетворюють виявлений ін-

цидент на кероване усунення й на знання, що зміцнює захист; їх сила — у підготовці, повноті, цілісності доказів і замкнутому циклі вдосконалення.

Питання для самоперевірки

1. Чому виявлення загрози є початком, а не кінцем роботи? Чим небезпечне невміле реагування?
2. Опишіть фази життєвого циклу реагування (рис. 16.1). Чому підготовка є критичною й недооціненою?
3. Що таке стримування? Як воно балансує швидкість зупинки шкоди й збереження доказів?
4. Чому усунення вимагає повноти? Що станеться, якщо лишити механізм персистентності?
5. Які умови має задовольняти відновлення, щоб уникнути повторного зараження?
6. Що таке цифрова форензика? Назвіть її ключові принципи (цілісність, ланцюг збереження, порядок летючості).
7. Поясніть порядок летючості (рис. 16.2). Чому пам'ять знімають до вимкнення вузла?
8. Що відновлює форензичний аналіз інциденту ЗПК? Як поєднуються зразок, артефакти й телеметрія?
9. Поясніть повноту розслідування (16.1). Чому прогалини визначають залишковий ризик?
10. Що таке антифорензика? Наведіть приклади технік приховування слідів.
11. Якими принципами долають антифорензику? Чому централізована телеметрія є найкращою протидією?
12. Як реагування на кіберінциденти регулюється в Україні? Яка роль CERT-UA та стандартів ISO?
13. Опишіть наскрізне реагування на зараження (кейс підрозділу 16.7). Як уроки замикають цикл?
14. Дослідницьке: розробіть план реагування й форензичного збору для модельного інциденту ЗПК та оцініть його відповідність ISO/IEC 27037.
15. Дослідницьке: дослідіть ефективність аналізу пам'яті проти набору технік антифорензики на модельних зразках і оцініть залишкові прогалини.

РОЗДІЛ 17

Організаційне управління та стратегія протидії злоякісному коду

Технічні засоби (розділи 6–16) дають ефект лише в межах зрілої системи **управління** (governance): без стратегії, процесів, ролей, бюджету, людей і культури найкращі технології реалізують мізерну частку потенціалу. Цей завершальний розділ піднімається з технічного на організаційно-стратегічний рівень: він розглядає протидію ЗПК як керовану функцію — із цілями, ризиками, інвестиціями, метриками й безперервним удосконаленням. Він узагальнює наскрізні принципи посібника (ешелонування, каскад вартості й покриття, перенесення на стійкі ознаки, припущення компрометації) у цілісну стратегію й показує, що протидія ЗПК — не набір засобів, а дисципліна управління ризиком в умовах активного супротивника. Розділ розглядає управління безпекою й рамкові моделі, управління ризиком та економіку безпеки, чинник людини й культуру, стратегію кіберстійкості, метрики й зрілість, а також національний вимір в Україні, і завершується кейсом побудови програми протидії ЗПК.

17.1 Управління безпекою й рамкові моделі

Управління безпекою визначає, *хто*, *що* і *навіщо* робить для захисту: цілі, політики, ролі, відповідальність, процеси, контроль. Рамкові моделі структурують цю діяльність. NIST Cybersecurity Framework [164] виділяє функції — *управляти*, *ідентифікувати*, *захищати*, *виявляти*, *реагувати*, *відновлювати*, — що охоплюють повний цикл і прямо відповідають структурі цього посібника (моделювання загроз, профілактика, виявлення, реагування). ISO/IEC 27001 [121] задає вимоги до системи управління інформаційною безпекою (СУІБ) із циклом постійного вдосконалення (плануй-виконуй-перевіряй-дій).

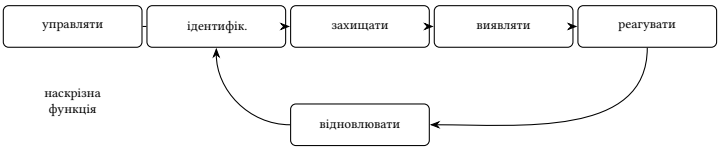


Рис. 17.1 — Функції управління кібербезпекою (за NIST CSF) як безперервний цикл

Значення рамкових моделей — не в бюрократії, а в *повноті* й *узгодженості*: вони гарантують, що жодну функцію не пропущено (наприклад, орга-

нізація не зосередилася лише на запобіганні, занедбавши виявлення й реагування), і що діяльність узгоджена з цілями й ризиком. Для магістранта-дослідника вони дають спільну мову для систематизації технічних методів у цілісну систему. Основні рамкові моделі та їх призначення зіставлено в табл. 17.1.

Таблиця 17.1 — Рамкові моделі управління кібербезпекою

Модель	Призначення	Форма
NIST CSF	функції захисту, спільна мова	добровільна рамка
ISO/IEC 27001	система управління (СУІБ)	сертифікована
C2M2	оцінка й розвиток зрілості	самооцінка

17.2 Управління ризиком та економіка безпеки

Ресурси завжди обмежені, тому протидія ЗПК — це управління *ризиком* (розділ 4), а не прагнення недосяжної абсолютної безпеки. Рішення про інвестиції в захист ґрунтуються на оцінці ризику (ймовірність $\times$ наслідки) і співвідношенні вартості заходу з очікуваним зниженням ризику [105]. Економіка інформаційної безпеки [52] показує, що багато проблем мають не технічну, а економічну природу: хибні стимули, перенесення ризику, зовнішні ефекти.

Класична модель Гордона–Лоба [111] формалізує оптимальний рівень інвестицій. Якщо v — потенційні збитки, а інвестиція z знижує ймовірність реалізації загрози, то чиста вигода

$$\mathrm{ENBIS}(z) = v\big[S(0) - S(z)\big] - z, \tag{17.1}$$

де $S(z)$ — залишкова ймовірність успішної атаки за інвестиції z, максимізується за скінченного рівня інвестицій, що *менший* за очікувані збитки. Формула (17.1) формалізує фундаментальну істину: раціональна мета — не нульовий ризик (невигідно й недосяжно), а оптимальний баланс, де гранична вигода від додаткового захисту дорівнює його граничній вартості. Це пояснює, чому розумна стратегія пріоритезує заходи з найбільшим відношенням зниження ризику до вартості — зокрема дешеву профілактику й ешелонування (розділ 10) перед дорогими точковими рішеннями.

17.3 Чинник людини й культура безпеки

Людина — водночас найслабша ланка й найцінніший актив захисту. Значна частина інцидентів починається з людини: фішинг, слабкі паролі, помилки конфігурації, зневага до оновлень [223]. Жодна технологія не компенсує повністю необережного користувача чи адміністратора. Тому куль-

тура безпеки — спільне усвідомлення загроз і відповідальна поведінка — є невіддільною частиною протидії.

Проте підвищення обізнаності часто неефективне [59]: разові тренінги не змінюють поведінки. Дієві підходи: вбудовування безпеки в процеси (щоб безпечна дія була й найзручнішою), реалістичні навчання, позитивна культура (без звинувачення, що заохочує повідомляти про помилки й підозри), узгодження стимулів. Ключовий зсув — від погляду на людину як на проблему до погляду на неї як на *сенсор* і учасника захисту: навчений працівник, що вчасно повідомляє про підозрілий лист, — цінний елемент виявлення (розділ 11).

Окрему категорію становлять *внутрішні* загрози — зловживання з боку осіб із легітимним доступом (незадоволений працівник, завербований інсайдер, скомпрометований обліковий запис адміністратора). Вони особливо небезпечні, бо діють із-за межі довіри, обходячи периметровий захист, і зливаються з легітимною діяльністю — та сама проблема контексту, що й у LotL (розділ 11). Протидія поєднує технічне (найменші привілеї, розділення обов'язків, моніторинг аномалій поведінки користувачів) з організаційним (перевірка при прийомі, чіткі політики, здорова культура, що знижує мотивацію до зловживань). Внутрішні загрози показово ілюструють, що управління безпекою не зводиться до технологій: воно невіддільне від управління персоналом, процесами й організаційним кліматом.

Зрештою, роль *керівництва* є вирішальною. Протидія ЗПК потребує бюджету, повноважень і пріоритету, які може надати лише вище керівництво; без його залучення програма безпеки лишається технічною ініціативою без ресурсів і впливу. Тому зрілі рамкові моделі [164] виділяють управління (governance) в окрему наскрізну функцію: безпека починається не з інструментів, а з рішення керівництва ставитися до кіберризику як до одного з ключових бізнес-ризиків, нарівні з фінансовим чи репутаційним.

17.4 Стратегія кіберстійкості

Сучасна стратегія виходить із **припущення компрометації** (assume breach, розділ 13): абсолютне запобігання недосяжне, тому мета зміщується від недопущення будь-якої компрометації до **кіберстійкості** [186] — здатності витримувати атаки, обмежувати їх вплив, швидко відновлюватися й продовжувати виконувати місію навіть під час атаки. Це фундаментальний зсув парадигми: не фортеця, яку або взяли, або ні, а організм, що виживає й адаптується попри ушкодження.

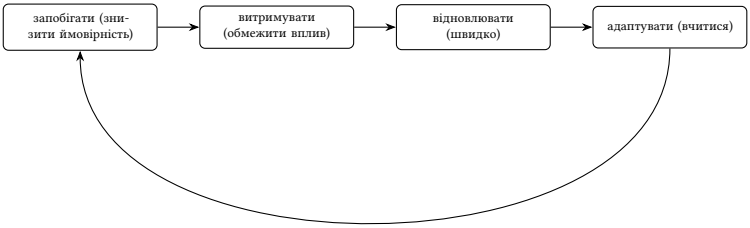


Рис. 17.2 — Кіберстійкість: від запобігання до здатності витримувати, відновлюватися й адаптуватися

Принципи стійкості: ешелонування (розділ 10), сегментація (обмеження поширення), надмірність і резервування, швидке виявлення й реагування (розділ 16), припущення компрометації й нульова довіра (розділ 15), безперервне навчання на інцидентах. Стійкість поєднує всі технічні розділи посібника в єдину стратегію: кожен метод — рубіж, що або знижує ймовірність, або обмежує вплив, або пришвидшує відновлення.

Практичний наслідок парадигми стійкості — перенесення частини інвестицій із запобігання на *обмеження наслідків* і *відновлення*. Організація, що вклала все в периметр, але не здатна швидко виявити, стримати й відновитися, зазнає катастрофічних збитків від єдиного прориву — саме це сталося з багатьма жертвами NotPetya (розділ 15), чиї пласкі мережі дозволили коду поширитися без перешкод, а відсутність перевірених офлайн-копій унеможливила швидке відновлення. Натомість стійка організація проєктує системи так, щоб компрометація частини не означала втрати цілого: сегментація локалізує ураження, надмірність зберігає функцію, перевірені резервні копії уможливлюють відновлення. Ця зміна акценту — від «не впустити» до «вистояти й відновитися» — є найважливішим стратегічним уроком сучасної протидії ЗПК.

17.5 Метрики, зрілість і національний вимір

Управління потребує **вимірювання**: не можна керувати тим, чого не міряють. Метрики протидії ЗПК охоплюють технічні (повнота й точність виявлення, час присутності (10.1), покриття технік) та управлінські (зрілість процесів, частка закритих вразливостей, час реагування, результати навчань). **Моделі зрілості** [219] описують еволюцію від реактивного й хаотичного стану до керованого, вимірюваного й оптимізованого, даючи дорожню карту вдосконалення. Приклади метрик обох груп наведено в табл. 17.2.

Таблиця 17.2 — Метрики протидії ЗПК

Група	Приклади
Технічні	повнота й точність виявлення, час присутності, покриття технік ATT&CK
Операційні	час реагування, потік тривог на аналітика, PPV
Управлінські	зрілість процесів, частка закритих вразливостей, результати навчань

Важливо добирати метрики обачно: погано дібрана метрика спотворює поведінку (що міряють, те й оптимізують, інколи на шкоду суті). Наприклад, гонитва за кількістю закритих тривог заохочує поверхневе їх закриття, а не якісне розслідування. Тому зрілі програми вимірюють *результати* (скорочення часу присутності, зниження реалізованого ризику), а не лише *активність* (кількість дій), і регулярно переглядають самі метрики.

Зрілість зручно характеризувати як ступінь просування рівнями. Якщо виділити L рівнів зрілості, а організація досягла рівня ℓ з якістю реалізації $\rho \in [0;1]$, то інтегральний показник зрілості

$$M = \frac{\ell - 1 + \rho}{L - 1} \in [0;1], \tag{17.2}$$

дозволяє відстежувати прогрес і порівнювати стан у часі. Формула (17.2) підкреслює, що зрілість — не разове досягнення рівня, а безперервний рух, бо загрози еволюціонують, і стан, достатній сьогодні, застаріває завтра.

Національний вимір для України має стратегічне значення: Стратегія кібербезпеки України [42] визначає державну рамку протидії, а найінтенсивніший у світі досвід відбиття атак на критичну інфраструктуру робить українські організаційно-управлінські практики [32] цінними й для світу. Протидія ЗПК на національному рівні поєднує державне регулювання (розділ 2), розвідку загроз (CERT-UA, розділ 12), захист критичної інфраструктури (розділ 14) й міжнародну співпрацю в єдину систему кіберстійкості держави.

17.6 Кейс: побудова програми протидії ЗПК

Розглянемо побудову організаційної програми протидії ЗПК (управлінська задача, що синтезує весь посібник).

Крок 1. Контекст і ризик. Визначаємо місію, активи, модель загроз (розділ 4), регуляторні вимоги (розділ 2) й апетит до ризику. Пріоритезуємо за ризиком, а не за технічною модою.

Крок 2. Рамка й цілі. Обираємо рамкову модель (NIST CSF, ISO 27001), ставимо вимірювані цілі за всіма функціями: ідентифікація, захист, виявлення, реагування, відновлення.

Крок 3. Ешелонований захист. Будуємо рубежі за каскадом вартості й покриття: профілактика й гартування (розділ 10), виявлення (сигнатури, поведінка, МН — розділи 6–11), розвідка й полювання (розділи 12–13), реагування й форензика (розділ 16), пріоритезуючи заходи з найкращим відношенням зниження ризику до вартості (17.1).

Крок 4. Люди й процеси. Визначаємо ролі (SOC/CSIRT), процеси реагування, навчання, культуру безпеки; перетворюємо людей на сенсори захисту.

Крок 5. Вимірювання й удосконалення. Запроваджуємо метрики й модель зрілості (17.2), безперервно вимірюємо, вчимося на інцидентах і вдосконалюємося, адаптуючись до еволюції загроз.

Кейс синтезує весь посібник: ефективна протидія ЗПК — це не набір засобів, а керована, вимірювана, безперервно вдосконалювана програма, що балансує ризик і вартість, поєднує технології, процеси й людей і прагне не абсолютної безпеки, а кіберстійкості в умовах активного супротивника.

Висновки до розділу

Технічні засоби реалізують свій потенціал лише в межах зрілого управління: протидія ЗПК — це керована функція з цілями, ризиками, інвестиціями, метриками й безперервним удосконаленням, а не набір технологій. Рамкові моделі (NIST CSF, ISO/IEC 27001) забезпечують повноту й узгодженість функцій — від ідентифікації до відновлення. Протидія є управлінням ризиком, а не прагненням абсолютної безпеки: економіка безпеки (17.1) показує, що раціональна мета — оптимальний баланс вартості й зниження ризику, а не нульовий ризик. Чинник людини — найслабша ланка й цінний сенсор водночас — вимагає культури безпеки, вбудованої в процеси, а не разових тренінгів. Сучасна стратегія виходить із припущення компрометації й зміщується до кіберстійкості — здатності витримувати, відновлюватися й адаптуватися, — що поєднує всі технічні розділи в єдине ціле. Управління потребує вимірювання й моделей зрілості (17.2) для безперервного вдосконалення. Для України протидія ЗПК має стратегічний національний вимір. Головний і наскрізний висновок посібника: ефективна протидія злоякісному коду — це дисципліна управління ризиком в умовах активного, мислячого супротивника, що поєднує технології, процеси й людей у кероване, вимірюване, кіберстійке ціле, яке не прагне недосяжної абсолютної безпеки, а невпинно підвищує вартість атаки й здатність організації вистояти та відновитися.

Питання для самоперевірки

1. Чому технічні засоби реалізують потенціал лише в межах зрілого управління?

2. Що визначає управління безпекою? Яку роль відіграють рамкові моделі (NIST CSF, ISO 27001)?
3. Як функції NIST CSF (рис. 17.1) відповідають структурі протидії ЗПК?
4. Чому протидія ЗПК є управлінням ризиком, а не прагненням абсолютної безпеки?
5. Поясніть модель Гордона–Лоба (17.1). Чому оптимальні інвестиції менші за очікувані збитки?
6. Чому багато проблем безпеки мають економічну, а не технічну природу?
7. Чому людина є водночас найслабшою ланкою й цінним активом? Чому разові тренінги неефективні?
8. Які підходи дієво формують культуру безпеки? Що означає погляд на людину як на сенсор?
9. Що таке припущення компрометації? Чим кіберстійкість відрізняється від парадигми фортеці?
10. Назвіть принципи кіберстійкості (рис. 17.2). Як вони поєднують технічні розділи посібника?
11. Навіщо потрібні метрики й моделі зрілості? Поясніть показник зрілості (17.2).
12. У чому національний вимір протидії ЗПК для України? Як поєднуються його складові?
13. Опишіть побудову програми протидії ЗПК (кейс підрозділу 17.6). Як він синтезує весь посібник?
14. Дослідницьке: розробіть програму протидії ЗПК для модельної організації за обраною рамковою моделлю й оцініть її зрілість (17.2) та пріоритети інвестицій (17.1).
15. Дослідницьке: проаналізуйте співвідношення вартості та зниження ризику для набору заходів протидії ЗПК і побудуйте оптимальний за Гордоном–Лобом портфель інвестицій.

Список використаних джерел

[1] Аналіз атаки NotPetya як прикладу компрометації ланцюга постачання програмного забезпечення в Україні. *Захист інформації.* 2023. Т. 25, № 1.

[2] Аналіз діяльності APT-угруповань, спрямованих проти критичної інфраструктури України. *Захист інформації.* 2023. Т. 25, № 2.

[3] Аналіз кібератак та діяльності APT груп в Україні. *Кібербезпека: освіта, наука, техніка.* 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/574.

[4] Архітектура та можливості систем виявлення й реагування на кінцевих точках (EDR) у корпоративних мережах. *Кібербезпека: освіта, наука, техніка.* 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/731.

[5] Виявлення запакованого шкідливого програмного забезпечення на основі аналізу ентропії та структурних ознак. *Захист інформації.* 2023. Т. 25, № 3.

[6] Виявлення цілеспрямованих атак на об'єкти критичної інфраструктури України: методи та підходи. *Захист інформації.* 2023. Т. 25, № 3.

[7] Дослідження методів виявлення вразливостей програмного забезпечення та ІКС на базі ОС Windows із розробленням засобу автоматизованого аналізу. *Кібербезпека: освіта, наука, техніка.* 2026. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1302.

[8] ДСТУ EN ISO/IEC 27001:2023. Інформаційна безпека, кібербезпека та захист приватності. Системи керування інформаційною безпекою. Вимоги. Київ: ДП «УкрНДНЦ», 2023.

[9] ДСТУ EN ISO/IEC 27002:2023. Інформаційна безпека, кібербезпека та захист приватності. Заходи інформаційної безпеки. Київ: ДП «УкрНДНЦ», 2023.

[10] Загальні вимоги до кіберзахисту об'єктів критичної інфраструктури: затв. постановою Кабінету Міністрів України від 19.06.2019 № 518. URL: https://zakon.rada.gov.ua/laws/show/518-2019-п.

[11] Загрози ланцюга постачання програмного забезпечення та методи забезпечення цілісності й довіри до компонентів. *Кібербезпека: освіта, наука, техніка.* 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/802.

[12] Застосування динамічного аналізу та пісочниць для дослідження шкідливого програмного забезпечення. *Кібербезпека: освіта, наука, техніка.* 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/615.

[13] Застосування методів машинного навчання для виявлення шкідливого програмного забезпечення. *Кібербезпека: освіта, наука, техніка.* 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/688.

[14] Застосування розвідки кіберзагроз (threat intelligence) для проактивного захисту інформаційних систем. *Кібербезпека: освіта, наука, техніка*. 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/759.

[15] Захист автоматизованих систем керування технологічними процесами об'єктів критичної інфраструктури України від кіберзагроз. *Кібербезпека: освіта, наука, техніка*. 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/788.

[16] Ільєнко А. В., Ільєнко С. С., Яковенко О. Л., Галич Є. О., Павленко В. Г. Застосування штучного інтелекту для прогнозування кіберзагроз. *Кібербезпека: освіта, наука, техніка*. 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/631.

[17] Команда реагування на комп'ютерні надзвичайні події України CERT-UA. Аналітичні звіти та індикатори компрометації. Державна служба спеціального зв'язку та захисту інформації України. URL: https://cert.gov.ua/.

[18] Кримінальний кодекс України: статті 361 (несанкціоноване втручання в роботу ЕОМ), 361-1 (створення з метою використання, поширення або збуту шкідливих програмних чи технічних засобів), 361-2, 362, 363, 363-1. URL: https://zakon.rada.gov.ua/laws/show/2341-14.

[19] Кримінальний процесуальний кодекс України від 13.04.2012 № 4651-VI (щодо вилучення й дослідження електронних доказів). URL: https://zakon.rada.gov.ua/laws/show/4651-17.

[20] Лунгол О. М. Багаторівнева модель атрибуції кібератак. *Кібербезпека: освіта, наука, техніка*. 2025. Спеціальний випуск. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/904.

[21] Методи обфускації програмного коду та підходи до їх подолання в задачах аналізу шкідливого програмного забезпечення. *Кібербезпека: освіта, наука, техніка*. 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/702.

[22] Методи цифрової криміналістики при розслідуванні інцидентів, пов'язаних зі шкідливим програмним забезпеченням. *Захист інформації*. 2023. Т. 25, № 2.

[23] Метод захисту програмного забезпечення на основі гібридного аналізу коду. *Кібербезпека: освіта, наука, техніка*. 2025. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/871.

[24] Моделі та методи виявлення шкідливої активності у середовищах Інтернету речей. *Захист інформації*. 2023. Т. 25, № 4.

[25] Моніторинг подій безпеки та виявлення аномальної активності на кінцевих вузлах. *Захист інформації*. 2023. Т. 25, № 4.

[26] НД ТЗІ 2.5-004-99. Критерії оцінки захищеності інформації в комп'ютерних системах від несанкціонованого доступу. Київ: ДСТСЗІ СБ України, 1999.

[27] НД ТЗІ 3.7-003-05. Порядок проведення робіт із створення комплексної системи захисту інформації в інформаційно-телекомунікаційній системі. Київ: ДСТСЗІ СБ України, 2005.

[28] НД ТЗІ 1.1-003-99. Термінологія в галузі захисту інформації в комп'ютерних системах від несанкціонованого доступу. Київ: ДСТСЗІ СБ України, 1999. URL: https://cip.gov.ua/ua/news/normativni-dokumenti-z-pitan-tzi.

[29] НД ТЗІ 1.4-001-2000. Типове положення про службу захисту інформації в автоматизованій системі. Київ: ДСТСЗІ СБ України, 2000.

[30] Нейромережеві моделі класифікації шкідливого програмного забезпечення за статичними ознаками. *Захист інформації*. 2024. Т. 26, № 1.

[31] Опірський І. Р., Мельничук М. Р. Застосування машинного навчання в статичному аналізі безпеки коду. *Кібербезпека: освіта, наука, техніка*. 2026. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/1193.

[32] Організаційно-управлінські засади побудови системи кіберзахисту підприємства в умовах сучасних загроз. *Кібербезпека: освіта, наука, техніка*. 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/830.

[33] Організація процесу реагування на кіберінциденти в установах України відповідно до вимог законодавства. *Кібербезпека: освіта, наука, техніка*. 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/816.

[34] Поведінкові методи виявлення шкідливого програмного забезпечення на основі аналізу системних викликів. *Захист інформації*. 2023. Т. 25, № 2.

[35] Полотай О. І. та ін. Інтеграція IDS/IPS у корпоративні мережі та перевірка playbook на кіберполігонах. *Кібербезпека: освіта, наука, техніка*. 2025. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/889.

[36] Проактивне полювання за кіберзагрозами (threat hunting) як складова сучасної стратегії кіберзахисту. *Кібербезпека: освіта, наука, техніка*. 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/773.

[37] Про електронні комунікації: Закон України від 16.12.2020 № 1089-IX. URL: https://zakon.rada.gov.ua/laws/show/1089-20.

[38] Про затвердження Загальних вимог до кіберзахисту об'єктів критичної інфраструктури та Порядку їх застосування: постанова Кабінету Міністрів України. URL: https://cip.gov.ua/.

[39] Про захист інформації в інформаційно-комунікаційних системах: Закон України від 05.07.1994 № 80/94-ВР. URL: https://zakon.rada.gov.ua/laws/show/80/94-вр.

[40] Про критичну інфраструктуру: Закон України від 16.11.2021 № 1882-IX. URL: https://zakon.rada.gov.ua/laws/show/1882-20.

[41] Про основні засади забезпечення кібербезпеки України: Закон України від 05.10.2017 № 2163-VIII (редакція від 19.10.2025). URL: https://zakon.rada.gov.ua/laws/show/2163-19.

[42] Про рішення Ради національної безпеки і оборони України «Про Стратегію кібербезпеки України». Указ Президента України. Київ, 2021.

[43] Про Стратегію кібербезпеки України: Указ Президента України від 26.08.2021 № 447/2021. URL: https://zakon.rada.gov.ua/laws/show/447/2021.

[44] Розроблення правил виявлення кіберзагроз для систем керування подіями безпеки (SIEM). *Кібербезпека: освіта, наука, техніка*. 2024. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/745.

[45] Трофименко О. Г., Дика А. І., Лобода Ю. Г. Порівняльний аналіз методів тестування безпеки застосунків (SAST, DAST, IAST). *Кібербезпека: освіта, наука, техніка*. 2023. Т. 4, № 20. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/467.

[46] Угода про асоціацію між Україною та Європейським Союзом: ратифіковано Законом України від 16.09.2014 № 1678-VII. URL: https://zakon.rada.gov.ua/laws/show/984_011.

[47] Формування та застосування індикаторів компрометації у процесах виявлення кіберінцидентів. *Захист інформації*. 2023. Т. 25, № 1.

[48] Шевченко С. М., Жданова Ю. Д., Гаркушенко А. М. Аналіз сучасних загроз інформаційній безпеці та підходів до їх класифікації. *Кібербезпека: освіта, наука, техніка*. 2025. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/945.

[49] Anderson H. S., Kharkar A., Filar B., Evans D., Roth P. Learning to Evade Static PE Machine Learning Malware Models via Reinforcement Learning. *arXiv preprint arXiv:1801.08917*. 2018.

[50] Anderson H. S., Roth P. EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models. *arXiv preprint arXiv:1804.04637*. 2018.

[51] Anderson R. Security Engineering: A Guide to Building Dependable Distributed Systems. 3rd ed. Indianapolis: Wiley, 2020. 1232 p. URL: https://www.cl.cam.ac.uk/~rja14/book.html.

[52] Anderson R., Moore T. The Economics of Information Security. *Science*. 2006. Vol. 314, № 5799. P. 610–613. DOI: 10.1126/science.1130992.

[53] Andriesse D., Chen X., van der Veen V., Slowinska A., Bos H. An In-Depth Analysis of Disassembly on Full-Scale x86/x64 Binaries. *Proceedings of the 25th USENIX Security Symposium*. USENIX Association, 2016. P. 583–600.

[54] Antonakakis M., April T., Bailey M., et al. Understanding the Mirai Botnet. *Proceedings of the 26th USENIX Security Symposium*. USENIX Association, 2017. P. 1093–1110.

[55] AV-TEST Institute. Malware Statistics & Trends Report. Magdeburg: AV-TEST GmbH, 2024. URL: https://www.av-test.org/en/statistics/malware/.

[56] Avast Threat Intelligence Team. Progress on CCleaner Investigation. Avast Blog, 2017.

[57] Axelsson S. The Base-Rate Fallacy and the Difficulty of Intrusion Detection. *ACM Transactions on Information and System Security*. 2000. Vol. 3, No. 3. P. 186–205. DOI: 10.1145/357830.357849.

[58] Aycock J. Computer Viruses and Malware. New York: Springer, 2006. 227 p. (Advances in Information Security, Vol. 22).

[59] Bada M., Sasse A. M., Nurse J. R. C. Cyber Security Awareness Campaigns: Why Do They Fail to Change Behaviour? *International Conference on Cyber Security for Sustainable Society*. 2015. P. 118–131.

[60] Bailey M., Oberheide J., Andersen J., Mao Z. M., Jahanian F., Nazario J. Automated Classification and Analysis of Internet Malware. *Recent Advances in Intrusion Detection (RAID 2007)*. LNCS Vol. 4637. Berlin: Springer, 2007. P. 178–197.

[61] Bartholomew B., Guerrero-Saade J. A. Wave Your False Flags! Deception Tactics Muddying Attribution in Targeted Attacks. *Virus Bulletin Conference*. Denver, 2016.

[62] Bayer U., Comparetti P. M., Hlauschek C., Kruegel C., Kirda E. Scalable, Behavior-Based Malware Clustering. *Proceedings of the 16th Annual Network and Distributed System Security Symposium (NDSS)*. San Diego: ISOC, 2009.

[63] Bianco D. The Pyramid of Pain. Enterprise Detection & Response Blog, 2013. URL: http://detect-respond.blogspot.com/2013/03/the-pyramid-of-pain.html.

[64] Biggio B., Roli F. Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning. *Pattern Recognition*. 2018. Vol. 84. P. 317–331.

[65] Blunden B. The Rootkit Arsenal: Escape and Evasion in the Dark Corners of the System. 2nd ed. Burlington: Jones & Bartlett Learning, 2012. 784 p.

[66] Bohannon D., Holmes L. Revoke-Obfuscation: PowerShell Obfuscation Detection Using Science. *Black Hat USA*. Las Vegas, 2017.

[67] Branco R. R., Barbosa G. N., Neto P. D. Scientific but Not Academical Overview of Malware Anti-Debugging, Anti-Disassembly and Anti-VM Technologies. *Black Hat USA*. Las Vegas, 2012.

[68] Brezinski D., Killalea T. Guidelines for Evidence Collection and Archiving. RFC 3227. IETF, 2002.

[69] Brumley D., Jager I., Avgerinos T., Schwartz E. J. BAP: A Binary Analysis Platform. *Computer Aided Verification (CAV 2011)*. LNCS Vol. 6806. Berlin: Springer, 2011. P. 463–469.

[70] Caltagirone S., Pendergast A., Betz C. The Diamond Model of Intrusion Analysis. Center for Cyber Intelligence Analysis and Threat Research, 2013.

[71] CAPEC: Common Attack Pattern Enumeration and Classification. The MITRE Corporation. URL: https://capec.mitre.org/.

[72] Carrier B. File System Forensic Analysis. Boston: Addison-Wesley, 2005. 600 p.

[73] Casey E. Digital Evidence and Computer Crime: Forensic Science, Computers, and the Internet. 3rd ed. Waltham: Academic Press, 2011. 840 p.

[74] CERT-UA. Огляд діяльності урядової команди реагування на комп'ютерні надзвичайні події України: щорічні звіти. Київ: Держспецзв'язку, 2022–2024. URL: https://cert.gov.ua/reports.

[75] Chen X., Andersen J., Mao Z. M., Bailey M., Nazario J. Towards an Understanding of Anti-Virtualization and Anti-Debugging Behavior in Modern Malware. *IEEE International Conference on Dependable Systems and Networks (DSN)*. Anchorage: IEEE, 2008. P. 177–186.

[76] Cherepanov A. Win32/Industroyer: A New Threat for Industrial Control Systems. ESET White Paper, 2017.

[77] Cherepanov A., Lipovsky R. Industroyer: Biggest Threat to Industrial Control Systems since Stuxnet. ESET White Paper. Bratislava: ESET, 2017.

[78] Chess D. M., White S. R. An Undetectable Computer Virus. *Proceedings of the Virus Bulletin Conference*. 2000.

[79] Christodorescu M., Jha S., Seshia S. A., Song D., Bryant R. E. Semantics-Aware Malware Detection. *2005 IEEE Symposium on Security and Privacy*. Oakland: IEEE, 2005. P. 32–46. DOI: 10.1109/SP.2005.20.

[80] Chuvakin A. Named: Endpoint Threat Detection & Response. Gartner Blog Network, 2013. URL: https://blogs.gartner.com/anton-chuvakin/2013/07/26/named-endpoint-threat-detection-response/.

[81] Cichonski P., Millar T., Grance T., Scarfone K. Computer Security Incident Handling Guide. NIST Special Publication 800-61, Revision 2. Gaithersburg: NIST, 2012. DOI: 10.6028/NIST.SP.800-61r2.

[82] CIS Critical Security Controls, Version 8.1. Center for Internet Security, 2024. URL: https://www.cisecurity.org/controls/v8-1.

[83] CISA. Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations (SolarWinds). Cybersecurity and Infrastructure Security Agency Alert AA20-352A, 2020.

[84] Cisco-Talos. ClamAV: Open Source Antivirus Engine — Signatures and Database Formats. URL: https://docs.clamav.net/.

[85] Cohen F. Computer Viruses. PhD Dissertation. University of Southern California, 1986.

[86] Cohen F. Computer Viruses: Theory and Experiments. *Computers & Security*. 1987. Vol. 6, No. 1. P. 22–35. DOI: 10.1016/0167-4048(87)90122-2.

[87] Collberg C., Nagra J. Surreptitious Software: Obfuscation, Watermarking, and Tamperproofing for Software Protection. Boston: Addison-Wesley, 2009. 792 p.

[88] Convention on Cybercrime (Budapest Convention). ETS No. 185. Budapest: Council of Europe, 2001. URL: https://www.coe.int/en/web/cybercrime/the-budapest-convention.

[89] Cuckoo Foundation. Cuckoo Sandbox: Automated Malware Analysis — Documentation. URL: https://cuckoo.readthedocs.io/.

[90] CVE: Common Vulnerabilities and Exposures. The MITRE Corporation. URL: https://www.cve.org/.

[91] Directive (EU) 2022/2555 of the European Parliament and of the Council of 14 December 2022 on measures for a high common level of cybersecurity across the Union (NIS 2 Directive). *Official Journal of the EU*. L 333. P. 80–152. URL: https://eur-lex.europa.eu/eli/dir/2022/2555/oj.

[92] Dobrovolsky G. et al. (CERT-UA, ESET, Microsoft). WhisperGate, HermeticWiper, IsaacWiper, CaddyWiper and the 2022 destructive campaigns against Ukraine: joint analyses. 2022.

[93] Dullien T., Rolles R. Graph-based comparison of Executable Objects. *Symposium sur la sécurité des technologies de l'information et des communications (SSTIC)*. 2005.

[94] Eagle C. The IDA Pro Book. 2nd ed. San Francisco: No Starch Press, 2011. 672 p.

[95] Egele M., Scholte T., Kirda E., Kruegel C. A Survey on Automated Dynamic Malware-Analysis Techniques and Tools. *ACM Computing Surveys*. 2012. Vol. 44, № 2. Art. 6. P. 1–42. DOI: 10.1145/2089125.2089126.

[96] Enck W., Octeau D., McDaniel P., Chaudhuri S. A Study of Android Application Security. *Proceedings of the 20th USENIX Security Symposium*. USENIX Association, 2011. P. 21–36.

[97] ENISA Threat Landscape 2024. Athens: European Union Agency for Cybersecurity, 2024. URL: https://www.enisa.europa.eu/publications/enisa-threat-landscape-2024.

[98] ENISA. Threat Landscape for Supply Chain Attacks. European Union Agency for Cybersecurity, 2021.

[99] ESET Research. New TeleBots backdoor: First evidence linking Industroyer to NotPetya. ESET WeLiveSecurity, 2018. URL: https://www.welivesecurity.com/.

[100] Falliere N., Murchu L. O., Chien E. W32.Stuxnet Dossier. Version 1.4. Symantec Security Response, 2011.

[101] Fawcett T. An Introduction to ROC Analysis. *Pattern Recognition Letters*. 2006. Vol. 27, No. 8. P. 861–874. DOI: 10.1016/j.patrec.2005.10.010.

[102] FIRST. Common Vulnerability Scoring System Version 4.0: Specification Document. Forum of Incident Response and Security Teams, 2023. URL: https://www.first.org/cvss/v4.0/.

[103] FIRST. Exploit Prediction Scoring System (EPSS). Forum of Incident Response and Security Teams. URL: https://www.first.org/epss/.

[104] Freund J., Jones J. Measuring and Managing Information Risk: A FAIR Approach. Oxford: Butterworth-Heinemann, 2014. 408 p.

[105] Freund J., Jones J. Measuring and Managing Information Risk: A FAIR Approach. Waltham: Butterworth-Heinemann, 2014. 408 p.

[106] Garfinkel S. Anti-Forensics: Techniques, Detection and Countermeasures. *Proceedings of the 2nd International Conference on i-Warfare and Security*. 2007. P. 77–84.

[107] Gartner. Market Guide for Extended Detection and Response. Gartner Research, 2021.

[108] Gibert D., Mateu C., Planes J. The Rise of Machine Learning for Detection and Classification of Malware: Research Developments, Trends and Challenges. *Journal of Network and Computer Applications*. 2020. Vol. 153. Art. 102526.

[109] Google. Android Security Model and Application Sandbox — Android Open Source Project Documentation. 2024. URL: https://source.android.com/docs/security.

[110] Gordon L. A., Loeb M. P. The Economics of Information Security Investment. *ACM Transactions on Information and System Security*. 2002. Vol. 5, No. 4. P. 438–457. DOI: 10.1145/581271.581274.

[111] Gordon L. A., Loeb M. P. The Economics of Information Security Investment. *ACM Transactions on Information and System Security*. 2002. Vol. 5, № 4. P. 438–457.

[112] Greenberg A. Sandworm: A New Era of Cyberwar and the Hunt for the Kremlin's Most Dangerous Hackers. New York: Doubleday, 2019. 348 p.

[113] Grosse K., Papernot N., Manoharan P., Backes M., McDaniel P. Adversarial Examples for Malware Detection. *Computer Security — ESORICS 2017*. LNCS Vol. 10493. Cham: Springer, 2017. P. 62–79.

[114] Grégio A. R. A. et al. Toward a Taxonomy of Malware Behaviors. *The Computer Journal*. 2015. Vol. 58, No. 10. P. 2758–2777. DOI: 10.1093/comjnl/bxv047.

[115] GTFOBins Project. Curated List of Unix Binaries That Can Be Used to Bypass Local Security Restrictions. URL: https://gtfobins.github.io/.

[116] Guo F., Ferrie P., Chiueh T. A Study of the Packer Problem and Its Solutions. *Recent Advances in Intrusion Detection (RAID 2008)*. LNCS Vol. 5230. Berlin: Springer, 2008. P. 98–115.

[117] Hoglund G., Butler J. Rootkits: Subverting the Windows Kernel. Boston: Addison-Wesley, 2005. 352 p.

[118] Hosseini A. Ten Process Injection Techniques: A Technical Survey of Common and Trending Process Injection Techniques. Elastic Security Research, 2017.

[119] Hutchins E. M., Cloppert M. J., Amin R. M. Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains. *Leading Issues in Information Warfare & Security Research*. 2011. Vol. 1. P. 80–106.

[120] International Electrotechnical Commission. IEC 62443: Industrial Communication Networks — Network and System Security. Geneva: IEC, 2018–2021.

[121] International Organization for Standardization. ISO/IEC 27001: Information Security Management Systems — Requirements. Geneva: ISO, 2022.

[122] International Organization for Standardization. ISO/IEC 27037: Guidelines for Identification, Collection, Acquisition and Preservation of Digital Evidence. Geneva: ISO, 2012.

[123] ISO 22301:2019. Security and resilience — Business continuity management systems — Requirements. Geneva: ISO, 2019.

[124] ISO 31000:2018. Risk management — Guidelines. Geneva: ISO, 2018. URL: https://www.iso.org/standard/65694.html.

[125] ISO/IEC 27005:2022. Information security, cybersecurity and privacy protection — Guidance on managing information security risks. Geneva: ISO, 2022. URL: https://www.iso.org/standard/80585.html.

[126] ISO/IEC 27032:2023. Cybersecurity — Guidelines for Internet security. Geneva: ISO, 2023. URL: https://www.iso.org/standard/76070.html.

[127] ISO/IEC 27035-1:2023. Information technology — Information security incident management — Part 1: Principles and process. Geneva: ISO, 2023. URL: https://www.iso.org/standard/78973.html.

[128] ISO/IEC 27037:2012. Guidelines for identification, collection, acquisition and preservation of digital evidence. Geneva: ISO, 2012. URL: https://www.iso.org/standard/44381.html.

[129] Jang J., Brumley D., Venkataraman S. BitShred: Feature Hashing Malware for Scalable Triage and Semantic Analysis. *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS '11)*. New York: ACM, 2011. P. 309–320.

[130] Junod P., Rinaldini J., Wehrli J., Michielin J. Obfuscator-LLVM — Software Protection for the Masses. *2015 IEEE/ACM 1st International Workshop on Software Protection (SPRO)*. Florence: IEEE, 2015. P. 3–9.

[131] Karantzas G., Patsakis C. An Empirical Assessment of Endpoint Detection and Response Systems against Advanced Persistent Threats Attack Vectors. *Journal of Cybersecurity and Privacy*. 2021. Vol. 1, № 3. P. 387–421.

[132] Kay S. M. Fundamentals of Statistical Signal Processing. Vol. II: Detection Theory. Upper Saddle River: Prentice Hall, 1998. 560 p.

[133] Kirat D., Vigna G., Kruegel C. BareCloud: Bare-metal Analysis-based Evasive Malware Detection. *Proceedings of the 23rd USENIX Security Symposium*. USENIX Association, 2014. P. 287–301.

[134] Klein T., Kotler I. Process Injection Techniques: Gotta Catch Them All. *Black Hat USA*. Las Vegas, 2019.

[135] Kolter J. Z., Maloof M. A. Learning to Detect and Classify Malicious Executables in the Wild. *Journal of Machine Learning Research*. 2006. Vol. 7. P. 2721–2744.

[136] Kordy B., Piètre-Cambacédès L., Schweitzer P. DAG-based Attack and Defense Modeling: Don't Miss the Forest for the Attack Trees. *Computer Science Review*. 2014. Vol. 13–14. P. 1–38. DOI: 10.1016/j.cosrev.2014.07.001.

[137] Kornblum J. Identifying Almost Identical Files Using Context Triggered Piecewise Hashing. *Digital Investigation*. 2006. Vol. 3 (Supplement). P. 91–97. DOI: 10.1016/j.diin.2006.06.015.

[138] Kumar S., et al. An Emerging Threat: Fileless Malware — A Survey and Research Challenges. *Cybersecurity*. 2020. Vol. 3. Art. 1.

[139] Lamb C., Zacchiroli S. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software*. 2022. Vol. 39, № 2. P. 62–70.

[140] Langner R. Stuxnet: Dissecting a Cyberwarfare Weapon. *IEEE Security & Privacy*. 2011. Vol. 9, № 3. P. 49–51. DOI: 10.1109/MSP.2011.67.

[141] Lee R. M., Bianco D. The Hunting Maturity Model. SANS Institute / Threat Hunting Project, 2016.

[142] Lee R. M., Lee R. The Who, What, Where, When, Why and How of Effective Threat Hunting. SANS Institute Reading Room, 2016.

[143] Li M., Vitányi P. An Introduction to Kolmogorov Complexity and Its Applications. 3rd ed. New York: Springer, 2008. 790 p.

[144] Ligh M. H., Case A., Levy J., Walters A. The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux, and Mac Memory. Indianapolis: Wiley, 2014. 912 p.

[145] Lindorfer M., Kolbitsch C., Comparetti P. M. Detecting Environment-Sensitive Malware. *Recent Advances in Intrusion Detection (RAID 2011)*. LNCS Vol. 6961. Berlin: Springer, 2011. P. 338–357.

[146] LOLBAS Project. Living Off The Land Binaries, Scripts and Libraries. URL: https://lolbas-project.github.io/.

[147] Lyda R., Hamrock J. Using Entropy Analysis to Find Encrypted and Packed Malware. *IEEE Security & Privacy*. 2007. Vol. 5, No. 2. P. 40–45. DOI: 10.1109/MSP.2007.48.

[148] Mandiant. APT1: Exposing One of China's Cyber Espionage Units. Mandiant Intelligence Center Report, 2013.

[149] Mandiant. Tracking Malware with Import Hashing (imphash). Mandiant Threat Research, 2014.

[150] Matrosov A., Rodionov E., Bratus S. Rootkits and Bootkits: Reversing Modern Malware and Next Generation Threats. San Francisco: No Starch Press, 2019. 504 p.

[151] McMillan R. Definition: Threat Intelligence. Gartner Research, 2013.

[152] Mell P., Grance T. The NIST Definition of Cloud Computing. NIST Special Publication 800-145. Gaithersburg: NIST, 2011.

[153] Microsoft. Antimalware Scan Interface (AMSI) — Technical Documentation. Microsoft Docs, 2024. URL: https://learn.microsoft.com/windows/win32/amsi/antimalware-scan-interface-portal.

[154] Microsoft. Event Tracing for Windows (ETW) — Technical Documentation. Microsoft Docs, 2024. URL: https://learn.microsoft.com/windows/win32/etw/about-event-tracing.

[155] Microsoft. PE Format Specification (Portable Executable and Common Object File Format). Microsoft Docs, 2024. URL: https://learn.microsoft.com/windows/win32/debug/pe-format.

[156] MISP: Open Source Threat Intelligence Platform & Standards for Threat Information Sharing. URL: https://www.misp-project.org/.

[157] MITRE ATT&CK. Tactic TA0003: Persistence. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0003/.

[158] MITRE ATT&CK. Technique T1055: Process Injection. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1055/.

[159] MITRE ATT&CK®: Adversarial Tactics, Techniques, and Common Knowledge. The MITRE Corporation. URL: https://attack.mitre.org/.

[160] MITRE D3FEND™: A Knowledge Graph of Cybersecurity Countermeasures. The MITRE Corporation. URL: https://d3fend.mitre.org/.

[161] MITRE. Cyber Analytics Repository (CAR). The MITRE Corporation. URL: https://car.mitre.org/.

[162] Murray J. Incident Handler's Handbook. SANS Institute Reading Room, 2015.

[163] Nakashima E., Timberg C. NSA Officials Worried about the Day Its Potent Hacking Tool Would Get Loose. Then It Did. *The Washington Post*. 2017. (аналіз кампанії WannaCry).

[164] National Institute of Standards and Technology. The NIST Cybersecurity Framework (CSF) 2.0. NIST CSWP 29. Gaithersburg: NIST, 2024. DOI: 10.6028/NIST.CSWP.29.

[165] National Security Agency. Defense in Depth: A Practical Strategy for Achieving Information Assurance in Today's Highly Networked Environments. NSA, 2010.

[166] National Telecommunications and Information Administration (NTIA). The Minimum Elements for a Software Bill of Materials (SBOM). U.S. Department of Commerce, 2021.

[167] Newman J., Torres-Arias S., et al. Sigstore: Software Signing for Everybody. *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. New York: ACM, 2022. P. 2353–2367.

[168] NIST. Guide for Conducting Risk Assessments. NIST SP 800-30, Rev. 1. Gaithersburg, MD: NIST, 2012. DOI: 10.6028/NIST.SP.800-30r1.

[169] OASIS. STIX Version 2.1: Structured Threat Information Expression. OASIS Standard, 2021. URL: https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html.

[170] OASIS. TAXII Version 2.1: Trusted Automated Exchange of Intelligence Information. OASIS Standard, 2021.

[171] Ohm M., Plate H., Sykosch A., Meier M. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020)*. LNCS Vol. 12223. Cham: Springer, 2020. P. 23–43.

[172] Oliver J., Cheng C., Chen Y. TLSH — A Locality Sensitive Hash. *2013 Fourth Cybercrime and Trustworthy Computing Workshop*. Sydney: IEEE, 2013. P. 7–13.

[173] Open Information Security Foundation (OISF). Suricata: Open Source Intrusion Detection and Prevention Engine — User Guide. URL: https://docs.suricata.io/.

[174] Open Source Security Foundation (OpenSSF). SLSA: Supply-chain Levels for Software Artifacts. URL: https://slsa.dev/.

[175] OWASP Foundation. OWASP Internet of Things (IoT) Top 10. OWASP Project, 2018.

[176] Patzke T. Sigma: Generic Signature Format for SIEM Systems. SigmaHQ Project Documentation. URL: https://github.com/SigmaHQ/sigma.

[177] Pols P. The Unified Kill Chain: Designing a Unified Kill Chain for analyzing, comparing and defending against cyber attacks: Master thesis. Cyber Security Academy, 2017.

[178] Raff E., Barker J., Sylvester J., Brandon R., Catanzaro B., Nicholas C. Malware Detection by Eating a Whole EXE. *AAAI Workshop on Artificial Intelligence for Cyber Security*. 2018.

[179] Regulation (EU) 2016/679 (General Data Protection Regulation). *Official Journal of the EU*. L 119. P. 1–88. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj.

[180] Regulation (EU) 2019/881 on ENISA and on information and communications technology cybersecurity certification (Cybersecurity Act). *Official Journal of the EU*. L 151. P. 15–69. URL: https://eur-lex.europa.eu/eli/reg/2019/881/oj.

[181] Regulation (EU) 2021/887 establishing the European Cybersecurity Industrial, Technology and Research Competence Centre and the Network of National Coordination Centres. *Official Journal of the EU*. L 202. 2021.

[182] Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act). *Official Journal of the EU*. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[183] Rice H. G. Classes of Recursively Enumerable Sets and Their Decision Problems. *Transactions of the American Mathematical Society*. 1953. Vol. 74, No. 2. P. 358–366. DOI: 10.1090/S0002-9947-1953-0053041-6.

[184] Rid T., Buchanan B. Attributing Cyber Attacks. *Journal of Strategic Studies*. 2015. Vol. 38, № 1–2. P. 4–37. DOI: 10.1080/01402390.2014.977382.

[185] Roesch M. Snort — Lightweight Intrusion Detection for Networks. *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*. USENIX Association, 1999. P. 229–238.

[186] Ross R., Pillitteri V., Graubart R., Bodeau D., McQuaid R. Developing Cyber-Resilient Systems: A Systems Security Engineering Approach. NIST Special Publication 800-160, Vol. 2, Rev. 1. Gaithersburg: NIST, 2021.

[187] Royal P., Halpin M., Dagon D., Edmonds R., Lee W. PolyUnpack: Automating the Hidden-Code Extraction of Unpack-Executing Malware. *22nd Annual Computer Security Applications Conference (ACSAC)*. Miami Beach: IEEE, 2006. P. 289–300.

[188] Russinovich M., Garnier T. System Monitor (Sysmon) — Windows Sysinternals Documentation. Microsoft, 2024. URL: https://learn.microsoft.com/sysinternals/downloads/sysmon.

[189] Russinovich M., Solomon D., Ionescu A., Allievi A. Windows Internals. 7th ed. Parts 1–2. Redmond: Microsoft Press, 2017–2021.

[190] Saltzer J. H., Schroeder M. D. The Protection of Information in Computer Systems. *Proceedings of the IEEE*. 1975. Vol. 63, № 9. P. 1278–1308.

[191] Saxe J., Berlin K. Deep Neural Network Based Malware Detection Using Two Dimensional Binary Program Features. *2015 10th International Conference on Malicious and Unwanted Software (MALWARE)*. Fajardo: IEEE, 2015. P. 11–20.

[192] Schneier B. Attack Trees. *Dr. Dobb's Journal*. 1999. Vol. 24, No. 12. P. 21–29.

[193] Schrittwieser E., Katzenbeisser S., Kinder J., Merzdovnik G., Weippl E. Protecting Software through Obfuscation: Can It Keep Pace with Progress in Code Analysis? *ACM Computing Surveys*. 2016. Vol. 49, № 1. Art. 4. P. 1–37.

[194] Schultz M. G., Eskin E., Zadok E., Stolfo S. J. Data Mining Methods for Detection of New Malicious Executables. *2001 IEEE Symposium on Security and Privacy*. Oakland: IEEE, 2001. P. 38–49.

[195] Second Additional Protocol to the Convention on Cybercrime on enhanced co-operation and disclosure of electronic evidence. CETS No. 224. Strasbourg: Council of Europe, 2022.

[196] Sedgewick A., Souppaya M., Scarfone K. Guide to Application Whitelisting. NIST Special Publication 800-167. Gaithersburg: NIST, 2015. DOI: 10.6028/NIST.SP.800-167.

[197] Selçuk A. A., Orhan F., Batur B. Undecidable Problems in Malware Analysis. *2017 12th International Conference for Internet Technology and Secured Transactions (ICITST)*. Cambridge: IEEE, 2017. P. 494–497.

[198] Shannon C. E. A Mathematical Theory of Communication. *Bell System Technical Journal*. 1948. Vol. 27, No. 3. P. 379–423.

[199] Sikorski M., Honig A. Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software. San Francisco: No Starch Press, 2012. 800 p.

[200] Slowik J., Hultquist J. et al. Industroyer2: Nozomi Networks Labs Analyses the IEC 104 Payload: technical report. 2022. URL: https://cert.gov.ua/article/39518.

[201] Smiliotopoulos C., Kambourakis G., Barbatsalou K. On the Detection of Lateral Movement through Supervised Machine Learning and an Open-Source Tool to Create Turnkey Datasets. *International Journal of Information Security*. 2023. Vol. 22. P. 1893–1919.

[202] Sommer R., Paxson V. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. *2010 IEEE Symposium on Security and Privacy*. Oakland: IEEE, 2010. P. 305–316.

[203] Souppaya M., Morello J., Scarfone K. Application Container Security Guide. NIST Special Publication 800-190. Gaithersburg: NIST, 2017.

[204] Souppaya M., Scarfone K. Guide for Cybersecurity Event Recovery. NIST Special Publication 800-184. Gaithersburg: NIST, 2016.

[205] Souppaya M., Scarfone K. Guide to Malware Incident Prevention and Handling for Desktops and Laptops. NIST Special Publication 800-83, Revision 1. Gaithersburg: NIST, 2013. DOI: 10.6028/NIST.SP.800-83r1.

[206] Spafford E. H. The Internet Worm Program: An Analysis. *ACM SIGCOMM Computer Communication Review*. 1989. Vol. 19, No. 1. P. 17–57. DOI: 10.1145/66093.66095.

[207] Spinellis D. Reliable Identification of Bounded-length Viruses is NP-complete. *IEEE Transactions on Information Theory*. 2003. Vol. 49, No. 1. P. 280–284. DOI: 10.1109/TIT.2002.806137.

[208] Sqrrl Team. A Framework for Cyber Threat Hunting. Sqrrl Data, 2016.

[209] Staniford S., Paxson V., Weaver N. How to 0wn the Internet in Your Spare Time. *Proceedings of the 11th USENIX Security Symposium*. USENIX Association, 2002. P. 149–167.

[210] Stillions R. The DML Model (Detection Maturity Level). ryan.stillions blog, 2014.

[211] Stoll C. The Cuckoo's Egg: Tracking a Spy Through the Maze of Computer Espionage. New York: Doubleday, 1989. 326 p.

[212] Stouffer K., Pillitteri V., Lightman S., Abrams M., Hahn A. Guide to Industrial Control Systems (ICS) Security. NIST Special Publication 800-82, Revision 2. Gaithersburg: NIST, 2015. DOI: 10.6028/NIST.SP.800-82r2.

[213] Strom B. E. et al. MITRE ATT&CK: Design and Philosophy. MITRE Technical Report MTR190121. McLean: The MITRE Corporation, 2020.

[214] Szor P. The Art of Computer Virus Research and Defense. Boston: Addison-Wesley, 2005. 744 p.

[215] TAXII Version 2.1: Trusted Automated Exchange of Intelligence Information. OASIS Standard, 2021. URL: https://docs.oasis-open.org/cti/taxii/v2.1/.

[216] The Volatility Foundation. The Volatility Framework: Volatile Memory Artifact Extraction Utility Framework. URL: https://www.volatilityfoundation.org/.

[217] Tool Interface Standard (TIS). Executable and Linkable Format (ELF) Specification. Version 1.2. TIS Committee, 1995.

[218] Torres-Arias S., Afzali H., Kuppusamy T. K., Curtmola R., Cappos J. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. *Proceedings of the 28th USENIX Security Symposium*. USENIX Association, 2019. P. 1393–1410.

[219] U.S. Department of Energy. Cybersecurity Capability Maturity Model (C2M2), Version 2.1. Washington: DOE, 2022.

[220] Ucci D., Aniello L., Baldoni R. Survey of Machine Learning Techniques for Malware Analysis. *Computers & Security*. 2019. Vol. 81. P. 123–147. DOI: 10.1016/j.cose.2018.11.001.

[221] Ugarte-Pedrero X., Balzarotti D., Santos I., Bringas P. G. SoK: Deep Packer Inspection — A Longitudinal Study of the Complexity of Run-Time Packers. *2015 IEEE Symposium on Security and Privacy*. San Jose: IEEE, 2015. P. 659–673. DOI: 10.1109/SP.2015.46.

[222] Verizon. Data Breach Investigations Report (DBIR). New York: Verizon Business, 2024. URL: https://www.verizon.com/business/resources/reports/dbir/.

[223] Verizon. Data Breach Investigations Report (DBIR). Verizon Business, 2024. (щорічний аналітичний звіт про чинник людини в інцидентах).

[224] Virvilis N., Gritzalis D. The Big Four — What We Did Wrong in Advanced Persistent Threat Detection? *2013 International Conference on Availability, Reliability and Security*. Regensburg: IEEE, 2013. P. 248–254.

[225] von Neumann J. Theory of Self-Reproducing Automata / ed. A. W. Burks. Urbana: University of Illinois Press, 1966.

[226] Wagner C., Dulaunoy A., Wagener G., Iklody A. MISP — The Design and Implementation of a Collaborative Threat Intelligence Sharing Platform. *Proceedings of the 2016 ACM on Workshop on Information Sharing and Collaborative Security (WISCS '16)*. New York: ACM, 2016. P. 49–56.

[227] Willems C., Holz T., Freiling F. Toward Automated Dynamic Malware Analysis Using CWSandbox. *IEEE Security & Privacy*. 2007. Vol. 5, № 2. P. 32–39.

[228] Yadegari B., Johannesmeyer B., Whitely B., Debray S. A Generic Approach to Automatic Deobfuscation of Executable Code. *2015 IEEE Symposium on Security and Privacy*. San Jose: IEEE, 2015. P. 674–691.

[229] You I., Yim K. Malware Obfuscation Techniques: A Brief Survey. *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*. Fukuoka: IEEE, 2010. P. 297–300.

[230] Zetter K. Countdown to Zero Day: Stuxnet and the Launch of the World's First Digital Weapon. New York: Crown, 2014. 448 p.

[231] Zimmerman C. Ten Strategies of a World-Class Cybersecurity Operations Center. McLean: MITRE Corporation, 2014. 353 p.

[232] Zou C. C., Towsley D., Gong W. Email Worm Modeling and Defense. *Proceedings of the 13th International Conference on Computer Communications and Networks (ICCCN)*. IEEE, 2004. P. 409–414.

[233] Álvarez V. M. YARA Documentation: Writing YARA rules. Release 4.x. VirusTotal. URL: https://yara.readthedocs.io/.

[234] Álvarez V. M. YARA: The pattern matching swiss knife for malware researchers. URL: https://virustotal.github.io/yara/.

Додатки

Додаток А. Короткий словник ключових термінів

Злоякісний програмний код (ЗПК). Програмне забезпечення, створене для несанкціонованих, шкідливих дій у комп'ютерній системі.

Сигнатура. Формалізований опис ознак, наявність яких свідчить про належність зразка до відомої загрози.

Статичний аналіз. Дослідження зразка без його виконання.

Динамічний аналіз. Дослідження зразка під час виконання в контрольованому середовищі.

Пісочниця. Ізольоване інструментоване середовище для виконання й спостереження зразка.

Обфускація. Перетворення коду, що зберігає функцію, але ускладнює розуміння.

Пакування. Перетворення виконуваного файлу на стиснений/зашифрований образ із розпакувальником.

Поліморфізм. Відтворення коду в різних байтових формах зі збереженням функції.

Індикатор компрометації (IoC). Спостережувана ознака ймовірної компрометації.

Розвідка загроз. Засноване на доказах знання про загрози, придатне для прийняття рішень.

APT. Цілеспрямована стала загроза — ресурсоване, наполегливе, адаптивне угруповання.

Полювання за загрозами. Проактивний, керований гіпотезами пошук прихованих загроз.

Час присутності (dwell time). Проміжок від компрометації до виявлення.

Кіберстійкість. Здатність витримувати атаки, обмежувати вплив і швидко відновлюватися.

Ешелонований захист. Поєднання кількох незалежних рубежів, що компенсують слабкості один одного.

Додаток Б. Зіставлення методів виявлення

Метод	Сильні риси	Обмеження
Сигнатурне виявлення	точність, швидкість для відомого	безсиле проти нового, пакування
Евристичний аналіз	виявляє невідоме	хибні тривоги
Поведінкове виявлення	інваріантне до запису коду	потребує виконання, телеметрії
Машинне навчання	узагальнення, масштаб	дані, дрейф, змагальність
Виявлення за подібністю	працює із сімействами	вразливе до обфускації
Полювання за загрозами	знаходить приховане, APT	трудомісткість, кваліфікація

Додаток В. Контрольний перелік зрілості протидії ЗПК

1. Визначено модель загроз і пріоритетні активи (розділ 4).
2. Забезпечено профілактику: гартування, контроль застосунків, найменші привілеї, оновлення (розділ 10).
3. Розгорнуто багаторівневе виявлення: сигнатури, поведінка, машинне навчання (розділи 6–11).
4. Налагоджено централізовану телеметрію кінцевих точок і мережі (розділ 10).
5. Інтегровано розвідку загроз і забезпечено обмін індикаторами (розділ 12).
6. Впроваджено проактивне полювання за загрозами (розділ 13).
7. Забезпечено цілісність ланцюга постачання й прозорість складу ПЗ (розділ 15).
8. Підготовлено план реагування на інциденти й форензичного збору (розділ 16).
9. Сформовано культуру безпеки й навчання персоналу (розділ 17).
10. Запроваджено метрики, модель зрілості й цикл безперервного вдосконалення (розділ 17).